

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

MASTER THESIS



论文题目 基于 RISC-V 的微内核虚拟化方案研究

学科专业 计算机科学与技术

学 号

作者姓名

指导教师

学院 信息与软件工程学院（示范性软件学院）

分类号 _____ 密级 _____

UDC^{注1} _____

学 位 论 文

基于 RISC-V 的微内核虚拟化方案研究

(题名和副题名)

(作者姓名)

指导教师 _____

电子科技大学 成都

(姓名、职称、单位名称)

申请学位级别 _____ 学科专业 计算机科学与技术

提交论文日期 _____ 论文答辩日期 _____

学位授予单位和日期 _____

答辩委员会主席 _____

评阅人 _____

注 1: 注明《国际十进分类法 UDC》的类号。

Research on RISC-V-Based Microkernel Virtualization Architecture

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline **Computer Science and Technology**

Student ID

Author

Supervisor

School **School of Information and Software Engineering**

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知,除了文中特别加以标注和致谢的地方外,论文中不包含其他人已经发表或撰写过的研究成果,也不包含为获得电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

作者签名: _____ 日期: _____ 年 _____ 月 _____ 日

论文使用授权

本学位论文作者完全了解电子科技大学有关保留、使用学位论文的规定,同意学校有权保留并向国家有关部门或机构送交论文的复印件和数字文档,允许论文被查阅。本人授权电子科技大学可以将学位论文的全部或部分内容编入有关数据库进行检索及下载,可以采用影印、扫描等复制手段保存、汇编学位论文。

(涉密的学位论文须按照国家及学校相关规定管理,在解密后适用于本授权。)

作者签名: _____ 导师签名: _____

日期: _____ 年 _____ 月 _____ 日

摘要

随着边缘异构计算的发展，嵌入式系统面临数据安全与隐私泄露挑战。支持机密虚拟机的可信执行环境技术作为保障敏感数据与模型资产的核心技术，已成为机密计算领域的研究热点。然而，现有商用机密虚拟机架构多受限于闭源体系，且传统基于 C/C++ 构建的底层安全固件易受内存破坏漏洞的威胁。此外，在资源受限的端侧环境中，兼顾隔离安全性与异构设备的复用效率仍是当前面临的技术挑战。针对上述问题，本文基于开源 RISC-V 指令集架构，设计并实现了一套微内核机密虚拟化系统 EasyAda-trust。本文的主要研究工作如下：

1. 提出了一种基于 Rust 内存安全特性的底层可信固件架构。结合 Rust 所有权机制重构页分配器，通过编译期静态检查消除二次释放等内存隐患；同时，将 RISC-V 物理内存保护机制抽象为统一的内存保护原语，实现机密虚拟机与不可信组件间的硬件级隔离，构建系统底层信任根。

2. 构建了支持机密虚拟机全生命周期的软硬协同虚拟化机制。实现了物理资源调度与安全鉴权的解耦。不可信虚拟机监视器负责宏观资源分配，而安全区域构建、二阶段页表映射及机密执行流的上下文切换等敏感操作，均由 TEE 安全管理器代理执行。此外，结合本地证明机制实现了机密虚拟机与硬件环境的双向信任度量。

3. 设计了防范 DMA 攻击的端侧机密异构计算环境。在 EasyAda-trust 中引入 RISC-V I/O 物理内存保护硬件扩展支持，在总线级实施外设动态鉴权与细粒度访问控制。在此基础上，构建加速器管理机密虚拟机，利用 I/O 半虚拟化与安全共享内存映射技术建立跨域信道，使机密应用能够旁路不可信软件栈，实现底层硬件加速器在机密环境内的零拷贝明文复用。

最后，在 QEMU 仿真环境下对系统开展了多维度量化评估。实验结果表明，EasyAda-SM 能够以较小的代价兼容现有 RISC-V TEE 方案，其核心计算（CoreMark 与 RV8）性能损耗低于 2%，验证 EasyAda-SM 的可扩展性；得益于静态内存预分配机制，EasyAda-trust 中机密虚拟机在系统时钟与访存等核心原语上的执行开销与原生系统相近；同时，安全直通架构有效保障了异构 I/O 的吞吐效率。评估结果表明，本文方案在内存安全性、异构设备复用率与总体计算效能之间实现了合理的架构折中，为端侧机密计算的部署提供了系统级解决方案。

关键词：Rust, RISC-V, 微内核虚拟化, 机密计算, 可信异构计算

ABSTRACT

With the rapid development of edge heterogeneous computing, embedded systems face severe challenges regarding data security and privacy leakage. As a core technology to safeguard sensitive data and high-value model assets, the Confidential Virtual Machine (CVM) has become a research hotspot in the field of confidential computing. However, existing commercial CVM architectures are mostly constrained by closed-source ecological barriers, and traditional underlying security firmware built with C/C++ suffers from critical memory safety vulnerabilities. Furthermore, achieving a balance between high-security isolation and the efficient reuse of heterogeneous devices in resource-constrained edge environments remains an urgent technical challenge. To address these pain points, this thesis designs and implements EasyAda-trust, a microkernel-based confidential virtualization system based on the open-source RISC-V instruction set architecture. The main research work and innovations of this thesis are as follows:

1. Proposed an underlying trusted firmware architecture based on the memory safety features of Rust. By deeply integrating Rust's ownership mechanism to reconstruct the page allocator, memory vulnerabilities such as Double-Free are statically eliminated at compile time. Meanwhile, the RISC-V Physical Memory Protection (PMP) mechanism is abstracted into a unified security primitive, constructing a hardware-level strong isolation boundary between the and untrusted components, thereby establishing the underlying root of trust of the system.

2. Constructed a hardware-software co-designed virtualization management mechanism supporting the full lifecycle of CVMs. The responsibility decoupling of physical resource scheduling and security authentication is innovatively implemented. The untrusted Hypervisor is solely responsible for macroscopic resource allocation, while core sensitive operations such as secure region construction, two-stage page table mapping, and context switching of confidential execution flows are mediated by the Trusted Security Manager (TSM). In addition, a local attestation mechanism is introduced to realize bidirectional trusted measurement between the CVM and the underlying hardware.

3. Designed an edge confidential heterogeneous computing environment to prevent DMA attacks. By introducing the RISC-V IOPMP hardware extension, peripheral dynamic authentication and fine-grained access control are enforced at the bus level. On this

basis, a dedicated "Accelerator Management CVM" is designed. Relying on I/O paravirtualization and secure shared memory mapping technologies to bridge cross-domain channels, confidential applications can bypass the untrusted software stack, successfully achieving the zero-copy plaintext utilization of underlying hardware accelerators within the confidential environment.

Finally, multi-dimensional quantitative evaluations of the system were conducted in the QEMU simulation environment. The experimental results show that EasyAda-SM can be compatible with the existing RISC-V TEE ecosystem with extremely low overhead, and the performance degradation of core computing (CoreMark and RV8) is strictly controlled within 2%, verifying its excellent scalability. Thanks to the static memory pre-allocation mechanism, the execution overhead of the CVM on core primitives such as system clock and memory access approaches that of the native system. Meanwhile, the secure pass-through architecture effectively guarantees the throughput efficiency of heterogeneous I/O. In summary, the proposed scheme achieves a reasonable architectural trade-off among memory safety, heterogeneous device reuse rate, and overall computing efficiency, providing an effective system-level solution for the generalized deployment of edge confidential computing.

Keywords: Rust Language; RISC-V; Microkernel Virtualization; Confidential Computing; Trust Heterogeneous Computing

目 录

第一章 绪论	1
1.1 研究工作的背景与意义	1
1.2 国内外研究背景与现状	3
1.2.1 虚拟化技术研究现状	3
1.2.2 虚拟化与 TEE 融合架构及演进现状	5
1.2.3 异构计算环境下加速器 TEE 安全隔离与防护技术研究	7
1.3 本文主要贡献和创新	8
1.4 论文结构	9
第二章 相关技术理论概论	11
2.1 RISC-V 背景知识	11
2.1.1 RISC-V 特权级介绍	11
2.1.2 RISC-V 中断与异常处理机制介绍	12
2.1.3 RISC-V 内存保护机制介绍	13
2.1.4 RISC-V 硬件虚拟化扩展 (H-extension) 介绍	14
2.2 可信执行环境 (TEE) 相关技术介绍	16
2.2.1 面向异构计算环境的 TEE	18
2.3 微内核相关知识介绍	19
2.3.1 微内核操作系统 EasyAda 介绍	19
2.3.2 微内核虚拟化方案 EasyAda-Hypervisor 介绍	20
2.4 Rust 相关知识介绍	22
2.5 本章小结	23
第三章 基于 RISC-V 的微内核虚拟化方案设计	24
3.1 使用场景与设计目标分析	24
3.2 EasyAda-trust 工作流	25
3.3 威胁模型	25
3.4 可信微内核虚拟化方案设计	26
3.4.1 总体设计	26
3.4.2 内存安全的模块化安全监视器设计	30
3.4.3 安全特性设计	31
3.5 本章小结	42

第四章 内存安全的模块化安全监视器实现	43
4.1 内存安全固件	43
4.2 安全区域和内存管理层实现	43
4.2.1 内存管理原语	44
4.2.2 内存保护原语	48
4.2.3 安全内存管理原语	49
4.3 面向机密虚拟机的飞地管理模块实现	52
4.3.1 飞地间内存安全	53
4.3.2 安全启动	54
4.3.3 中断和异常安全	55
4.3.4 IOPMP 管理	56
4.4 本章小结	58
第五章 基于微内核的可信虚拟化方案实现	59
5.1 安全启动流程实现	59
5.2 安全 CPU 虚拟化实现	61
5.2.1 安全上下文管理	61
5.2.2 安全异常处理	64
5.3 安全内存虚拟化实现	65
5.3.1 基于 TSM 接口的安全区域生命周期管理	66
5.4 安全 I/O 虚拟化实现	67
5.4.1 安全直通设备生命周期管理	68
5.4.2 基于 IOPMP 的设备侧防 DMA 攻击隔离	68
5.4.3 I/O 数据平面安全与端到端保护	69
5.5 机密异构计算环境实现	70
5.5.1 硬件加速器的保护	70
5.5.2 加速器管理机密虚拟机初始化	70
5.5.3 建立信道	72
5.6 本章小结	72
第六章 系统测试与分析	74
6.1 测试目标	74
6.2 测试硬件环境	74
6.3 EasyAda-SM 可扩展性测试	74

6.4 EasyAda-trust 测试	77
6.4.1 系统启动测试	77
6.4.2 机密虚拟机安全启动测试	78
6.4.3 机密虚拟机性能测试	80
6.5 防御链路分析	82
6.5.1 恶意设备提权与 DMA 攻击	83
6.5.2 二次释放攻击	83
6.6 本章小结	84
第七章 总结和展望	85
7.1 全文总结	85
7.2 典型场景：基于机密虚拟机的自动驾驶混合架构	86
7.3 后续工作展望	87
致 谢	89
参考文献	90
附录 A 核心伪代码	94
A.1 页分配器	94
A.2 IOPMP 动态配置	94
攻读硕士学位期间取得的成果	95

图目录

图 1-1 虚拟化技术分类	4
图 2-1 未引入硬件虚拟化扩展时 RISC-V 系统架构	11
图 2-2 引入硬件虚拟化扩展后的 RISC-V 系统架构	15
图 2-3 基于系统抽象层级与软件执行模型的 TEE 分类	16
图 2-4 微内核操作系统 EasyAda 总体架构	20
图 2-5 EasyAda-Hypervisor 总体架构	21
图 3-1 非机密隔离域架构	27
图 3-2 机密执行域架构	28
图 3-3 EasyAda-trust 生命周期状态机	29
图 3-4 EasyAda-SM 总体架构	30
图 3-5 EasyAda-SM 的内存管理模式	32
图 3-6 机密虚拟机二阶段页表初始化	35
图 3-7 机密虚拟机的 I/O 数据保护机制	39
图 3-8 硬件加速器管理机密虚拟机设计	41
图 4-1 安全区域和内存管理层结构图	44
图 4-2 PageAllocator 结构	46
图 4-3 PMP 槽结构	49
图 4-4 PMP 运行时配置	51
图 4-5 不同类型飞地与区域类型和分配器的关联关系	52
图 4-6 机密虚拟机管理器结构	53
图 4-7 EasyAda -安全监视器中非安全/安全设备的 IOPMP 管理	57
图 5-1 EasyAda-SM 签名验证流程	59
图 5-2 机密虚拟机验证信息封装流程	60
图 5-3 EasyAda-trust 中双向验证流程	62
图 5-4 EasyAda-trust 下 Hart 分区示意图	63
图 5-5 引入安全设备后的机密虚拟机安全内存视图	71
图 6-1 原生环境与 EasyAda-SM 环境下 Coremark 的对比测试	75

图 6-2	原生环境与 EasyAda-SM 环境下 RV8 benchmark 测试结果对比	76
图 6-3	SBI 固件初始化	78
图 6-4	EasyAda-SM 初始化测试	79
图 6-5	可信机密虚拟机在可信平台上启动测试	79
图 6-6	不可信机密虚拟机在可信平台上启动测试	80
图 6-7	可信机密虚拟机在不可信平台上启动测试	80
图 6-8	LEBench 测试结果	81
图 6-9	安全设备直通架构与半虚拟化机制（不可信共享内存）下的磁盘 I/O 对比测试结果	82
图 7-1	基于 EasyAda-trust 的自动驾驶混合架构	86

缩略词表

英文缩写	英文全称	中文全称
AES	Advanced Encryption Standard	高级加密标准
AIA	Advanced Interrupt Architecture	高级中断架构
API	Application Programming Interface	应用程序编程接口
ASID	Address Space Identifier	地址空间标识符
CoVE	Confidential Virtual-Machine Extension	机密虚拟机扩展
CSR	Control and Status Register	控制与状态寄存器
CVM	Confidential Virtual Machine	机密虚拟机
CXL	Compute Express Link	计算扩展链接
DMA	Direct Memory Access	直接内存访问
DoS	Denial of Service	拒绝服务
EPC	Enclave Page Cache	受保护内存
FDT	Flattened Device Tree	扁平设备树
FFI	Foreign Function Interface	外部函数接口
FPGA	Field Programmable Gate Array	现场可编程逻辑门阵列
GPA	Guest Physical Address	客户物理地址
GPC	Granule Protection Check	粒度保护检查
GPT	Granule Protection Table	受护页表
GPU	Graphics Processing Unit	图形处理器
HAL	Hardware Abstraction Layer	硬件抽象层
Hart	Hardware Thread	硬件线程
HPA	Host Physical Address	宿主物理地址

英文缩写	英文全称	中文全称
HS-Mode	Hypervisor-extended Supervisor Mode	Hypervisor 扩展 监管者模式
IOMMU	Input/Output Memory Management Unit	输入/输出内存管 理单元
IOPMP	I/O Physical Memory Protection	I/O 物理内存保护
IoT	Internet of Things	物联网
IP	Intellectual Property	知识产权
ISA	Instruction Set Architecture	指令集架构
M-Mode	Machine Mode	机器模式
MCS	Mixed-Criticality System	混合关键系统
MCU	Microcontroller Unit	微控制器
MD	Memory Domain	内存域
MEE	Memory Encryption Engine	内存加密引擎
MMIO	Memory-Mapped I/O	内存映射 I/O
MML	Machine Mode Lock	机器模式锁定
MMU	Memory Management Unit	内存管理单元
MMWP	Machine Mode Whitelist Policy	机器模式白名单 策略
NAPOT	Naturally Aligned Power-of-Two	自然对齐的 2 的 幂次匹配模式
NPU	Neural Processing Unit	神经网络处理器
NW	Normal World	普通世界
OS	Operating System	操作系统
PCIe	Peripheral Component Interconnect Express	高速串行计算机 扩展总线标准
PCR	Platform Configuration Register	平台配置寄存器
PMP	Physical Memory Protection	物理内存保护
RISC	Reduced Instruction Set Computer	精简指令集

英文缩写	英文全称	中文全称
RLB	Rule Lock Bypass	规则锁定旁路
RMP	Reverse Map Table	反向映射表
RoT	Root of Trust	信任根
RPC	Remote Procedure Call	远程过程调用
RTOS	Real-Time Operating System	实时操作系统
S-Mode	Supervisor Mode	监管者模式
SBI	Supervisor Binary Interface	监管者二进制接口
SGX	Software Guard Extensions	软件防护扩展
SID	Source Identifier	设备唯一标识
SMEPMP	Supervisor Mode Execution Prevention and Machine-mode Physical Memory Protection	增强的物理内存保护
SoC	System on Chip	系统级芯片
SW	Secure World	安全世界
TA	Trusted Application	可信应用
TAP	TEE Attest Payload	TEE 证明载荷
TCB	Trusted Computing Base	可信基
TEE	Trusted Execution Environment	可信执行环境
TPM	Trusted Platform Module	可信平台模块
TSM	TEE Security Manager	TEE 安全管理器
U-Mode	User Mode	用户模式
VMM	Virtual Machine Monitor	虚拟机监视器
VS-Mode	Virtual Supervisor Mode	虚拟监管者模式
VU-Mode	Virtual User Mode	虚拟用户模式

第一章 绪论

1.1 研究工作的背景与意义

近年来，物联网（Internet of Things, IoT）和嵌入式技术的迅速发展正在推动各行业向智能化方向升级。物联网通过将大量边缘设备纳入网络管理，实现对边缘数据的收集、传输和处理，并且已在智能家居、工业自动化、智慧城市和医疗健康等领域得到了广泛应用。作为物联网的核心，嵌入式系统在设备控制、传感器数据采集与监测、实时数据处理等方面发挥着关键作用，为各种智能设备提供技术支撑。

然而，随着物联网设备功能的不断扩展，嵌入式技术正面临更加复杂的应用需求和安全问题：（1）在场景需求方面，多样化的应用场景对嵌入式系统的实时性、可靠性和资源利用效率提出了差异化要求。在关键安全场景中，系统需要确保高实时性、可靠性和安全性；而消费电子产品则更关注低成本和高能效。传统嵌入式系统通常只运行单一操作系统，在需要兼顾多种功能需求时，往往依赖物理硬件的堆叠，从而导致系统成本、复杂性和功耗的显著提升；而基于 OpenAMP^[1]的多操作系统部署方案虽然可以在一定程度上兼顾多种功能需求，但无法很好地确保操作系统间的资源隔离与安全性。（2）在安全威胁方面，传统嵌入式系统通常针对特定场景高度定制化，软硬件高度固化，因此攻击面较小。然而，随着边缘计算和物联网的发展，嵌入式设备需要承担更复杂的计算任务，并依赖互联网实现设备控制和数据处理。系统代码量的膨胀和开放的网络环境导致嵌入式系统的攻击面急剧增大，对其安全性提出了更高的要求。

为了应对嵌入式硬件资源丰富化和应用场景复杂化的趋势，现代嵌入式系统正在从单一操作系统架构演变为混合关键系统（Mixed-Criticality System, MCS），以提升系统资源利用率和安全性，并降低重复开发成本。现代 MCS 通过引入虚拟化技术对物理硬件进行抽象，实现在同一硬件平台上部署多个操作系统，从而应对不同应用场景的差异化需求；并且通过隔离不同运行环境间的内存、CPU 和 I/O 等资源，增强了系统的稳定性和可靠性。虽然虚拟化技术以虚拟机监视器（Hypervisor）作为可信基（Trusted Computing Base, TCB），可以为不同的虚拟机划分出相互隔离的执行环境，但通用 Hypervisor 通常具有复杂的执行逻辑和庞大的代码量（例如 Linux KVM 大量复用了 Linux 的现有代码和功能），导致其攻击面较大，极易遭受攻击^[2]。因此，Hypervisor 虽然提高了系统资源的利用率，却难以有效保护用户的隐私数据。

为了确保数据得到有效保护，用于隔离敏感代码与数据的可信执行环境（Trusted Execution Environment, TEE）在过去十年间得到了广泛普及。目前，所有主流 CPU 架构均已推出或支持各自的 TEE 解决方案（如 ARM 的 TrustZone^[3]、Intel 的 SGX^[4]、AMD 的 SEV^[5]，以及 RISC-V 的 Penglai^[6] 和 Keystone^[7] 等），以提供一个安全的执行环境，该环境通常被称为飞地（Enclave）或安全世界（Secure World）。TEE 的应用场景广泛，可部署在云服务器、移动设备、物联网设备、传感器以及硬件令牌等环境中。

目前，嵌入式场景下基于可信操作系统的 TEE 方案（例如基于 ARM TrustZone 扩展与 GP-TEE^[8] 规范的实现体系）虽然应用广泛，但仍存在部分缺陷：（1）闭源性限制：ARM 指令集的闭源特性导致开发者难以根据具体需求进行精细化定制；（2）扩展性不足：ARM TrustZone 将运行环境划分为安全世界（Secure World, SW）和普通世界（Normal World, NW），所有的可信应用（Trusted Application, TA）均共享同一个安全世界，可信应用间的资源隔离主要依赖 TEE 操作系统（TEE Operating System）。这导致可信基较大，容易引入代码漏洞或被攻破；并且两个世界间的资源难以在运行时动态调整，系统灵活性较差；（3）开发成本高昂：在该框架下，厂商必须开发专用的 TEE-OS 并基于此系统开发可信应用，开发门槛和难度较高。边缘计算的发展进一步放大了这一成本，端侧 AI 需要处理大量用户隐私数据，必须得到严格保护。然而，将庞大的 AI 模型完整移植到资源受限且运行专有 TEE-OS 的 TEE 中，或者仅使用 TrustZone 保护关键计算路径并通过复杂的软硬件逻辑保护上下文及 I/O 数据，都极大地增加了可信应用的开发难度。

为了保证可信应用安全性的同时充分复用现有软硬件生态，从而降低 TEE 的准入门槛与厂商的开发成本，亟需将虚拟化技术的灵活抽象能力与 TEE 的硬件级隔离能力相互结合。为此，主要 CPU 厂商均提出了基于虚拟机抽象的混合 TEE 方案如 ARM CCA^[9]、Intel TDX^[10]、RISC-V 机密虚拟机拓展（Confidential Virtual-Machine Extension, CoVE）等。通过将复杂的 Hypervisor 移出可信基，并在 TEE 中直接运行未经修改的机密虚拟机（Confidential Virtual Machine, CVM），使得原有的可信应用能够无缝迁移至 TEE 中，极大地降低了厂商的开发成本。

然而，现有基于虚拟机抽象的 TEE 方案主要面向服务器场景，无法直接扩展至资源和成本受到严格限制的嵌入式场景。此外，基于虚拟机的 TEE 方案由于引入了完整的虚拟机，导致系统整体可信基剧增，显著增加了 TEE 内部的代码漏洞数量和攻击面，严重威胁系统的安全性和稳定性。目前披露的 TEE 代码漏洞中，多数源于非内存安全语言（如 C/C++）编写的程序因不安全的内存访问而引发的内存破坏^[11]。为了尽可能消除系统中的安全隐患，必须严格限制可信基（尤其是运行在最高特权级的安全固件）中的不安全内存访问行为。

Rust^[12] 作为一种内存安全的编程语言，能够通过编译时的严格检查机制避免内存越界等问题，从而兼顾嵌入式场景对可靠性、安全性、实时性和性能的严苛要求。尽管目前已有部分研究^[13, 14] 采用 Rust 实现了 TA 以消除内存漏洞，但采用 Rust 构建底层软件可信基的相关研究工作依然处于空白阶段。

RISC-V 是一种基于经典精简指令集（Reduced Instruction Set Computer, RISC）原理的开源指令集架构（Instruction Set Architecture, ISA）。得益于其开放与模块化的架构特性，RISC-V 能够在保持简洁性与可靠性的同时支持多种硬件平台。系统可根据场景需求集成可选的 RISC-V 标准扩展，甚至实现用户自定义的扩展指令，这使得 RISC-V 能够覆盖从嵌入式微控制器到高性能计算的广泛场景。近年来，RISC-V 在嵌入式领域得到了广泛应用，主流工具链和操作系统均已完善对 RISC-V 的支持。虽然目前 RISC-V 已发布硬件辅助的虚拟化扩展（H 扩展）并提供了支持构建 TEE 方案的硬件基础，但相关的系统级融合研究仍处于起步阶段。

综上所述，结合开源、可扩展的 RISC-V 指令集，实现支持运行机密虚拟机的 TEE 方案，并消除可信基中存在的内存漏洞，对于在确保嵌入式系统实时性与可靠性的同时，提升系统安全性与资源利用率、降低 TEE 开发成本及准入门槛，具有重大的学术价值与工程研究意义。

1.2 国内外研究背景与现状

1.2.1 虚拟化技术研究现状

虚拟化技术的逻辑起源可追溯至 20 世纪 70 年代，其核心本质在于通过构建硬件抽象层实现物理资源与执行环境的解耦。1974 年，Popek 与 Goldberg 提出了虚拟机监视器（Virtual Machine Monitor, VMM）的经典理论框架，界定了合格 VMM 须具备的等效性（Equivalence）、高效性（Efficiency）及资源控制（Resource Control）三大关键属性^[15]。

虚拟化的实现基石在于陷入-模拟（Trap-and-Emulate）机制，该机制深度依赖于指令集中特权指令与敏感指令的正交性逻辑：当硬件架构满足敏感指令集为特权指令集的子集时，即具备原生虚拟化能力。在此架构下，客户机操作系统（Guest OS）执行敏感指令将触发特权级陷阱（Trap），由处于最高特权级的 Hypervisor 捕获异常并实施语义模拟，从而在保障物理资源受控的前提下，实现多虚拟机间的强隔离与资源复用^[16]。根据实现路径及客户 OS 的感知程度，虚拟化技术演进为两种主流范式：全虚拟化通过完全的硬件模拟支持未经修改的客户 OS，具备极佳的兼容性；而半虚拟化则通过超级调用（Hypercall）接口建立 Guest 与 Hypervisor 的协同演进机制，以适度的非侵入式修改换取更高的交互效率。

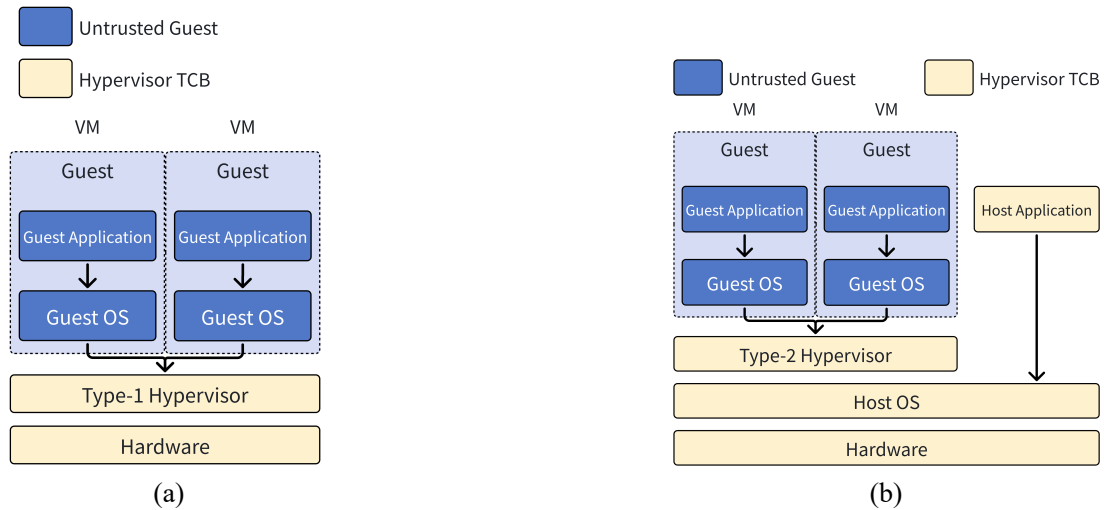


图 1-1 虚拟化技术分类

根据 Hypervisor 与宿主环境的依赖关系，其部署架构可分为 Type-1（裸金属型）与 Type-2（宿主型）两类，如1-1所示。Type-1 架构直接运行于物理硬件之上，拥有对底层资源的绝对控制权，因其访存路径极短且指令处理链路精简，在性能表现与硬件利用率上占据显著优势。相比之下，Type-2 架构寄生于宿主操作系统（Host OS），需跨越宿主 OS 的抽象层间接访问硬件。尽管这种间接调用机制引入了额外的性能损耗，但其能够充分复用 Host OS 成熟的驱动体系与管理工具，展现出更强的环境适配性与部署灵活性。

面向传统云服务场景的虚拟机监视器（如 KVM^[17]、Xen^[18] 等）侧重于大规模租户的资源隔离。尽管已有研究尝试对其进行轻量化改造以向嵌入式场景迁移^[19, 20]，但受限于其宏内核架构下庞大的可信基，仍难以满足嵌入式环境对计算资源受限及严苛实时性的双重要求。为此，学术界与工业界相继提出了 Bao、ACRN、XtratuM 等针对高实时性与低功耗定制优化的轻量级方案^[21-23]。然而，此类方案在实际部署中仍面临资源动态调配率偏低及系统架构可扩展性受限等瓶颈。

在车载与工业控制等高实时性关键场景中，基于微内核的虚拟化技术凭借其极简的特权层级与卓越的系统响应性能，成为突破上述瓶颈的优选路径。依托 seL4 微内核构建的 CAmkES-VMM 框架，通过极致的架构精简与低延迟设计，确立了高实时与低开销需求的理想范式^[24]。同时，以 QNX 为代表的商用微内核方案，通过跨架构的强兼容性与关键任务（Mission-critical）调度策略，在自动驾驶等领域印证了微内核架构在兼顾强实时性与高安全性方面的固有优势^[25]。

综上所述，虚拟化技术已从早期的资源模拟演变为支撑现代复杂系统的核心基底。微内核虚拟化技术通过架构级的解耦与精简，在系统灵活性与执行效率之间取得了较好的平衡，契合现代嵌入式系统的复杂虚拟化需求，并已在严苛的实

时性场景中确立了显著的技术优势地位。

1.2.2 虚拟化与 TEE 融合架构及演进现状

随着机密计算技术的不断发展，如何平衡执行环境的安全性、灵活性与生态兼容性成为了学术界与工业界关注的核心议题。根据系统对信任边界的划分及飞地抽象级别的不同，现代隔离技术主要经历了纯虚拟化方案、基于应用/OS 级抽象的 TEE 方案，以及基于机密虚拟机抽象的软硬融合 TEE 方案三个演进阶段。

1.2.2.1 基于纯虚拟化 TEE 技术研究

主流指令集架构均已引入成熟的硬件辅助虚拟化技术。传统的虚拟化技术侧重于资源复用与系统级隔离，使 Hypervisor 能够有效屏蔽底层异构硬件的差异。为提高系统的安全性，早期研究尝试通过极度精简 Hypervisor 或将其移出可信基来构建安全隔离域。

为优化 Hypervisor 自身的稳健性，Steinberg 等人^[26]借鉴微内核设计理念提出了 NOVA 架构。该方案将资源管理等逻辑移出特权内核，使核心代码缩减至约 9KLoC，并为每个虚拟机分配用户态 VMM 实例以防范单点失效。Jia 等人^[27]提出的 HyperEnclave 架构则通过软件定义实现了与 SGX 兼容的飞地抽象，成功将 TEE 保护能力与特定硬件微码解耦，并采用内存安全的 Rust 语言构建监视器。针对 ARM 架构，Pereira 等人^[28]利用静态分区技术构建了 Bao-Enclave，将安全功能从庞大的特权操作系统中剥离，实现了对硬件资源的精细化访问控制。

基于纯虚拟化的方案凭借硬件平台的广泛兼容性，能够在不依赖特定安全微码的前提下实现基础的物理级隔离。然而，庞大的通用 Hypervisor 一旦被攻破，将直接威胁所有虚拟机的核心数据安全。即使是经过极度精简的轻量级 Hypervisor，其作为特权层软件仍面临软件可信基冗余的问题，极易遭受复杂的侧信道分析，且难以防御针对物理内存的硬件探测攻击。纯虚拟化在底层硬件加密原语上的缺失，促使研究者转向架构级的硬件 TEE 方案。

1.2.2.2 基于进程与应用级抽象的细粒度 TEE 技术研究

为弥补纯虚拟化防御硬件攻击能力的不足，硬件厂商引入了具备芯片级内存加密与硬核隔离能力的专有扩展。早期架构级 TEE 主要聚焦于进程、应用或操作系统级别的细粒度抽象。这类方案仅将应用自身及必要的硬件纳入可信基，以换取极高的理论安全性。

在 x86 架构中，Intel SGX (Software Guard Extensions)^[4]定义了极其严苛的威胁模型，假定包括 OS 与 Hypervisor 在内的所有系统软件栈均不可信。然而，由

于传统可信应用（Trusted Application, TA）多采用 C/C++ 构建，易受内存破坏攻击威胁（Wang 等人^[13] 虽提出 Rust-SGX 框架予以缓解，但仍需对代码进行深度重构）。更关键的是，SGX 基于进程的隔离范式受限于受保护内存（EPC）容量与复杂的编程模型，导致深度学习等资源密集型应用面临巨大的迁移挑战。

在嵌入式主导的 ARM 架构中，机密计算常以基于可信 OS 的 TEE 方案作为基础（例如基于 ARM TrustZone 扩展与 GP-TEE^[8] 规范的示例实现）。针对 TrustZone 安全世界资源独占的局限，Brasser 等人^[29] 提出了 SANCTUARY 架构以动态划分用户态飞地；Wang 等人^[30] 提出的 RT-TEE 确保了系统在遭遇拒绝服务（Denial of Service, DoS）攻击时的实时性；RusTEE^[14] 则在编译阶段静态消除了 TEE 软件栈的内存隐患。

在新兴的开源指令集方面，Keystone 和 Penglai 是当前 RISC-V 主流的 TEE 方案。Penglai^[6, 31, 32] 通过引入受护页表（Granule Protection Table, GPT）等机制实现了高达 512GB 的动态隔离空间支持与高可扩展的 I/O 隔离，旨在解决云场景下的性能瓶颈；Keystone^[7] 则引入了可编程的安全监视器，将核心功能模块化与可重用化。此外，Ozga 等人^[33] 利用形式化验证框架确保了 RISC-V TEE 可信根的确定性。

纯粹的 TEE 技术虽然提供了芯片级的强隔离，但基于进程或 OS 抽象的方案在适配复杂生态时面临极高的开发门槛。开发者必须手动剥离敏感代码以划分可信基、重写应用程序，或高度依赖特定的 TEE-OS。这种架构不可避免地导致了严重的生态割裂，显著抬高了现有应用向机密计算环境迁移的综合成本。

1.2.2.3 基于机密虚拟机抽象的软硬融合 TEE 技术研究

为了彻底打破纯虚拟化在数据安全性上的瓶颈，同时解决应用/OS 级 TEE 面临的开发门槛高与生态割裂困境，将虚拟化技术的灵活抽象能力与 TEE 专用安全指令扩展相互结合，成为了机密计算演进的必然选择。该范式提出了基于机密虚拟机抽象的混合 TEE 架构。通过在硬件级加密的 TEE 中直接运行未经修改的客户 OS 与大规模应用，最大程度地复用了现有操作系统生态，极大降低了厂商的开发与迁移成本。

为破解灵活性与安全性的双重瓶颈，主流硬件厂商相继推出了支持机密虚拟机抽象的体系结构。以 x86 架构为例，AMD SEV^[5]（Secure Encrypted Virtualization）利用硬件虚拟化扩展，为每个虚拟机分配唯一的地址空间标识符（Address Space Identifier, ASID），并结合硬件级高级加密标准（Advanced Encryption Standard, AES）引擎确保内存数据以密文形式存储，实现了虚拟机与 Hypervisor 的语义脱钩；其增强版 SEV-SNP^[34] 则引入反向映射表（Reverse Map Table, RMP）机制，强制防止 Hypervisor 篡改内存权限。Intel 提出的 TDX^[10] 架构引入了全新的 SEAM

特权层级及专用模块，在架构层面彻底剥离了不可信 Hypervisor 对可信域生命周期的控制权。在 ARM 架构中，ARM CCA 同样引入了机密领域（Realm）的概念，以支持虚拟机级别的硬件强隔离。

与此同时，在开源的 RISC-V 生态中，传统多租户计算平台（Multi-tenant computing platforms）由于将平台固件、操作系统、虚拟机监视器及操作人员均纳入租户的可信基，导致其无法满足隐私导向型工作负载对可信基最小化的需求。针对这一局限性，Sahita 等人^[35]提出了面向 RISC-V 平台的机密虚拟机扩展 CoVE 的开源参考软件栈实现 Salus。CoVE 充分利用了 RISC-V 架构无历史包袱的优势，定义了全新的指令集与非指令集规范，以及系统级芯片（System on Chip, SoC）要求。该架构通过提供具备硬件认证（HW-attested）的 TEE 来实现对使用中数据（Data-in-use）的保护，为构建具备可量化可信基的原生机密计算架构奠定了基础。

此外，机密计算不仅在隔离云端庞大的不可信代码中发挥着关键作用，更亟需向资源受限的嵌入式场景延伸。针对这一需求，Ozga 等人^[33]提出了面向嵌入式 RISC-V 系统的机密计算框架 ACE（Assured Confidential Execution）。作为一种开源且免授权费（Royalty-free）的技术方案，ACE 旨在将云端机密计算的安全能力下沉，以构建更安全、更可靠的任务关键型（Mission-critical）嵌入式系统，进一步完善了基于虚拟化扩展的 RISC-V 机密计算应用生态。

综上所述，TEE 架构的研究已从早期的纯软件隔离与进程级细粒度保护，全面转向兼顾高安全性与生态兼容性的虚拟机级保护，通过软硬协同的深度演进，逐步确立了现代机密计算的技术底座。

1.2.3 异构计算环境下加速器 TEE 安全隔离与防护技术研究

随着端侧人工智能与云端大模型业务的爆发式增长，保障异构计算平台如 CPU+ 图形处理器（Graphics Processing Unit, GPU）或神经网络处理器（Neural Processing Unit, NPU）的数据隐私与模型完整性已成为系统安全领域的研究热点。在架构实现上，高性能服务器场景通常采用独立显存的离散式 GPU，TEE 仅需隔离 GPU 的内存映射 I/O（Memory-Mapped I/O, MMIO）区域；而在端侧场景中，集成式硬件加速器通常与 CPU 共享物理内存，不仅需隔离 MMIO，更要求对 GPU 缓冲区实施动态物理隔离。

在基于 CPU 安全特性的监控机制研究中，Wu 等人^[36]提出的 GEVisor 利用现有 Intel SGX 构建了经过形式化验证的安全参考监视器。面向端侧统一内存架构，Deng 等人^[37]在 StrongBox 中利用 ARM TrustZone 技术，通过动态拦截 MMIO 与二阶段页表控制实现了高效隔离。为突破传统 TEE 频繁数据加解密导致的性能瓶颈，Fan 等人^[38]提出的 XpuTEE 框架引入微型监控器 XM，通过联合抽象与直接

物理映射实现了“去加密化”监控，将大模型推理开销显著压缩至 2.5% 以下。

为应对新一代指令集架构与存量基础设施的兼容性挑战，机密计算研究逐渐转向软硬协同的深度解耦范式。Wang 等人^[39]提出的 CAGE 框架借助粒度保护检查（Granule Protection Check, GPC）原语率先实现了 ARMv9 架构下的 GPU 硬件级隔离。此外，针对现有机密加速器高度依赖特定硬件更迭的问题，Wang 等人^[40]设计了具备强兼容性的 ccAI 系统。该方案在物理层动态截获并重定向 GPU 任务提交路径，平滑支持了 DeepSeek、Llama 等大模型的机密推理。

尽管异构隔离技术不断演进，Lu 等人^[41]在 MOLE 研究中揭示了新型硬件侧攻击面：GPU 内部集成的微控制器（Microcontroller Unit, MCU，如 Falcon/GSP）拥有超越系统 I/O 内存管理单元（I/O Memory Management Unit, IOMMU）限制的特权级访问能力。攻击者可通过劫持 MCU 固件绕过 CPU 侧监控机制直接窃取 TEE 显存数据。该发现打破了直通即安全的传统假设，表明未来的异构机密计算亟需建立涵盖外设内部组件完整性度量的全路径防护体系。

1.3 本文主要贡献和创新

本文以基于 RISC-V 硬件虚拟化扩展及机密虚拟机安全生命周期管理为核心研究对象。为了克服传统虚拟化在机密计算场景中的安全隔离瓶颈，本文依托既有的基于微内核的虚拟化方案 EasyAda-Hypervisor，重点开展了底层安全固件设计、虚拟化组件安全扩展及端侧异构机密计算环境的构建。其主要研究内容与创新工作如下：

1. 提出并实现了一种基于 Rust 内存安全特性的模块化底层安全监视器 EasyAda-SM：针对传统 C 语言构建的安全固件极易受内存破坏漏洞威胁的局限性，本文采用 Rust 语言从底层独立设计并实现了 EasyAda-SM。在架构抽象层面，构建了安全区域与内存管理层，并深度结合 Rust 的所有权模型，实现了一套基于页分配器（Page Allocator）的安全内存调度机制，从而在编译期静态消除了二次释放等内存隐患。更进一步，为了实现执行环境间的强隔离，本文将 CPU 和设备侧物理内存保护机制抽象为统一的内存保护原语，严格切断了不可信软硬件对机密虚拟机安全内存的物理级访问路径。由此可见，该设计从软件自身安全性与硬件物理隔离两个维度，为整个系统确立了坚实的信任根基。

2. 提出并实现基于微内核的可信虚拟化方案 EasyAda-trust：为适应机密计算的严苛安全需求，本文对现有的 EasyAda-Hypervisor 进行了深度的安全扩展与架构增强。具体而言，在资源调度方面，本文引入了职责解耦机制，将常规的物理内存分配与设备发现保留在不可信的 Hypervisor 层，而将二阶段页表隔离映射、安

全区域动态构建及可信上下文切换等核心敏感操作全权委托给 EasyAda-SM 处理。此外，在运行态安全防护方面，结合本文设计的基于本地证明机制的安全启动链与机密执行流，确保了机密虚拟机的 CPU 上下文及内存视图对 Hypervisor 的绝对隐匿。基于上述软硬协同的设计，本文在不破坏原有微内核架构的前提下，成功实现了不可信虚拟机与机密虚拟机的安全共存与统一调度。

3. 扩展 EasyAda-trust 架构并构建端侧机密异构计算环境：针对端侧异构硬件中支持 DMA 的外设可能绕过 CPU 隔离机制发起的恶意攻击，本文进一步扩展了 EasyAda-trust 的系统设计。首先在硬件防护层，通过引入 RISC-V I/O 物理内存保护（输入输出物理内存保护（I/O Physical Memory Protection, IOPMP）扩展，实现了基于设备唯一标识（Source Identifier, SID）的外设动态鉴权与细粒度内存访问控制。然而，仅依靠底层硬件隔离难以满足复杂外设在多执行环境间的动态调度需求；因此在此基础上，本文创新性地提出并构建了专用的加速器管理机密虚拟机代理架构。该架构利用 I/O 半虚拟化技术与安全共享内存映射机制，在严格保障用户 AI 模型权重及隐私数据机密性与完整性的前提下，成功实现了底层硬件加速器在不可信与机密执行环境间的时空复用。值得注意的是，这一系列设计不仅化解了跨域直接内存访问（Direct Memory Access, DMA）攻击威胁，更为端侧异构算力的泛化安全部署确立了完备的技术范式。

1.4 论文结构

本文的内容组织与结构安排如下：

第一章：绪论。首先详细阐述了智能物联网与端侧异构计算环境下面临的安全性挑战，进而深入探讨了当前底层固件与虚拟化隔离所面临的核心痛点与科学问题。针对上述问题，引出了基于开放指令集与微内核架构构建可信执行环境的研究背景和意义，最后概述了本文的主要研究内容与章节结构安排。

第二章：相关技术理论概述。为解决第一章提出的架构痛点，本章重点探讨了本文方案所依赖的底层技术底座。首先，详细阐述了底层硬件隔离机制的相关概念（包括特权级模型、物理内存保护体系及硬件虚拟化扩展）；其次，介绍了机密计算的演进范式，并深入探讨了微内核虚拟化的相关理论；最后，进一步说明了异构计算场景下信任边界延伸的架构兼容性需求，从而为后续的系统设计奠定理论基础。

第三章：基于 RISC-V 的微内核虚拟化方案设计。本章从宏观层面介绍了在端侧强隔离需求下如何精准界定系统可信基边界。基于该信任边界，阐述了安全启动、内存隔离与中断路由等多维防护机制的构建思路。值得注意的是，本章最后

对基于动态外设隔离的硬件加速器安全复用模型进行了具体说明，从全局视角确立了系统的架构蓝图。

第四章：内存安全的模块化安全监视器实现。本章详细阐述了针对最高特权级固件内存脆弱性的防护机制。具体而言，首先探讨了如何从语言层面消除内存破坏漏洞，随后介绍了细粒度安全区域管理的抽象方法。此外，本章还对机密上下文切换、信任链推演及外设安全授权的理论模型进行了深入探讨，旨在筑牢系统的底层信任根基。

第五章：基于微内核的可信虚拟化方案实现。本章介绍了 CPU 状态隐匿与内存管控解耦的具体集成方案。为了应对复杂的 I/O 安全威胁，进一步阐述了防 DMA 攻击与端到端数据保护的部署流程。最后，对跨域通信信道与加速器机密部署机制进行了具体说明，由此打通了微内核机密虚拟化的全链路技术栈。

第六章：系统测试与分析。为验证上述架构设计与系统集成的有效性，本章介绍了多维度定量评估与安全分析体系的构建方法。在性能评估方面，通过基准测试量化了底层固件的生态兼容性与计算开销；在安全验证方面，设计边界条件检验了双向证明与可信启动链的稳健性。基于上述结果，对系统在安全隔离与执行效能间的架构折中进行了全面评估，充分证明了本文方案在实际嵌入式场景中的可行性。

第七章：总结与展望。综合前述理论、设计与实验验证，对本文的研究工作与核心理论贡献进行系统性总结。同时，客观审视了目前微内核架构与底层安全机制融合中存在的局限性，并由此提出多核并发调度优化、核心组件形式化验证及异构总线安全等后续的研究方向。

第二章 相关技术理论概论

2.1 RISC-V 背景知识

x86 架构因其漫长的发展历史与向后兼容性负担，指令集日趋复杂庞大。尽管在服务器等高性能计算领域表现良好，却难以契合嵌入式场景对严苛功耗与成本的约束。

ARM 架构作为精简指令集（RISC）的代表，凭借 IP 授权模式赋予下游厂商一定的定制自由度，在嵌入式市场占据主导地位。然而，受限于其封闭的商业壁垒，硬件开发面临高昂的知识产权（Intellectual Property, IP）授权成本；同时，ARM 架构跨代演进（如 ARMv7 至 ARMv8）差异显著，导致了严重的指令集碎片化问题，系统高度依赖特定版本的编译工具链以维持跨平台兼容性与性能优化。

相比之下，新兴的 RISC-V 架构凭借开源开放与高度模块化的设计理念，与系统软件的解耦演进需求高度契合。硬件厂商可根据具体场景按需组合指令集扩展，软件生态亦能据此实现灵活伸缩，有效规避了跨代架构更迭带来的高昂代码移植成本。在云边端协同计算趋势下，资源受限的边缘节点可仅实现基础指令子集以满足极低功耗约束，而云端高性能设备则可集成复杂扩展以跃升算力。RISC-V 这种高度弹性的架构特性，为构建跨越异构硬件的通用、可扩展的可信执行环境（TEE）提供了理想的底层硬件支撑。

2.1.1 RISC-V 特权级介绍

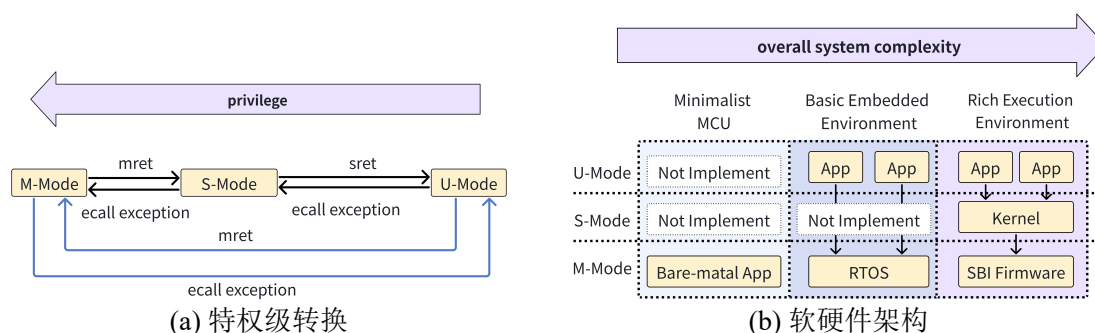


图 2-1 未引入硬件虚拟化扩展时 RISC-V 系统架构

RISC-V 架构将独立的 CPU 核心抽象为硬件线程（Hard Thread, Hart）。在任何运行时态下，Hart 均处于特定的特权级中。RISC-V 规范定义了由低至高的三种主要特权级：用户模式（U-Mode）、监管者模式（S-Mode）与机器模式（M-Mode）。如图2-1(b)所示，根据指令集标准，M 态为强制实现的基础特权级，而 U 态与 S 态

为可选扩展。实际商用硬件通常基于特定应用场景与功耗考量进行架构裁剪，如运行 RTOS 或裸机应用的 MCU 场景可以仅实现 U 态和 M 态，甚至仅实现单 M 态。

各特权级之间具备严格的权限隔离边界^[42]：U-Mode 作为最低特权级，仅获准访问通用寄存器并执行基础指令，其访存操作严格受限于页表映射机制；S-Mode 在此基础上，进一步获准访问特定的控制与状态寄存器（Control Status Register, CSR）并执行相关特权指令，其访存操作同样受限于内存管理单元（Memory Management Unit, MMU）提供的页表机制；M 态作为系统最高特权级，拥有对所有物理硬件、CSR 及特权指令的无限制访问权，并可绕过页表直接寻址物理内存。任何低特权级尝试执行越权指令的行为均会触发硬件异常拦截。

在特权级状态转换机制上，系统上电重置时 Hart 默认初始化为 M 态。控制流从高特权级向低特权级转移需显式执行 mret 或 sret 指令完成状态降级；反之，当系统发生外部中断或执行异常时，硬件将根据 CSR 中的委派（Delegation）配置，自动陷入（Trap）至 S 态或 M 态的异常向量入口。此外，低特权级软件可主动触发 ecall 或 ebreak 指令发起系统调用，以同步陷入的方式将控制权安全移交至高特权级执行环境。各特权级间的状态转换逻辑如图2-1(a)所示。

2.1.2 RISC-V 中断与异常处理机制介绍

RISC-V 架构将所有打破正常程序控制流的突发事件统称为陷入（Trap）。在微架构层面，陷入被严格划分为两类：同步的异常（Exception）与异步的中断（Interrupt）。异常通常由当前执行的指令直接引发（如非法指令解码、缺页异常或主动发起的 ecall 系统调用），而中断则由独立于当前执行流的外部异步事件触发。RISC-V 手册规定中断按类型分为外部中断（External Interrupt）、软件中断（Software Interrupt）、时钟中断（Timer Interrupt）三种。

根据 RISC-V 基础特权级规范，系统发生的所有陷入默认均强制路由至最高特权级 M 态处理。然而，在部署了复杂操作系统或 Hypervisor 的场景中，频繁的 M 态陷入与上下文切换将引入不可接受的性能开销。为此，RISC-V 创新性地设计了陷入委派机制。通过精细化配置 M 态异常委派寄存器（medeleg）与中断委派寄存器（mideleg），底层安全固件可将特定类型的陷入事件（如 S 态下的缺页异常或时钟中断）显式旁路，使其直接触发 S 态的陷入响应而无需 M 态介入。这一机制极大地缩短了关键中断的响应延迟，是实现高效软硬件协同与机密虚拟化资源调度的核心架构支撑。

2.1.3 RISC-V 内存保护机制介绍

2.1.3.1 物理内存保护机制

PMP 机制是 RISC-V 架构标准提供的核心安全原语，也是在该架构上构建 TEE 的必要底层硬件支撑。在微架构层面，每个 Hart 均集成了一组专用的 PMP 地址寄存器（`pmpaddrx`）与配置寄存器（`pmpcfgx`），且这些寄存器的读写权限被严格限制在 M 态。

在使能了 PMP 机制的硬件平台中，系统默认采用严格的隔离访问策略：在未显式授权的情况下，运行在更低特权级的软件被默认限制对任何物理地址的访问。通过在 M 态下对 PMP 寄存器组进行精细化配置，系统能够以 Hart 为粒度，为其划分特定的连续物理地址空间，并显式授予相应的读、写及执行权限。简言之，PMP 机制从硬件底层提供了一种强隔离的访问控制手段，能够精准限制特定 Hart 对任意物理内存区域的访存边界与操作能力。

2.1.3.2 增强的物理内存保护（Smepmp）机制

Smepmp（Supervisor Mode Execution Prevention and Machine-mode Physical Memory Protection）扩展是对 RISC-V 基础 PMP 机制的关键安全增强。在原生 PMP 架构中，锁定位（L 位）的语义存在天然的耦合局限：当 $L=0$ 时，M 态默认拥有对全量物理内存的无限制访问权；当 $L=1$ 时，M 态与低特权级将被强制绑定相同的 R/W/X 权限。这种粗粒度的设计导致系统无法在赋予 S 态合法访存权限的同时有效剥夺 M 态的对应权限，难以防御针对高特权级固件的混淆代理（Confused Deputy）攻击，亦无法从架构层面彻底杜绝 M 态意外执行驻留于低特权级内存中的恶意载荷。

为突破上述架构瓶颈，Smepmp 扩展创新性地引入了机器安全配置寄存器（`mseccfg`）。该寄存器通过三个核心控制位重塑了底层的物理访存控制逻辑：首先是机器模式锁定（Machine Mode Lock, MML）位，当 MML 被置位时，系统将彻底重载传统 PMP 配置中 L 位与 R/W/X 位的组合语义，实现了 M 态与更低特权级内存权限的深度解耦。例如系统可配置仅低特权级可读写、M 态不可访问的隔离区域，从而在物理硬件层面严格贯彻了最小特权原则。其次是机器模式白名单策略（Machine Mode Whitelist Policy, MMWP）位，使能该位将逆转 M 态的默认访存行为，从原有的默认放行切换为严苛的默认拒绝（Default-Deny），强制要求 M 态的所有访存操作必须命中显式授权的 PMP 条目。最后是规则锁定旁路（Rule Lock Bypass, RLB）位，它在维持整体隔离强度的前提下，允许授权的安全固件动态修改已锁定的 PMP 条目，极大提升了 TEE 运行时的内存管理弹性。

综上所述，Smepmp 机制在微架构层面弥补了原生 PMP 权限颗粒度粗糙的设计缺陷，而原生 PMP 机制以最小的硬件开销构筑了坚实的宏观物理隔离边界，为嵌入式环境提供了兼具高实时性与高性价比的底层安全原语。

2.1.3.3 I/O 物理内存保护机制

IOPMP 是 RISC-V 架构中用于保障系统级外设访存安全的关键硬件扩展。尽管原生的 PMP 机制能够有效限制 Hart 的越权访存行为，但它仅作用于 CPU 侧，无法约束具备 DMA 能力的外设。恶意或受控的外设可轻易绕过 CPU 的 PMP 隔离边界，直接对受保护的物理内存发起读取或篡改操作（即 DMA 攻击），这构成了机密计算环境下的重大安全隐患。

为弥补这一架构级安全短板，RISC-V 引入了 IOPMP 扩展。在微架构实现上，IOPMP 通常部署于系统互连总线与外设控制器之间，充当所有 I/O 访存事务的硬件级安全网关。其核心访问控制逻辑基于设备 SID 展开：当外设发起总线级读写请求时，IOPMP 会首先捕获该请求携带的 SID，并通过内部的路由表将其映射至特定的内存域（Memory Domain，MD）。每个内存域内配置了一组类似于 PMP 的规则条目，用于精确界定该 SID 对应的设备所能访问的连续物理地址范围及其读写权限。任何未命中授权条目的 DMA 请求均会被 IOPMP 硬件直接拦截并上报异常。

2.1.4 RISC-V 硬件虚拟化扩展（H-extension）介绍

RISC-V 硬件虚拟化扩展（H-extension）是 RISC-V 指令集架构中专为提升 Hypervisor 运行效率而设计的关键硬件扩展。传统软件模拟的虚拟化方案面临高昂的陷入-模拟开销，而 H 扩展通过在处理器微架构层面引入新的特权级、寄存器副本以及二阶段地址翻译机制，从硬件底层为高效构建 Type-1 或 Type-2 型 Hypervisor 提供了系统级支撑。

如图2-2所示，使能 H 扩展后，原有的 S 态被扩展为 Hypervisor 扩展监管者模式（HS-Mode），同时新增了虚拟监管者模式（VS-Mode）与虚拟用户模式（VU-Mode）。在 HS 态下运行的 Hypervisor 拥有对物理硬件与虚拟机的完全控制权；而客户 OS 及 Guest App 则分别运行在 VS 态与 VU 态下。这种设计确保了客户 OS 无需修改核心代码即可无感运行，并在试图执行影响全局的敏感特权指令时安全地触发硬件异常并陷入 HS 态处理。

频繁的虚拟机退出（虚拟机-Exit）是制约虚拟化性能的核心瓶颈之一。为此，H 扩展为 VS 态引入了一套完整的 S 态寄存器影子副本（如 vsstatus、vstvec、vsepc、vsscratch 等）。当处理器运行在虚拟化环境（V=1）时，客户 OS 对常规 S 态寄存

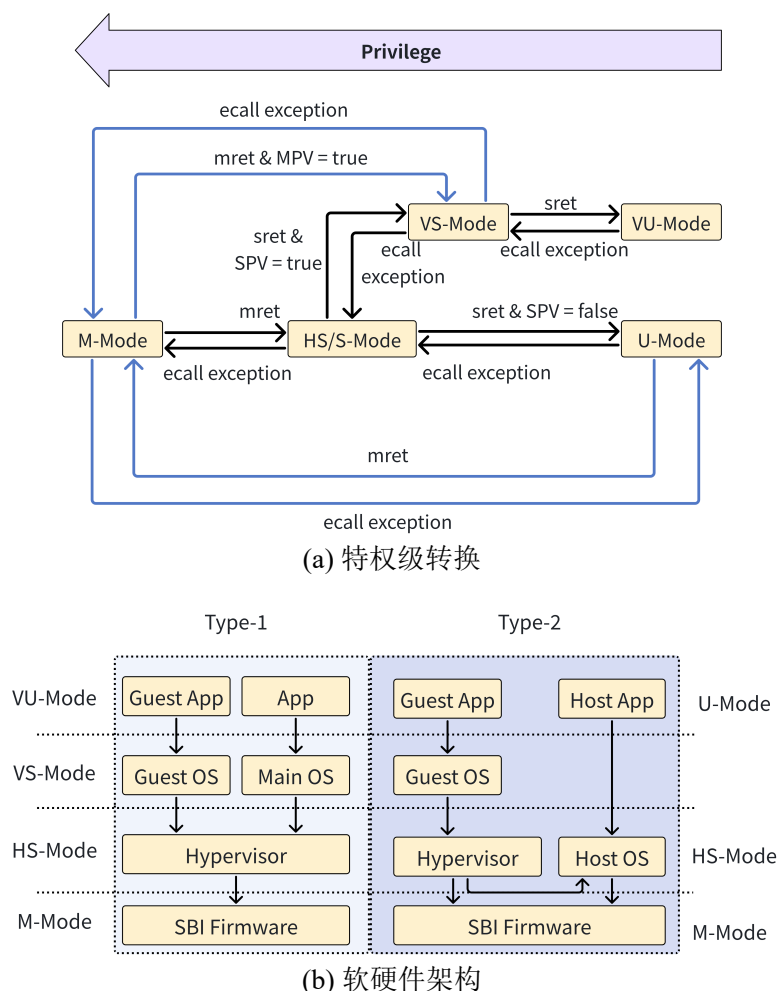


图 2-2 引入硬件虚拟化扩展后的 RISC-V 系统架构

器的读写操作会被硬件自动重定向至对应的虚拟控制状态寄存器中。这一机制使得客户系统在配置自身上下文时无需陷入 HS 态，极大地降低了寄存器访问带来的上下文切换开销。

为实现虚拟机间的物理内存强隔离，H 扩展引入了二阶段页表机制。第一阶段（Stage-1）完全由客户 OS 控制，负责将虚拟地址翻译为客户物理地址（Guest Physical Address, GPA），其页表基址由虚拟监管者地址转换与保护寄存器（*vsatp*）管理；第二阶段（Stage-2）由 Hypervisor 严密管控，负责将 GPA 强制翻译为真实的宿主物理地址（Host Physical Address, HPA），其页表基址由 Hypervisor 客户地址转换与保护寄存器（*hgatp*）管理。MMU 在处理 VS/VU-Mode 的访存请求时，会自动串行执行这两阶段翻译，彻底阻断了虚拟机越权访问宿主物理内存或其他虚拟机地址空间的可能。

同时，H 扩展增强了 RISC-V 原有的委派机制。通过配置 Hypervisor 级的中断与异常委派寄存器（*hideleg* 与 *hedeleg*），系统可将原本默认陷入 HS 态的特定异常

（如 VS 态系统调用或缺页异常）直接旁路委派至 VS 态，交由客户 OS 内部的处理程序直接响应。此外，H 扩展设计了完善的虚拟中断注入机制（如通过 hvip 寄存器配置 VSSIP、VSTIP、VSEIP），允许 Hypervisor 在软件层面灵活模拟虚拟硬件外设，并高效地向目标 VS 态挂起并注入虚拟中断信号。

2.2 可信执行环境（TEE）相关技术介绍

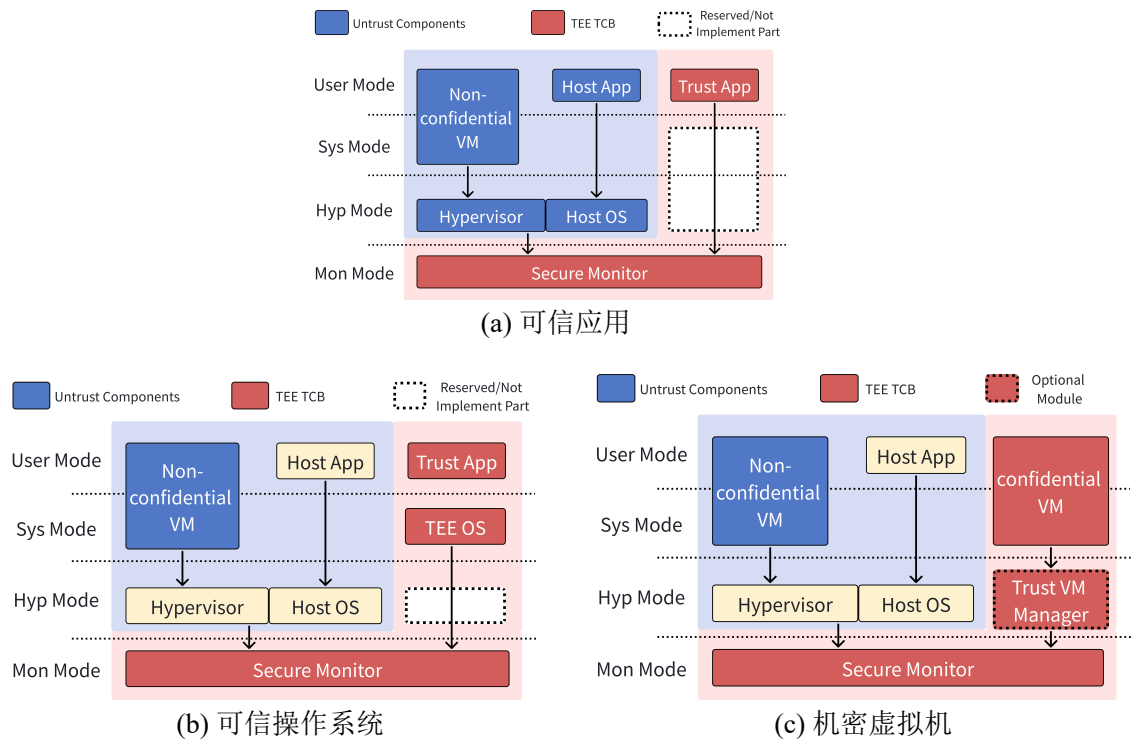


图 2-3 基于系统抽象层级与软件执行模型的 TEE 分类

TEE 通过在硬件微架构层面构建受保护的隔离执行空间，通过对传入和传出的数据进行加密，为敏感代码与执行状态确立了系统级的机密性与完整性保障。尤为关键的是，TEE 能够从指令集架构底层重塑信任边界，有效抵御来自系统中任何潜在恶意实体或高特权级系统软件（如不可信的宿主操作系统内核或虚拟机监视器）的越权访问与潜在攻击。相较于全同态加密或多方安全计算等纯密码学计算范式，TEE 凭借以下三大核心特性展现出显著的技术实用性与工程优势^[43]：

1. **通用可编程性（General Purpose Computation）**：TEE 内部所支持的执行功能通常表现为用户自定义程序，这赋予了其图灵完备的通用计算能力，使其能够广泛适配各类复杂的业务功能与算法形态。

2. **强机密性与执行完整性（Integrity and Privacy）**：TEE 在物理信任边界内同时提供双重安全属性。机密性（Confidentiality）^[44] 确保了外部潜在攻击者无法从

计算过程中窥探或获取任何敏感信息，而完整性（Integrity）则从底层机制上保证了目标程序的执行逻辑免遭篡改并被严格且正确地执行。

3. 近似原生的计算性能（High Performance）：TEE 的核心计算范式是在安全飞地内部将所有加密输入解密后，直接依托物理 CPU 在明文状态下执行目标自定义程序。因此，除了处理加密数据输入输出所产生的额外开销外，TEE 能够充分释放并利用 CPU 的原生运算速度，彻底规避了纯密码学方案中高昂的算力惩罚，具备极高的高性能优势。

根据系统抽象层级与软件执行模型的差异，如图2-3所示，当前业界的隔离与机密计算架构主要经历了以下三种演进范式：

1. 基于纯虚拟化的安全隔离方案：传统的纯虚拟化方案（如基于 KVM 或 Xen 的隔离）通过 Hypervisor 在系统级提供虚拟机间的强隔离。然而，这种方案的安全性高度依赖于 Hypervisor 自身的代码可靠性。由于通用 Hypervisor 体量庞大且极其复杂，其作为可信基存在巨大的攻击面。一旦 Hypervisor 被攻破，所有虚拟机的核心数据将面临严重威胁；且纯虚拟化缺乏底层硬件密码学机制的支撑，难以抵御物理层面的硬件探测与内存窃取攻击。这种安全性的内生不足，促使业界向具备芯片级强隔离的 TEE 技术演进。

2. 面向可信应用和操作系统的 TEE 架构：早期硬件辅助的 TEE 架构主要聚焦于细粒度的应用级或 OS 级 TEE 抽象。此类方案的典型代表包括基于应用级抽象的 Intel SGX 和基于 OS 抽象的 ARM TrustZone 扩展与 GP-TEE 规范的示例实现。Penglai 和 Keystone 是当前 RISC-V 主流的 TEE 方案，它们分别探索了灵活的应用级与 OS 级隔离机制。这类方案虽然弥补了纯虚拟化的安全性短板，提供了芯片级的强隔离与内存保护，但存在显著的局限性：开发者必须手动剥离敏感代码以划分可信基、重写应用程序或高度依赖特定的专属 TEE-OS。这种极高的开发门槛不可避免地导致了严重的生态割裂，显著抬高了现有庞大应用向机密计算环境平滑迁移的综合成本。

3. 面向机密虚拟机的 TEE 架构：为了彻底打破纯虚拟化在数据安全性上的瓶颈，同时解决基于应用/OS 级 TEE 面临的生态割裂与开发门槛高的困境，将虚拟化技术的灵活抽象能力与 TEE 专用的硬件安全指令扩展相互结合，成为了机密计算演进的必然选择。该范式提出了基于机密虚拟机抽象的混合 TEE 架构如 Intel TDX、AMD SEV-SNP、ARM CCA 以及 RISC-V CoVE 等。通过将复杂的 Hypervisor 移出可信基，并在硬件级加密的 TEE 中直接运行未经修改的客户 OS 与大规模应用，该架构最大程度地复用了现有的操作系统生态。这不仅有效收敛了可信基的攻击面，还极大降低了厂商的开发与应用迁移成本，代表了当前机密计算的最前沿趋势。

2.2.1 面向异构计算环境的 TEE

业界已经在 CPU 侧建立起了相对成熟的 TEE 机制用于隔离和保护敏感数据。然而，随着机器学习、大数据处理等数据密集型应用的爆发，单核或多核 CPU 的算力已无法满足海量数据的高性能计算需求。为了在利用 GPU、NPU 和现场可编程逻辑门阵列（Field Programmable Gate Array, FPGA）等硬件加速器提供澎湃算力的同时，依然能够享受高强度的机密性、完整性与隔离性保护，将 TEE 的安全信任边界从主机 CPU 延伸至异构加速器设备（即构建加速器 TEE 或异构 TEE）成为了学术界与工业界的必然演进方向。这种信任边界的延伸不仅需要保护传统的明文数据和代码，还必须涵盖 AI 模型参数、输入输出特征、页表映射以及底层硬件环境状态。

根据加速器与主机 CPU 在物理架构与互连方式上的差异，当前异构计算环境的集成范式主要可划分为以下三类。不同的范式对底层安全隔离机制与 TEE 架构设计提出了截然不同的挑战：

1. 指令集扩展：该范式将 AI 或密码学运算逻辑直接嵌入 CPU 的执行流水线中，通过 RISC-V 等架构预留的编码空间实现自定义指令扩展（如矩阵运算或向量增强）。加速单元共享处理器的通用寄存器组或向量寄存器，加速指令由 CPU 译码后直接分发至执行单元。此模式消除了数据在异构组件间的移动开销，具有极高的指令响应速度和极低的上下文切换延迟。在 TEE 视角下，其信任边界完全收敛在 CPU 物理核心内部，复用原生的 CPU 特权级与内存隔离机制即可实现高安全性的保护。它适用于细粒度、高频次的张量运算，是追求极致单核计算密度及实时性场景的首选理论模型。

2. 片上协处理器：此范式下，加速器作为 SoC 内部独立的硬件 IP 存在，通过片上系统总线与 CPU 交互。加速器拥有独立的控制逻辑与局部存储空间，由 CPU 通过 MMIO 进行任务配置和启动，并通过 DMA 自主完成大规模数据的搬运。通过异步工作机制，CPU 仅负责高层调度，而加速器专注于流水化处理大体量数据。从安全维度来看，协处理器通常与 CPU 共享片上物理内存，其 TEE 设计需重点防范恶意的越权 DMA 攻击，高度依赖诸如 IOPMP 等总线级物理内存保护硬件来确立硬件级隔离边界。这种模式在能效比与系统灵活性之间进行了平衡，是当前主流嵌入式 AI SoC 的硬件范式。

3. 片外独立加速器：加速器位于独立的物理芯片或插卡上，通过高速串行计算机扩展总线（Peripheral Component Interconnect Express, PCIe）或计算扩展链接（Compute Express Link, CXL）与主机互联。此类加速器通常具备独立的高带宽全局显存和复杂的层次化缓存结构。系统通过驱动程序管理设备内存空间，并采用

命令队列模式进行大规模任务的批处理提交。其逻辑设计的复杂度不再受限于主机 SoC 的热设计功耗与封装面积，能够提供极高的峰值算力。然而在机密计算场景中，由于跨物理总线通信的存在，该范式不仅对主机侧的安全隔离机制提出了更高的时延容忍度要求，还必须引入端到端的内存加密引擎（Memory Encryption Engine, MEE）与总线加密协议，以彻底抵御总线探测、数据篡改与物理重放攻击。

尽管异构 TEE 研究在近年来呈现出爆发式增长，但其在真实物理设备上的大规模部署仍面临严峻的工程与理论痛点。首先是系统架构的碎片化与通用性缺失，现存的大量加速器 TEE 方案往往强绑定于特定的“CPU-加速器”硬件组合，缺乏标准化的软硬件交互抽象，导致跨平台迁移极其困难。其次是可信计算基的急剧膨胀，为了支持复杂的异构加速器调度，传统方案往往被迫将体量庞大的设备驱动程序（如动辄百万行代码的 GPU 软件栈）直接纳入可信环境中，严重违背了安全固件微内核化的极简设计初衷，极大地增加了系统的潜在攻击面。

2.3 微内核相关知识介绍

2.3.1 微内核操作系统 EasyAda 介绍

EasyAda 是一款自研微内核操作系统，如图2-4所示，其设计理念遵循最小特权原则，旨在构建高安全性、高可靠性的操作系统内核，适用于对稳定性和安全性要求较高的嵌入式应用场景。EasyAda 将内核态精简为仅保留必要的特权功能模块，如核间通信、权能管理、中断与异常处理、时钟管理、地址空间管理和内核对象管理等核心组件，内核将对外提供的服务封装为系统调用接口，仅暴露较小的攻击面。同时，通过硬件抽象层（Hardware Abstraction Layer, HAL）抽象了操作系统依赖的硬件原语，如启动/暂停 CPU 核心，中断配置和处理，MMU 配置等，实现了统一的硬件操作接口。这降低了操作系统在不同指令集和架构间移植的难度，并且有效降低了内核代码的复杂度，确保 EasyAda 可以在异构环境上对外提供统一的软件平台。

EasyAda 将除核心功能外的其余大部分服务迁移至用户态，在地址空间隔离的上下文中运行，并由根服务（Rootserver）统一管理。根服务作为一个运行在用户态的多线程进程，集成了多个功能子系统。这些子系统既包括独立的模块化组件，如文件系统、进程管理、网络协议栈等，也包括执行特定任务的独立线程，如 Shell 交互模块等。根服务将对外提供的服务封装为 RPC 接口，为操作系统提供统一、透明的服务访问接口，并且对 RPC 参数进行严格的检查以防外部攻击。其他服务或用户应用可以通过系统共享库提供的 POSIX 标准的 API 前端，将函数调用转换为远程过程调用（Remote Procedure Call, RPC）调用。同时，所有应用和

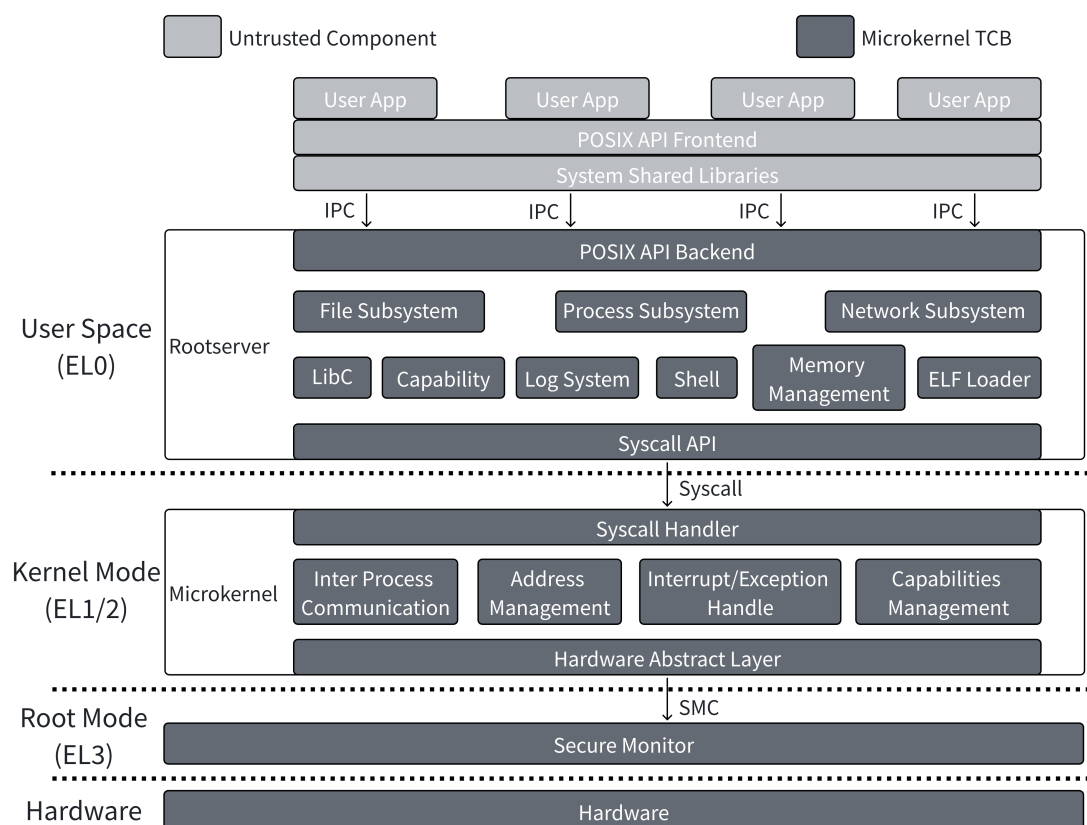


图 2-4 微内核操作系统 EasyAda 总体架构

系统服务均拥有独立的权能空间，仅能访问当前权能空间支持的 RPC 接口和系统调用，进一步缩小了系统的攻击面。

基于微内核的设计思想，EasyAda 相比 Linux 等宏内核操作系统具备更小的软件可信基，而相比传统的 RTOS 如 FreeRTOS 等，EasyAda 又具备强大的可扩展性，可以通过扩展系统服务实现与宏内核对等的功能和性能。因此 EasyAda 通过灵活调整用户态系统服务配置，可以适应从边缘到中高端嵌入式等多维度的场景和需求，为云边端场景提供通用的 OS 底座。

由于 EasyAda 被设计为面向多样化场景的通用操作系统，因此其应用和系统服务运行在用户态，内核运行在内核态（未开启虚拟化）或 Hypervisor 态（开启虚拟化支持），并且通过安全固件接口与运行在最高特权级的安全固件进行交互以实现硬件管理。

2.3.2 微内核虚拟化方案 EasyAda-Hypervisor 介绍

图2-5展示了支持管理机密虚拟机生命周期的、基于 RISC-V 的 EasyAda-Hypervisor 设计。EasyAda-Hypervisor 严格遵守了 EasyAda 微内核的设计哲学和思路：

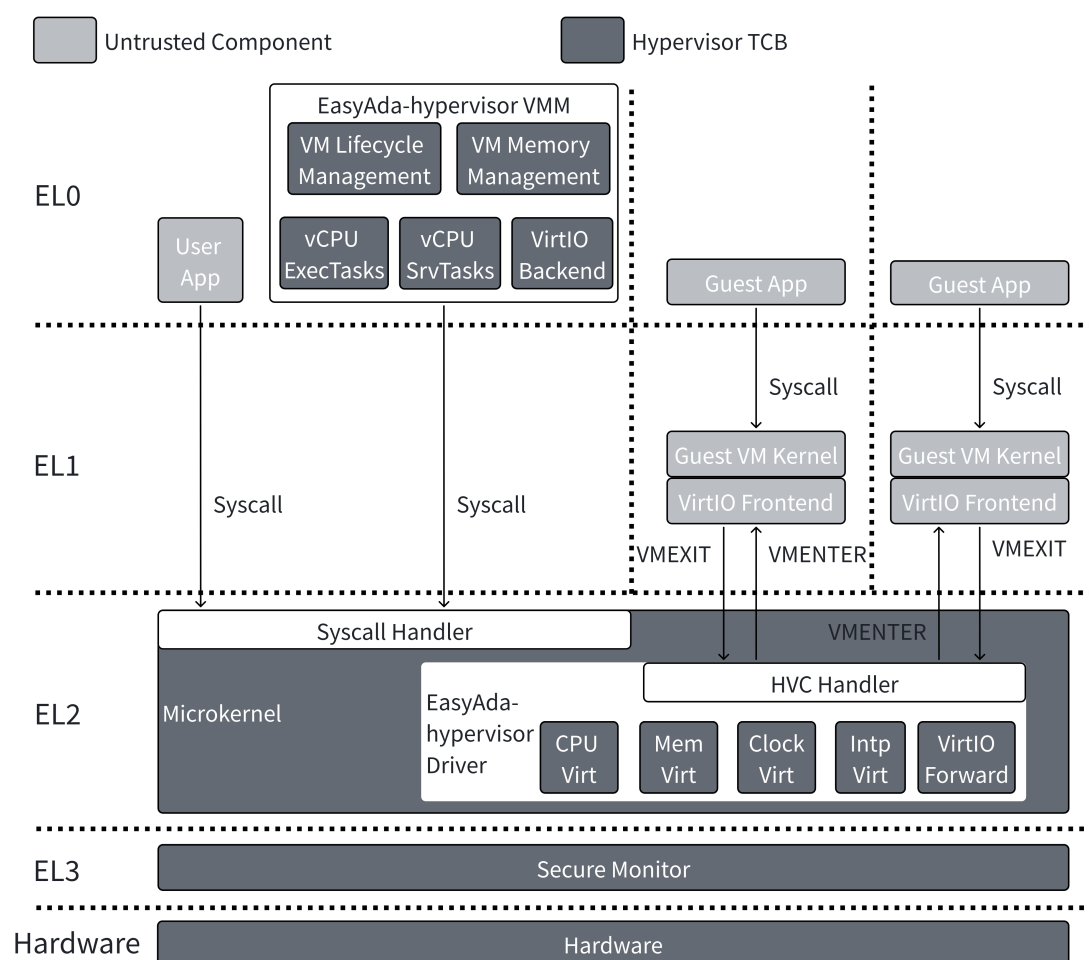


图 2-5 EasyAda-Hypervisor 总体架构

1. 统一资源管理：主要系统资源由 EasyAda 负责管理和分配。充分复用微内核操作系统现有代码和能力，实现统一的资源管理并减少 EasyAda-Hypervisor 的代码量，缩小其可信基。

2. Hypervisor 解耦：Hypervisor 被划分为运行在 EasyAda 用户态下的 VMM 和内核态下的 Hypervisor 驱动，以尽可能复用 EasyAda 现有代码和功能。前者负责主要的虚拟机管理工作，包括管理虚拟机生命周期和软硬件资源，模拟虚拟化设备后端，以及管理虚拟机服务线程等；后者作为屏蔽不同架构硬件虚拟化扩展实现的抽象层，对外提供基于虚拟化硬件原语抽象的统一接口，完成对硬件虚拟化扩展相关寄存器的配置，以及接收和处理虚拟机中断和异常等。

3. 静态分区：Hypervisor 采用静态分区策略。虽然该策略在资源动态调配上略显不足，但考虑到嵌入式场景应用高度定制化、负载相对确定的特点，静态分区策略足以满足其功能需求。更为关键的是，静态分区策略可以简化代码逻辑，缩小 Hypervisor 的攻击面。这一点在引入机密虚拟机支持之后更加重要，因为标准

RISC-V CoVE 扩展要求安全监视器实现数十个 SBI 接口以实现对机密虚拟机资源的细粒度控制，这不仅显著增大了可信基的攻击面，还导致 Hypervisor 需要多次 SBI 调用才能完成单一功能，严重影响了系统的实时性和性能。

EasyAda-Hypervisor 通过在 EasyAda 中引入两个核心组件实现基于硬件虚拟化扩展的虚拟机生命周期管理。其中，运行在 Hypervisor 态的 Hypervisor 驱动负责抽象虚拟化硬件原语并对上提供虚拟化能力接口。其虚拟化能力主要可以分为以下五类：（1）CPU 虚拟化：基于虚拟特权级为虚拟机提供符合其运行要求数量的 vCPU，并在隔离的硬件环境中运行虚拟机上下文，在实现操作系统无感迁移运行的同时，避免虚拟机访问和篡改 Hypervisor 或其他虚拟机的上下文信息；（2）内存虚拟化：基于硬件虚拟化提供的二阶段地址翻译能力，通过动态配置和管理虚拟机的二阶段页表，实现细粒度的、虚拟机无感的强制内存隔离；（3）中断虚拟化：基于硬件提供的中断控制器，通过中断注入或中断直通等手段实现虚拟机中断管理，避免虚拟机接收和配置不属于当前虚拟机的中断；（4）时钟虚拟化：基于虚拟硬件时钟，通过虚拟化计时器和虚拟时钟补偿计时器，实现虚拟机无感退出与恢复；（5）I/O 虚拟化：基于共享物理内存和虚拟中断，通过为不同虚拟机模拟虚拟设备，在保证虚拟机无感迁移的同时，提升虚拟机的 I/O 性能和安全性。

用户态 VMM 主要包括三个部分：虚拟机资源和生命周期管理模块、vCPU 执行线程以及 vCPU 服务线程，负责提供完整的虚拟机管理与运行支持。虚拟机资源和生命周期管理模块统筹虚拟机的创建、运行、暂停、恢复及删除等功能，并且通过 Shell 和 RPC 接口对外提供交互式命令行，简化用户对虚拟机实例的管理。

vCPU 执行线程可以视为 vCPU 在 Hypervisor 侧的调度实体和用户接口，而 vCPU 服务线程则作为半虚拟化设备的后端对虚拟机的 I/O 请求进行响应和异步处理，以实现 I/O 虚拟化和中断处理的高效执行。vCPU 服务和执行线程均运行于 VMM 进程上下文中，与 VMM 及内核态虚拟化组件协同工作，优化虚拟机的计算与 I/O 任务调度。

2.4 Rust 相关知识介绍

Rust^[12] 是一种旨在兼顾可靠性与执行效率的系统级编程语言。为了在内存安全（Memory-safety）与线程安全（Thread-safety）这两个核心维度上实现可靠性，Rust 提供了以下机制：（1）确立每个数据对象的所有权（Ownership）；（2）自动检查每个对象的读写权限（即可变性，Mutability）；（3）强制执行针对所有对象的生命周期管理（Lifetime management）；（4）禁止不安全的类型转换以保障类型安全（Type-safety）；（5）禁用危险的裸指针（Raw pointer）操作，例如指针别名（Pointer

aliasing) 或悬垂指针 (Dangling pointers)。

在程序编译阶段, 若代码违反了 Rust 的任何安全准则, 编译器将抛出异常并生成详尽的错误诊断信息, 以辅助开发者进行相应的代码修正。除了显著提升代码安全性外, Rust 还带来了其他多项工程优势, 例如高效的并行处理能力、对开发者友好的编译提示, 以及成千上万个用于支撑各类开发需求的 Crate (功能类似于 C 语言中的 Library)。

尽管 Rust 默认被设计为遵循严格的安全准则, 但为了确保其具备编写任何底层系统程序的能力, 它同样提供了 `unsafe` 关键字, 允许开发者显式地注入非内存安全的代码片段。Rust 保留此 `unsafe` 选项主要基于两方面核心考量: (1) 允许开发者实现某些无法通过编译器默认静态检查的“特殊”功能逻辑; (2) 允许代码直接与底层操作系统或物理硬件组件进行交互。被标记为 `unsafe` 的代码域能够绕过 Rust 的内置安全检查器, 因此可能会执行存在潜在脆弱性的操作, 例如对不可变变量发起写操作、执行非标准的数据类型转换, 或是直接操作并解引用裸指针。在 Rust 开发中, 使用 `unsafe` 代码段的一个典型场景是必须调用基于 C 语言构建的底层函数, 该机制在 Rust 中被严格定义为外部函数接口 (Foreign Function Interface, FFI)。然而, `unsafe Rust` 在带来底层控制力与能力扩展优势的同时, 亦不可避免地极易为系统引入潜在的安全漏洞。

2.5 本章小结

本章系统性地阐述了构建异构机密计算底座相关的核心技术理论。首先, 详细分析了 RISC-V 架构的特权级模型与陷入委派机制, 探讨了包含 PMP、Smepmp 和 IOPMP 在内的物理内存保护扩展机制, 并论述了硬件虚拟化扩展在微架构层面提升虚拟机运行效率的实现原理。其次, 归纳了 TEE 的基本概念、核心安全特性与主流架构演进范式。针对异构计算场景, 分析了信任边界由主机 CPU 向外部硬件加速器延伸过程中存在的硬件隔离与架构兼容性问题。随后, 介绍了自研微内核操作系统 EasyAda 及其虚拟化方案 EasyAda-Hypervisor 的架构设计和实现, 论证了其基于最小特权原则与统一资源抽象在降低可信基及提升系统可扩展性方面的架构优势。最后, 阐述了 Rust 语言在保障内存安全与线程安全方面的核心机制, 并客观分析了其在底层系统编程中的技术优势与潜在安全隐患。

第三章 基于 RISC-V 的微内核虚拟化方案设计

3.1 使用场景与设计目标分析

本文提出的基于 RISC-V 的微内核虚拟化方案 EasyAda-trust，主要面向支持硬件虚拟化的中高端嵌入式应用场景（如智能驾驶域控制器、端侧人工智能网关等）。在此类复杂的异构场景中，系统通常需要在一套物理硬件上，并发运行实时控制任务（如车辆底盘控制）、通用富执行环境（如车载娱乐系统）以及安全敏感的计算任务（如端侧 AI 模型推理、生物特征识别）。

此类混合关键性（Mixed-Criticality）业务形态对底层系统架构提出了严苛的挑战。首先在功能维度，系统必须具备跨隔离域的资源抽象与复用能力；其次在安全维度，系统必须应对复杂的环境威胁，以保护隐私数据和高价值模型资产不受侵犯。此外，在工程实现层面，受限于嵌入式设备的成本与功耗约束，系统对安全机制引入的性能损耗高度敏感；同时，当前端侧 TEE 方案高度定制化的特性显著抬高了应用的迁移门槛。

针对上述场景挑战与工程约束，EasyAda-trust 的核心设计目标确立为：构建兼顾高安全性与低开销的底层可信基，在实现 CPU 与 I/O 侧数据隔离保护的同时，严格控制内存隔离等机制引入的性能折损，并系统性降低可信执行环境的适配与开发门槛。为确保该设计目标的达成，本文将 EasyAda-trust 的系统需求分解为以下四个核心维度：

1. 安全性需求：安全性涵盖功能安全与数据机密性两个层面。在功能隔离层面，底层架构需确保普通不可信代码中的漏洞或异常行为无法波及更高特权级的系统状态，且单一实体（应用或虚拟机）的崩溃应被严格限制在当前域内，避免引发全局失效。在数据保护层面，系统内所有软件实体必须严格遵循最小特权原则。值得注意的是，传统架构中高特权级软件的无限制访问权必须被剥夺，即底层 Hypervisor 等系统软件不应具备访问上层机密应用数据与代码的权限。

2. 可扩展性需求：系统架构需在软硬件双端具备良好的延展性。在硬件接口层面，方案应支持特定业务场景下定制化异构加速器的接入，以及后续高级指令集扩展（如向量计算扩展等）的平滑演进；在软件架构层面，系统需提供模块化的部署能力，以支撑针对不同负载类型应用的高效开发。

3. 硬件兼容性需求：鉴于 RISC-V 生态的多样性特征，系统方案需规避对非标准硬件特性的过度依赖，从而实现对现有主流商用 RISC-V 硬件平台的有效兼容，并保留对未来先进硬件架构的演进潜力。

4. 生态兼容性需求：底层隔离机制应尽可能对上层应用透明，保持对主流基础软件生态及标准开发工具链的兼容。由此可见，通过复用既有生态（涵盖不可信操作系统与机密虚拟机镜像），可从根本上降低开发人员的适配难度与代码迁移成本。

3.2 EasyAda-trust workflow

在深入剖析 EasyAda-trust 的具体架构设计之前，为清晰界定后续威胁模型中的信任边界，本节首先明确系统生命周期内所涉及的关键角色及其交互 workflow。针对嵌入式系统复杂的供应链特征，本文将该生命周期内的参与实体抽象并归纳为以下四大类：1. 芯片 IP 供应商：负责底层微架构与核心硬件 IP 的设计与授权。2. 硬件设备供应商：基于芯片 IP 完成 SoC 或硬件平台的设计、流片与制造。值得注意的是，该角色不仅负责物理硬件的组装，还负责确立系统的信任根（Root of Trust, RoT），并提供硬件密钥对以及底层安全监视器二进制文件与基础配置环境。3. 基础软件/平台供应商：负责微内核操作系统、Hypervisor 等系统级基础软件的研发、移植与配置。其核心 workflow 是向下适配特定硬件环境，向上提供统一的系统服务抽象。4. 平台用户/应用开发者：依托基础软件供应商提供的软硬件生态与软件开发工具包，根据具体业务场景需求，开发并部署定制化的不可信虚拟机、机密虚拟机及其上层应用负载。

以典型的智能驾驶域控制器场景为例，其完整的工作流及信任链延展过程如下：首先，由 ARM 或 RISC-V 等芯片 IP 供应商提供基础指令集与架构授权；其次，高通或联发科等硬件设备供应商据此制造车载 SoC，并预置不可篡改的硬件信任根与底层安全固件；随后，智能驾驶方案供应商作为基础软件供应商，基于该硬件平台统一部署 EasyAda-trust 的微内核及虚拟化底座，并封装为标准化工具链对外发布；最后，车企的业务部门或第三方算法团队作为平台用户，基于上述工具链实例化不同的虚拟机隔离方案，完成自动驾驶算法或侧载应用的二次开发与部署。由此可见，整个嵌入式机密计算系统通过上述角色的接力协作，最终以软硬协同的形态向终端驾驶员提供服务。需要补充的是，在实际的产业生态中，上述多个生命周期角色可能由同一实体（例如具备全栈自研能力的整车制造商）合并承担。

3.3 威胁模型

基于上一节对 EasyAda-trust 全生命周期中角色的界定，本节明确系统所面临的安全威胁模型。威胁模型将从硬件物理边界、软件信任假设、攻击者能力以及攻击目标四个维度展开定义：

首先，在硬件与物理边界层面，本文假定底层硬件环境的设计与实现符合 RISC-V 架构标准且无固有后门缺陷。在系统发生特权级上下文切换时，所有微架构层的运行轨迹（如分支预测器、缓存状态等）均能被硬件机制正确清除。此外，本文的威胁模型不考虑针对内存、总线及处理器的物理层攻击。其核心原因在于，当前工业界已有成熟的防护机制应对此类威胁（例如锁步故障检测机制、基于纠错码的内存保护以及总线加密协议等）。对于面临严峻物理攻击风险的目标系统，可正交部署上述防御技术，而无需对 EasyAda-trust 的软件架构进行侵入式修改。

其次，在软件信任假设层面，鉴于嵌入式系统的供应链封闭特性，其基础软件（操作系统与 Hypervisor）通常由合规供应商提供。然而，由于此类系统软件代码体量庞大且逻辑复杂，极易成为漏洞攻击的突破口。因此，本文采取保守的安全性假设：假定普通虚拟机、微内核操作系统以及 Hypervisor 的根权限均可能因代码漏洞利用或网络攻击而陷落，从而处于完全被控的不可信状态。值得注意的是，运行于最高特权级的安全监视器作为系统的核心可信基，在架构上被定义为绝对可信；但考虑到传统固件常受内存破坏漏洞威胁，本文假设攻击者具备对安全监视器发起内存越界等软件层攻击的能力。

在攻击者能力层面，由于基础系统软件已被认定为不可信，攻击者不仅能够普通虚拟机内部部署恶意的第三方侧载应用，还可能尝试实例化恶意的机密虚拟机以发起跨域攻击。更为严峻的是，掌控 Hypervisor 的攻击者能够通过其管控的硬件寄存器、共享内存缓冲区、虚拟 I/O 设备，甚至通过篡改机密虚拟机的初始镜像状态与硬件配置文件，实现对目标机密虚拟机全生命周期调度及输入数据的完全操控。

在此极端威胁场景下，软件层攻击者的核心目标主要集中于以下三个维度：（1）篡改受保护机密虚拟机的控制流或数据流执行逻辑，破坏其运行时的完整性；（2）窃取驻留于机密虚拟机内存中的用户隐私数据或高价值模型资产，破坏其机密性；（3）伪造或冒充合法的机密虚拟机实体，向平台所有者或外部依赖组件发起欺骗性交互。

3.4 可信微内核虚拟化方案设计

3.4.1 总体设计

围绕前文定义的威胁模型，EasyAda-trust 在总体架构上采用了一种软硬协同的双层隔离机制。根据信任边界的严格划分，系统的运行环境可解耦为常规执行域与机密执行域两部分：

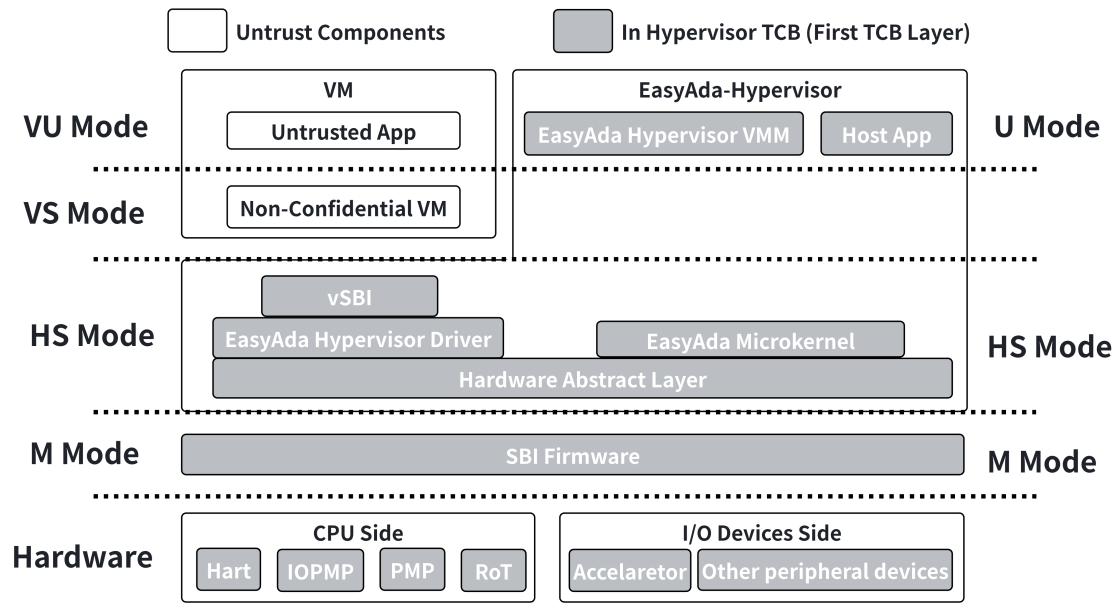


图 3-1 非机密隔离域架构

1. 常规执行域（非机密隔离域）：如图3-1所示，该域主要承载常规的业务负载与基础系统软件，涵盖第三方不可信组件、微内核操作系统 EasyAda 以及虚拟化扩展 EasyAda-Hypervisor。在该域中，不可信组件（如侧载应用与常规虚拟机）可能包含恶意代码，并试图跨界窃取或篡改其他虚拟机的敏感数据。而对于由平台供应商提供的 EasyAda 与 EasyAda-Hypervisor，虽然在常规业务中作为控制中枢，但由于其代码体量庞大且存在潜在漏洞的风险，因此在机密计算的严格视角下，它们被明确排除在核心可信基之外。就功能定位而言，常规虚拟机为上层应用提供统一的基础软件环境，降低开发难度；EasyAda-Hypervisor 负责宏观的资源分配与虚拟机生命周期调度，并向下层提供虚拟 SBI 固件接口；微内核 EasyAda 则是整个常规域的基础底座，负责底层硬件资源的统一分派，并支持高实时性任务的直接部署。

2. 机密虚拟机可信基：如图3-2所示，该部分为 EasyAda-trust 专为机密计算引入的核心 TCB，旨在弥补常规执行域在面对特权级恶意软件时攻击面过大、隔离性不足的架构缺陷。该域构成了系统的安全信任根，涵盖运行于最高特权级的安全监视器（EasyAda-SM）、内嵌的 TEE 安全管理器（TEE Security Manager, TSM），以及运行在独立飞地（Enclave）中的机密虚拟机与可信应用。在协作机制上，EasyAda-SM 是实现物理级执行环境隔离的核心软件；不可信的 EasyAda-Hypervisor 仅能通过安全监视器暴露的受限接口来代理请求对机密虚拟机的管理。基于此隔离架构，机密虚拟机得以向开发者提供透明且统一的机密计算平台，显著降低了安全 AI 等敏感业务的开发门槛。

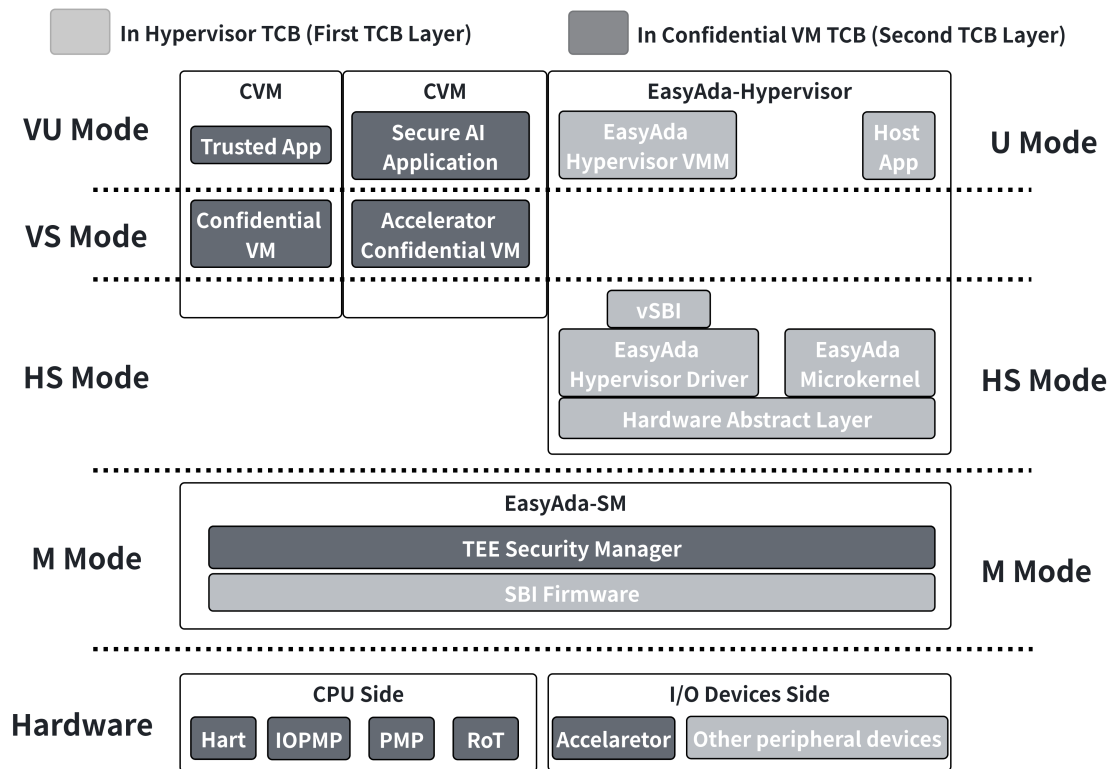


图 3-2 机密执行域架构

为兼顾广泛的设备兼容性与严格的物理隔离要求，EasyAda-trust 设定了明确的架构基准：底层物理硬件必须实现 RISC-V 硬件虚拟化扩展——H 扩展、PMP 以及 IOPMP 扩展。基于这一基准，资源受限的边缘节点可依托轻量级硬件实现基础机密隔离；而对算力要求严苛的端侧场景，则可通过引入自定义指令集扩展与定制化硬件，实现安全与计算性能的弹性扩展。

针对核心组件 EasyAda-SM 的设计实现，EasyAda-trust 采取了多维度的架构加固策略。首先在软件自身安全性上，利用 Rust 的内存安全特性构建安全监视器，从根源上消除传统固件中不安全内存访问的隐患。其次在架构扩展性上，安全监视器采用模块化与分层抽象设计，支持开发者在复用底层组件的基础上，针对不同异构平台定制专属的飞地管理模块。此外，在接口安全性设计上，为避免复杂的中间状态导致安全监视器攻击面膨胀，机密虚拟机管理模块严格收敛了 Hypervisor 的调用权限，仅提供转换（promote）、运行（run）和销毁（destroy）三种原子化、无中间状态的生命周期管控接口。同时，鉴于嵌入式设备通常受限于功耗与成本，难以部署重量级的 IOMMU，具备 DMA 能力的恶意设备极易绕过 CPU 侧的内存隔离边界。为此，EasyAda-trust 在安全监视器内部专门集成并实现了一套 IOPMP 管控模块，以总线级物理内存鉴权机制抵御来自恶意外设的 DMA 攻击^[45]。

如图3-3所示，EasyAda-trust 在复用既有微内核不可信虚拟机调度逻辑的基础

上，通过引入无中间状态的 TSM 代理接口，完成了一套面向机密虚拟机全生命周期的安全状态机流转与统一管理机制。

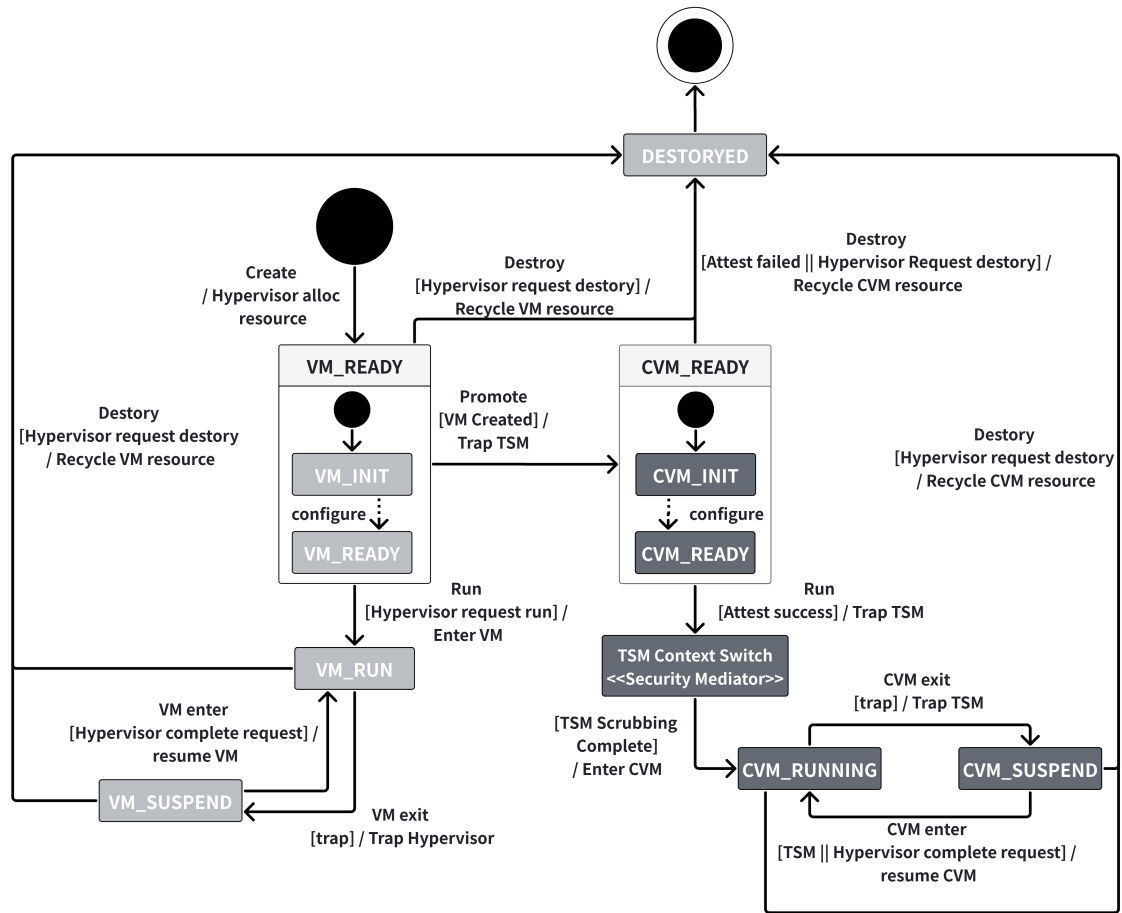


图 3-3 EasyAda-trust 生命周期状态机

虽然 EasyAda-trust 将 EasyAda-SM 和机密虚拟机纳入了机密虚拟机可信基中，但是可信并不等于安全。用户在开发过程中可能因为缺乏测试等原因引入破坏 TEE 安全性的代码漏洞，尤其是不安全的内存访问导致的内存泄漏等问题。针对用户可能引入的代码漏洞，EasyAda-SM 和机密虚拟机分别采取了不同措施以尽可能减少或隔离代码漏洞，避免单一漏洞导致整个 TEE 被攻破。本文开发了内存安全的 EasyAda-SM，并通过分层设计将不安全的内存操作进行封装和限制以降低测试和形式化验证的难度。同时机密虚拟机管理模块为每个机密虚拟机启用了硬件虚拟化扩展，通过二阶段地址翻译隔离不同机密虚拟机的执行环境，避免单一机密虚拟机的安全漏洞影响 TEE 整体稳定性和安全性。具体设计和实现将在后文详细介绍。

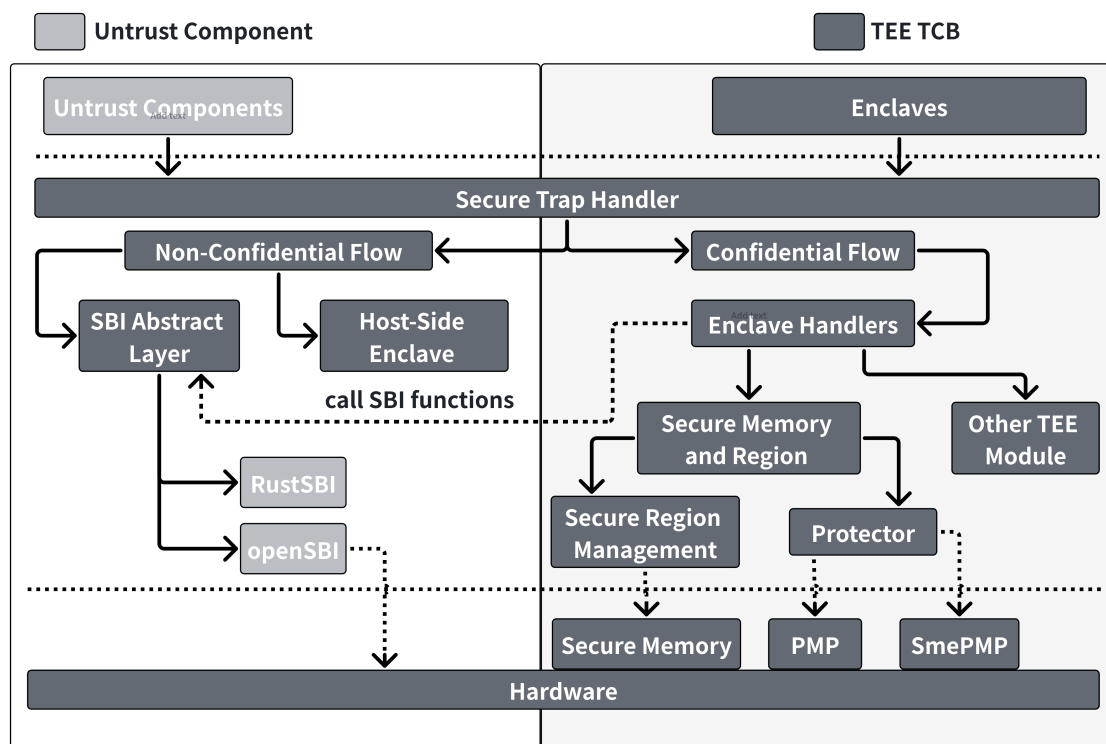


图 3-4 EasyAda-SM 总体架构

3.4.2 内存安全的模块化安全监视器设计

鉴于 RISC-V 架构精简与高度可定制的特性，现有的 RISC-V TEE 方案普遍依赖运行于最高特权级的安全监视器，并结合特定的指令集扩展来提供机密计算原语。然而，当前主流的安全监视器设计面临两大严峻挑战：（1）内存安全性缺失：传统固件多基于 C/C++ 等非内存安全语言开发，易引入越界访问、释放后重用等内存漏洞，从而从根本上危及 TEE 的全局安全性；（2）架构扩展性不足：现有安全监视器往往与特定的 TEE 场景强绑定，设计缺乏通用抽象，难以作为统一的安全底座适配多样化的异构应用需求。

针对上述问题，如图3-4所示，EasyAda-SM 采用 Rust 语言进行底层重构以保障内存安全，并在架构抽象上实施了深度的解耦。值得注意的是，传统方案（如 Penglai 与 Keystone）通常将飞地管理模块作为 SBI 扩展直接硬编码嵌入 SBI 固件中，导致严重的代码耦合。相反，EasyAda-SM 采用了控制反转的架构逻辑：将其作为独立的安全监视器主体，而将底层 SBI 固件降级为静态链接库，并引入了 SBI 能力抽象层。基于此设计，EasyAda-SM 能够通过标准接口调用底层 SBI 原语，且支持灵活替换不同的 SBI 底层实现（如不同芯片厂商的定制固件），从而最大化利用底层硬件的差异化能力。

在模块化设计层面，EasyAda-SM 将 TEE 的共性操作（如内存隔离、密钥管

理、镜像度量与安全异常处理等)解耦为彼此独立的子模块。每个子模块对外暴露标准化的操作原语,有效屏蔽了底层指令集与硬件拓扑的差异。如图3-4所示,开发者既可以针对特定的 TEE 方案快速组装专属的飞地管理模块,也可针对特异性硬件对单一模块进行定制修改。同时,每个模块仅授予完成其特定任务所必需的最小权限集合,从而从架构层面将潜在漏洞的影响限制在单一模块内部。

通过对现有 RISC-V TEE 生态的深入剖析,本文发现:尽管各方案在安全启动、中断路由与 I/O 隔离等策略上趋于一致,但在安全内存的分配与隔离机制上却存在显著分歧。例如, Penglai 追求细粒度的动态内存管理,要求安全监视器具备一定的内存分配能力; Keystone 则采用一次性静态分配策略,安全监视器仅负责物理保护而不介入动态分配;而 ACE 的 TSM 为了适配机密虚拟机的二阶段页表构建,专门设计了基于 Rust 所有权机制的页分配器以规避重复分配风险。为兼容上述多样化的内存管理范式, EasyAda-SM 建立了一套层次化的统一内存抽象模型,将系统内存划分为以下几类:

1. 通用区域与通用内存:指未被安全监视器标记与保护的片上/片外物理内存。为最大化系统的整体内存利用率,该区域默认归属于不可信宿主 OS 进行统一的宏观调度。

2. 安全区域:指由安全监视器显式标记并实施硬件级保护的连续物理内存域。该区域从通用区域中动态或静态划分而来,安全监视器会为其绑定专属的硬件隔离单元(如 PMP 条目)。在安全区域内部,进一步细分为两类状态:

- 2.1 安全内存:已受安全监视器保护,但尚未分配给具体飞地实例的空闲内存。此类内存由安全监视器进行集中管控。

- 2.2 飞地内存:已受保护且已正式授权给特定飞地实例使用的私有内存。需要强调的是,该内存的访问边界由安全监视器固化,但在飞地生命周期主动终止或释放前,安全监视器自身亦无权对其进行强行回收或重分配。

由此可见,图3-5清晰展示了 EasyAda-SM 的安全内存状态流转模型。在该模型中,通用内存代表初始状态(未初始化且不可信),飞地内存代表最终状态(被专属独占且绝对机密)。而引入安全内存这一中间状态的核心价值在于:它作为通用内存与飞地内存转换的安全缓冲层,在剥离不可信操作系统的越权访问风险与避免安全监视器过度干预飞地内部管理之间取得了合理的架构折中,从而在有效收敛安全监视器攻击面的同时,显著提升了系统的整体内存周转效率。

3.4.3 安全特性设计

围绕前文确立的威胁模型与架构需求,本节将系统性地阐述 EasyAda-trust 针对机密虚拟机所设计的核心安全机制。

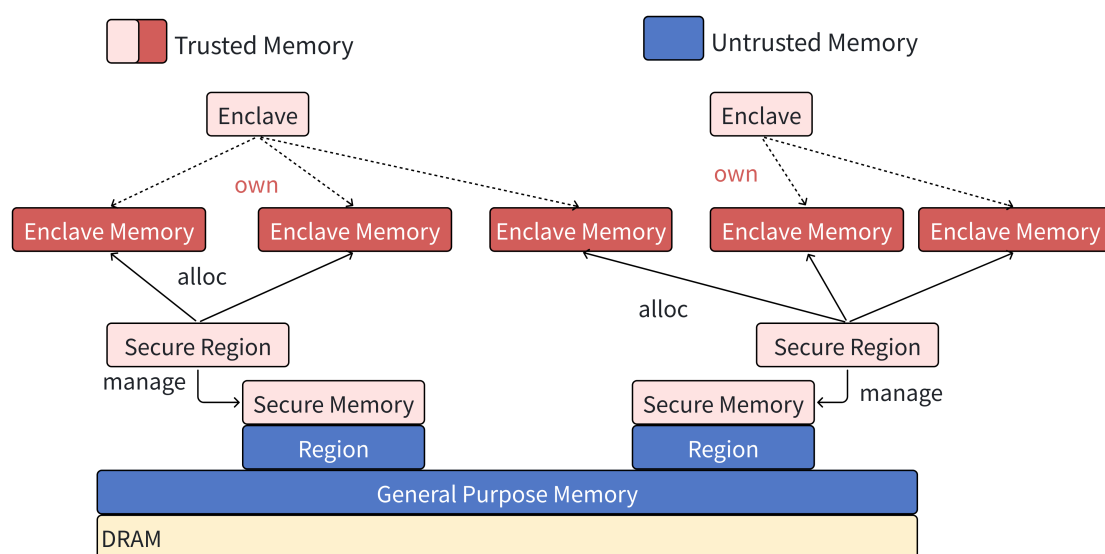


图 3-5 EasyAda-SM 的内存管理模式

3.4.3.1 安全启动流程设计

安全启动的核心目标是构建一条自底向上的可信链（Trust Chain, TC）。该机制旨在确保仅有由授权开发者签名且未经篡改的可信组件能够被加载至 TEE 中运行；同时，保证高价值的安全应用仅在经过严格双向度量验证的合法平台上完成实例化，并安全地获取敏感 I/O 数据。

为实现上述目标，该安全启动机制依赖于两类非对称密钥对的协同工作：（1）硬件供应商信任背书的签名密钥对；（2）各物理节点独有的设备密钥对。具体而言，签名私钥由硬件供应商离线安全保管，专用于对安全监视器的固件镜像生成数字签名；而对应的签名公钥则被硬编码于每台设备底层的信任根中，以便在系统上电时对安全监视器进行完整性校验。另一方面，设备密钥对作为物理硬件的唯一密码学身份标识，其公钥对开发者公开，用于加密绑定至特定设备的机密信息或用户秘密；设备私钥则被严格封存在设备的受保护存储区内，仅允许在信任链验证通过后，用于解密相关的载荷数据。

信任根是整条可信链构建的物理起点。依据 EasyAda-trust 的威胁模型，系统将硬件供应商提供的物理安全模块（如 TPM）及固化代码（驻留于 eFUSE 或不可篡改的 ROM 中）严格定义为硬件信任根。在实际的启动时序中，当硬件设备上电复位后，RoT 首先启动安全引导序列。在此阶段，RoT 负责对拟加载的安全监视器镜像计算当前的哈希度量值，并利用预置的签名公钥对其数字签名进行密码学匹配。唯有在度量值与签名完全吻合的前提下，控制权才会被安全移交至安全监视器，从而开启下一阶段的系统初始化流程。

在控制权向上层系统软件移交的过程中，由于 EasyAda-trust 的架构设计已将 Hypervisor 与不可信虚拟机明确排除在核心可信基之外，因此在默认的安全策略下，安全监视器不对 Hypervisor 进行强制的度量与验证。这一架构折中在保障核心信任链精简的同时，也赋予了系统高度的灵活性：平台供应商与硬件厂商可根据具体的业务合规需求，自行扩展信任链，以实现 Hypervisor 及不可信操作系统的可选验证。

当系统进入运行态后，Hypervisor 需要承担动态部署机密虚拟机的职责。然而，在本文定义的威胁模型中，既存在软件层攻击者尝试拉起恶意机密虚拟机以发起权限提升攻击的风险，也存在被攻陷的 Hypervisor 强行部署合法机密虚拟机以窃取内部状态的威胁。为有效应对此类跨特权级攻击，机密虚拟机的动态部署过程必须强制执行机密虚拟机镜像与底层硬件环境的双向信任验证。为防范恶意平台强行探测内部状态，EasyAda-trust 要求机密虚拟机开发者必须使用设备公钥对包含敏感代码与数据的载荷进行加密封装，从而确保该载荷仅能在身份合法且未被篡改的授信硬件环境中被解密与实例化。

针对上述双向验证需求，传统的云服务厂商通常依赖远程证明（Remote Attestation），即通过远端的可信第三方服务器对当前节点环境及机密虚拟机镜像进行在线校验。相反，鉴于嵌入式系统通常部署于网络连接有限或完全物理隔离的边缘环境中，高度依赖持续网络连接的远程证明机制存在显著的可用性瓶颈。因此 EasyAda-trust 默认采用本地证明^[46]（Local Attestation）协议，通过硬件信任根与安全监视器的本地密码学交互完成机密虚拟机与物理硬件环境的双向信任度量，从而坚实保障了端侧机密计算的独立性与可靠性。

3.4.3.2 安全 CPU 虚拟化设计

硬件虚拟化扩展通过引入虚拟化专用特权级，有效实现了普通虚拟机之间以及虚拟机与宿主 OS 之间的 CPU 上下文隔离。然而，尽管 EasyAda-Hypervisor 能够利用该机制确保普通虚拟机间 vHart 的横向隔离以避免同级上下文泄漏，但其在原生架构上无法防御来自更高特权级系统软件的纵向越权访问（即无法实现 Hypervisor 自身与机密虚拟机间的绝对上下文隔离）。针对这一安全缺陷，EasyAda-trust 在 Hart 的调度机制中引入了机密与非机密执行流的抽象，旨在确立不同执行环境间的物理级上下文边界。

在底层异常处理的路由机制上，对于不可信虚拟机，系统默认将大多数系统级异常委派至 Hypervisor 进行处理，以便其监控和调度虚拟机的常规敏感行为。相反，对于机密虚拟机，若沿用该机制将异常直接委派给不可信的 Hypervisor，将不可避免地导致机密虚拟机内部的通用寄存器与运行时状态等高价值上下文信息对

Hypervisor 物理裸露。因此，必须引入 TSM 来强制拦截并代理机密虚拟机的所有敏感行为与硬件陷入。

在生命周期管控中，为遵循资源分配权与数据访问权解耦的原则，机密虚拟机的底层物理 Hart 分配及其与 vHart 的绑定关联仍由 Hypervisor 负责完成。值得注意的是，为彻底阻断不可信组件的篡改与窃听路径，在控制流正式切换至机密虚拟机上下文之前，TSM 会动态重构硬件异常委派寄存器，强制将原本应由 Hypervisor 处理的敏感异常直接路由并陷入至系统的最高特权级 M 态。陷入 M 态后，执行流首先触发轻量级上下文切换并交由 TSM 进行安全校验与处理；若该异常最终确需不可信 Hypervisor 介入协助（例如特定物理 I/O 设备的外部调度），TSM 将强制执行的安全上下文清洗操作——即在将控制权移交至 Hypervisor 执行环境前，彻底擦除物理 CPU 寄存器与微架构缓存中的所有机密残留状态，从而在物理层面确保机密虚拟机运行时上下文的绝对隐匿。

3.4.3.3 安全内存虚拟化设计

内存虚拟化的核心职责在于为虚拟机分配物理内存资源，并构建与维护 GPA 至 HPA 的二阶段页表映射。针对机密计算场景，EasyAda-trust 进行了严密的安全扩展。

不可信组件与机密虚拟机间内存隔离：为确保机密虚拟机免受不可信组件的窥探，机密虚拟机严禁直接使用由不可信 Hypervisor 分配的常规内存作为其运行时物理内存。因此，EasyAda-trust 引入了安全区域与安全内存的核心抽象。安全内存特指由可信安全监视器直接管控、并依托底层硬件（如 PMP）实施强制物理隔离的专有内存空间。任何承载机密虚拟机敏感代码与数据的载体均必须驻留于此，以绝对保障其机密性与完整性。在管理粒度上，系统以安全区域为基本单元，每个区域映射一段连续的物理地址，并绑定专属的硬件保护规则。在内存状态流转上，安全内存由常规不可信内存动态转换而来：Hypervisor 可主动申请一段连续的不可信内存并捐献给安全监视器；安全监视器接管后，立即对其执行物理级隔离并标记为安全内存。自此，任何不可信系统组件均被从硬件层面剥夺对该区域的直接访问权限。

机密虚拟机间内存隔离：尽管各机密虚拟机均驻留于安全内存中，但由于机密虚拟机内部代码漏洞可能引发安全越界，且受控的 Hypervisor 可能恶意实例化构造的机密虚拟机以对目标机密虚拟机发起同级攻击，因此，机密虚拟机之间同样必须实施严格的横向隔离。值得注意的是，在现有的 ACE^[33] 架构中，所有机密虚拟机共享同一庞大的安全区域，仅依托硬件虚拟化的二阶段页表来实现内部隔离。尽管该机制提供了一定程度的安全防护，但在嵌入式场景中，静态共享单块安

全区域难以满足系统对安全内存动态收缩与扩容的细粒度需求。为克服上述局限，EasyAda-trust 支持安全区域的动态增删与回收机制，并允许一组相互信任的机密虚拟机共享特定安全区域以优化内存利用率；而对于相互隔离的机密虚拟机实例，则严格依赖各自独立的二阶段页表映射机制实现地址空间的横向物理阻断。

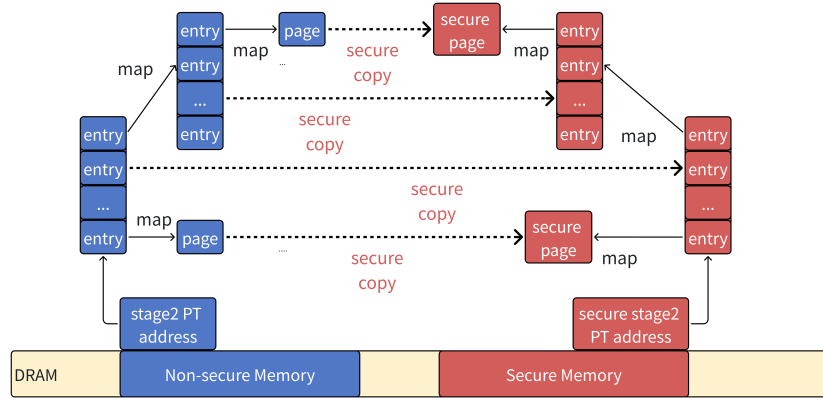


图 3-6 机密虚拟机二阶段页表初始化

机密内存虚拟化：在机密虚拟机实例的初始创建阶段，其流程与普通虚拟机相似，暂由 Hypervisor 完成初始内存分配与伪二阶段页表的构建。然而，当系统需要将该实例正式提升为机密虚拟机时，Hypervisor 驱动必须通过 TSM 暴露的专有接口，将伪二阶段页表与 FDT 的基址交付给 TSM。

如图3-6所示，在此接管流程中，TSM 首先遍历 Hypervisor 预先维护的页表结构，并在安全内存池中重新分配物理页，旨在为机密虚拟机构建一份完全处于 TSM 控制下的、安全的影子二阶段页表。同时，TSM 按序申请安全内存以安全拷贝原不可信内存中的初始数据载荷，从而从根本上阻断 Hypervisor 实施检查时间与使用时间篡改攻击的可能。随后，TSM 对机密虚拟机重构后的二阶段页表进行遍历，并对 FDT、内核及初始内存盘计算初始哈希度量值。唯有当该运行时度量值与机密虚拟机开发者预置的基准度量值严格匹配时，方可证实该虚拟机环境符合预期且未遭 Hypervisor 篡改。验证通过后，系统方允许拉起机密虚拟机，并向其安全解封由开发者提供的、用于后续 I/O 数据加解密的根密钥。

安全 MMIO 区域隔离：鉴于 RISC-V 采用统一编址模式，外设 MMIO 空间同样被纳入内存物理地址统一管理。因此，EasyAda-trust 复用上述安全区域机制，实现了设备 MMIO 区域与不可信组件的物理隔离。在此基础上，系统为安全区域引入了类型标识符，以精确区分常规 RAM 区与 MMIO 区，并支持向下细分为特定的安全设备类型，从而实现总线级的细粒度鉴权。

不可信共享内存：除受保护的私有安全内存外，机密虚拟机与不可信宿主系统之间必须保留数据交换通道（尤其是针对块设备等高吞吐 I/O 虚拟化场景）。为

此, EasyAda-trust 支持机密虚拟机向宿主 OS 申请“不可信共享内存”。该内存由宿主 OS 分配, 并分别由 Hypervisor 和 TSM 映射至各自的可见地址空间。由于该内存区域本质上处于不可信域, 机密虚拟机在执行读写操作时, 必须严格执行边界检查, 并主动调用密码学组件对敏感输出进行加密、对输入进行完整性校验, 以防范数据泄露与恶意注入。

EasyAda-trust 在架构上并未支持在不同机密虚拟机之间直接分配可信共享内存。这一设计基于以下三个考量: (1) 在嵌入式与域控制器场景下, 应用负载通常在系统构建期便已静态确定, 架构设计上应避免将强耦合、高频数据交换的逻辑强行拆分至不同的机密虚拟机实例中; (2) 作为系统的核心可信基, TSM 必须恪守极简设计原则, 支持复杂的安全内存共享映射将不可避免地导致可信基体积与逻辑复杂度的剧增; (3) 对于低频的跨机密虚拟机数据同步, 完全可通过前述的不可信共享内存 + 应用层端到端加密的安全信道机制予以替代。综上所述, EasyAda-trust 在设计层面舍弃了跨机密虚拟机的可信共享内存机制。

3.4.3.4 安全中断虚拟化设计

硬件中断虚拟化的核心安全目标在于确保中断控制流与数据负载的机密性与完整性。具体而言, 中断安全可划分为中断发送(注入)安全与中断接收(响应)安全两个维度。两者均严格要求目标虚拟机的中断事件免受不可信组件的拦截、伪造或篡改。

在中断发送机制上, 根据 RISC-V 架构规范, 跨核处理器间中断——软件中断最终必须在 M 态执行。针对不可信虚拟机, 其发送中断的请求首先陷入至 Hypervisor 驱动层, 由 Hypervisor 解析目标 vHart, 随后通过 SBI 调用陷入底层 SBI 固件完成物理发送。在此传统流程中, Hypervisor 对虚拟机的中断路由具有绝对管控权。然而, 在机密计算场景下, 不可信的 Hypervisor 无权干预机密虚拟机的内部通信。因此, 机密虚拟机发出的中断请求必须在硬件配置上旁路 Hypervisor, 直接触发陷入 TSM。进一步地, TSM 内部需专门维护机密虚拟机 vHart 至 Hart 的安全映射表, 以确保中断精确且机密地送达目标执行单元。

在中断接收机制上, 系统需对普通虚拟机与机密虚拟机采取差异化的委派策略。对于不可信虚拟机, 底层安全监视器通常将除外部中断外的所有中断默认委派至 HS 态, 交由 Hypervisor 统一调度。得益于 EasyAda-Hypervisor 的静态分区策略, Hypervisor 可进一步将虚拟时钟中断直接委派至 VS 态, 允许虚拟机内部闭环处理, 从而规避频繁陷入 HS 态引发的性能损耗。至于软件中断等其他异步事件, 则需陷入 Hypervisor 中进行处理, 由其决定是否向目标 vHart 注入中断或切换执行上下文; 由于该调度过程完全在不可信域内完成, 故无需执行严格的 CPU 上下文

清洗。

相比之下，机密虚拟机的中断响应机制面临着更为严苛的隔离要求。除已安全委派至 VS 态的时钟中断外，针对机密虚拟机的其他中断必须在微架构层面撤销向 HS 态的委派，强制其直接陷入 M 态的 TSM 中进行安全鉴权。TSM 接管后，将根据内建的安全策略决定是否向机密虚拟机注入该中断。值得注意的是，若该中断处理不可避免地需要切换至 Hypervisor 上下文（如请求底层异步 I/O 服务），TSM 必须在控制权移交前，彻底清洗机密虚拟机的物理寄存器与微架构状态，以杜绝任何敏感数据的残余泄露。相应地，机密虚拟机的所有中断控制与配置寄存器均由 TSM 独占接管。

除了保障中断收发的机密性与完整性外，中断虚拟化设计还必须应对潜在的可用性破坏如 DoS 攻击。根据发起方的不同，DoS 攻击可分为机密虚拟机针对宿主 OS 的攻击，以及宿主 OS 针对机密虚拟机的攻击。对于前者，由于不可信宿主 OS 在架构上仍掌握宏观的物理资源调度权，其可通过 TSM 提供的合法生命周期接口直接终止并销毁恶意的机密虚拟机实例。对于后者（即宿主 OS 作为攻击者），则需根据攻击路径作进一步的防御划分：其一，若宿主 OS 通过恶意生成海量伪造中断请求，企图迫使频繁陷入 TSM 从而耗尽其计算资源，TSM 可通过引入中断监控与限流机制，对源自宿主 OS 侧的高频异常中断进行阈值管控和静默屏蔽；其二，若宿主 OS 采用拒绝响应策略（即完全丢弃来自的合法 I/O 或调度请求），鉴于宿主 OS 在当前架构中本就承担宏观资源分配者的合法角色，防范此类基于底层资源剥夺的 DoS 攻击已超出可信基的安全职责范畴，故 EasyAda-trust 在架构设计上将其明确排除在防御目标之外。

3.4.3.5 安全 I/O 虚拟化设计

EasyAda-Hypervisor 原有的 I/O 虚拟化方案主要解决了两个基础隔离问题：（1）不同虚拟机之间数据流与控制流的横向隔离；（2）确保同一物理时刻，单一外设仅能由唯一授权的软件实体（虚拟机或 Hypervisor）直接访问。

然而，在机密计算架构下，由于 Hypervisor 已被移出可信基，上述机制已无法保障机密虚拟机使用外部设备时的绝对安全。为此，EasyAda-trust 在 I/O 虚拟化层面必须额外构建以下两大核心安全能力：

1. 外设可信身份鉴权：必须建立严格的设备准入机制，在机密虚拟机正式与外部硬件建立连接前，对其进行可信度量与验证，从根源上阻断恶意或仿冒外设发起的注入攻击与数据窃取。

2. 端到端 I/O 数据机密性与完整性保护

- 2.1 CPU 侧数据安全性：在通过驱动栈访问外设的过程中，必须依托密码学

加密、设备独占映射及数据哈希校验等手段，确保敏感输入输出数据不被不可信 Hypervisor 拦截或篡改。

2.2 设备侧防护：鉴于具备 DMA 能力的外设能够物理绕过 CPU 内置的 MMU/PMP 隔离单元，恶意外设极易被操纵以直接读写的私有安全内存。因此，必须在总线层级引入物理内存访问控制机制，严格限制各类外设的 DMA 寻址边界。

针对上述 CPU 侧的数据保护需求，业界通常存在三种演进思路，它们在安全性与性能开销上呈现出不同的架构折中：

1. 基于纯加密的数据保护：如图3-7(a)所示。由于大多数通用设备由宿主 OS 集中管控，最直接的方案是要求机密虚拟机利用内部安全密钥，对所有出栈的 I/O 载荷进行软件级加密与哈希度量，随后通过不可信的共享内存与 VirtIO 协议栈，交由宿主 OS 代理完成底层硬件读写。

该模式极其适用于网卡、块设备等无需明文感知的纯传输型外设。其优势在于，任何驻留于 Hypervisor 层的窃听者仅能获取密文，有效保障了通道机密性。然而其局限性也十分显著：一方面，依赖不可信组件代理传输，强制要求应用层引入复杂的超时与重传机制以应对完整性破坏；另一方面，高频的软件级密码学计算将引入极大的性能惩罚，且完全无法适配要求明文输入的底层异构计算加速器。

2. 基于临时设备授权的数据保护：如图3-7(b)所示。该方案将设备的调度权从 Hypervisor 上收至 TSM。TSM 在不可信组件与机密虚拟机之间实施严格的外设分时授权。

该模式适用于全局唯一且需频繁切换归属权的底层硬件。当机密虚拟机获得短期授权时，其拥有对该设备的绝对独占权，可直接以明文形态进行高速读写；当授权被剥夺或转移时，TSM 将负责彻底清洗设备的内部微架构状态（如清空缓冲区），以切断残留数据的泄露路径。尽管该模式免除了密码学开销，但其需要对底层内核驱动引入复杂的时分复用上下文切换逻辑，易导致系统实时性降级与较高的工程适配成本。

3. 基于硬件分区的数据保护：如图3-7(c)所示。若目标硬件平台具备冗余外设或专为安全场景设计的隔离硬件，系统可借鉴静态分区的理念，在物理层面将特定外设（如安全 UART、独立 Flash 区域）硬连线或永久分配给机密虚拟机独占使用，从而在物理电路上彻底切断不可信组件的访问路径。

此模式是面向生物特征传感器（如指纹/面部识别）或专用硬件加速器的最优解。由于设备被独占映射至机密虚拟机的物理地址空间内，机密虚拟机可实现免加密的零拷贝明文通信。其通信链路完全独立，I/O 效率极高。其缺陷在于对底层物理硬件的资源冗余度要求较高，缺乏动态调配的弹性。

综合评估上述三种机制的优劣，结合嵌入式业务特征，EasyAda-trust 将外设划

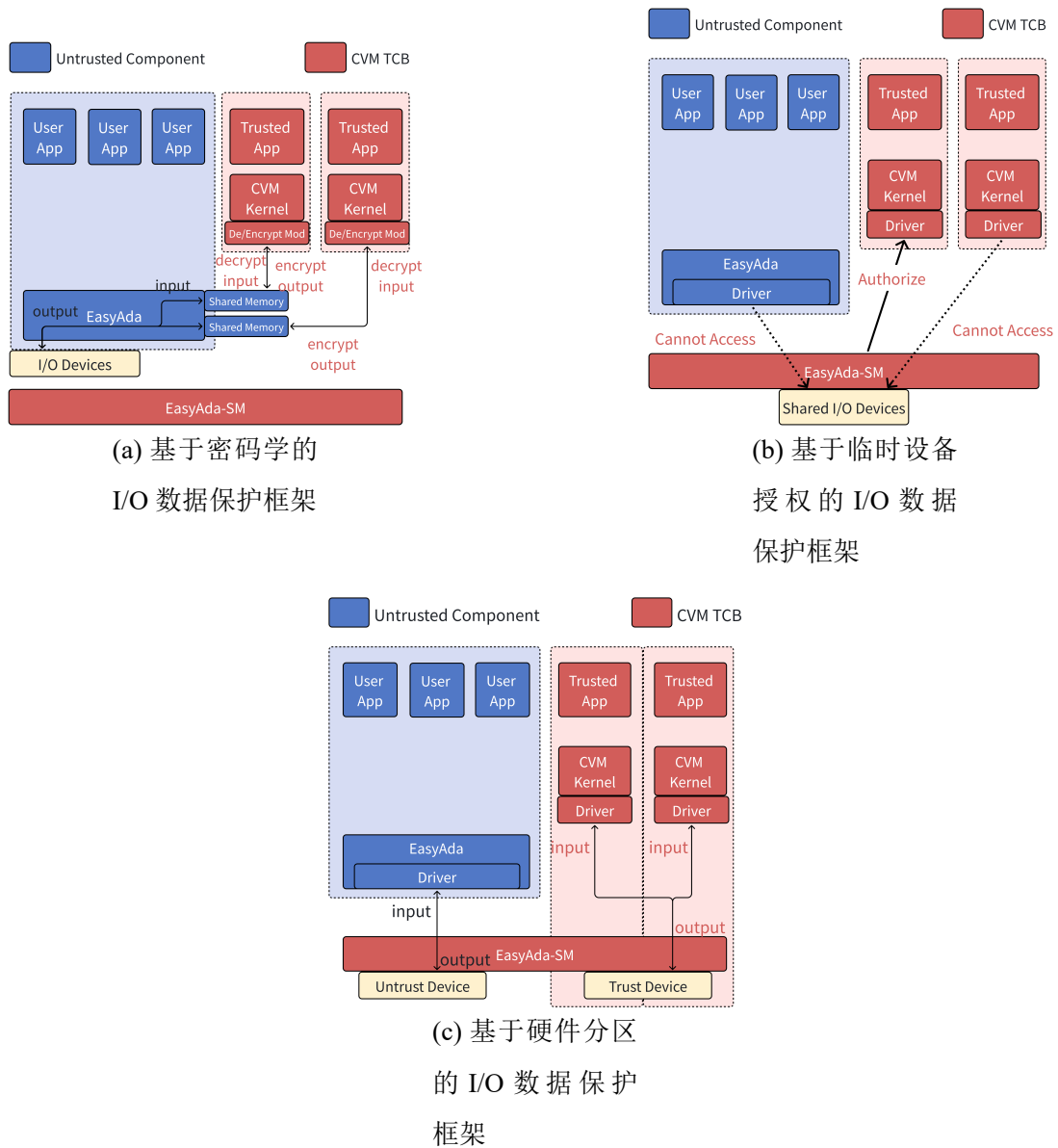


图 3-7 机密虚拟机的 I/O 数据保护机制

分为普通设备与安全设备两类并针对不同设备采取了不同的保护机制：

对于普通 I/O 设备（如常规网卡、磁盘等对载荷内容不敏感的传输部件），系统直接将其授权给宿主 OS 集中管理。机密虚拟机采用纯密码学保护方案，通过不可信共享内存机制委托宿主 OS 代理 I/O 请求，以此简化设备管理逻辑并最大化设备复用率。对于安全设备（如指纹采集器、AI 硬件加速器等对明文数据极度敏感的部件），系统采用硬件分区方案。机密虚拟机将静态独占此类设备的 MMIO 与缓冲区，由 TSM 在硬件底层实施物理隔离。为避免复杂的运行时设备发现带来的攻击面，EasyAda-trust 要求硬件开发商在流片或集成阶段，将可信外设的白名单固化于硬件信任根或静态绑定至 TSM 中。机密虚拟机仅需在安全启动阶段比对该拓

扑清单，即可建立对底层外设的身份互信。

在解决 CPU 侧数据保护后，针对设备侧的保护，EasyAda-trust 深度整合了 RISC-V 架构的 IOPMP 硬件扩展。为规避频繁动态切换 IOPMP 规则带来的系统性能抖动，TSM 采取了静态总线路由策略：仅对经过严格鉴权的安全设备开放其专属安全内存域的 DMA 写权限，而将所有未分类的普通外设严格禁锢在不可信宿主内存域内。

考虑到部分端侧芯片出于极简功耗考量，并未集成复杂的 IOMMU，导致具备 DMA 能力的外设只能直接寻址系统 HPA。在此类受限环境中，机密虚拟机直接向外设下发其内部使用的 GPA 将不可避免地引发总线越界异常或数据损坏。为弥补这一硬件架构短板，EasyAda-trust 允许机密虚拟机在准备大批量 DMA 传输前，主动向 TSM 发起批处理地址转换请求。TSM 介入并在安全域内完成 GPA 至 HPA 的映射计算，再将正确的物理地址装载至硬件加速器寄存器中。通过单次陷入完成大块内存的地址映射，在确保隔离安全的前提下，降低了因缺失 IOMMU 而带来的性能惩罚。

基于上述坚实的软硬协同 I/O 安全基础，下文将深入阐述 EasyAda-trust 针对异构硬件加速器所构建的机密计算运行环境。

3.4.3.6 安全异构计算环境设计

相较于常规的传输型 I/O 设备，硬件加速器（如 NPU、GPU 等）不仅集成了海量的计算与存储单元，且承担着高度性能敏感的密集型计算任务（如 AI 模型推理）。同时，在复杂的嵌入式系统中，此类昂贵的算力资源通常需要在多个跨执行环境的软件实体间进行时分复用。前文虽然探讨了基于纯密码学与设备静态独占的 I/O 数据保护框架，但在硬件加速器场景下面临着以下局限性：

1. 纯密码学方案的吞吐开销：在受限于面积与功耗（PPA）的端侧嵌入式场景中，硬件加速器通常并未内置专用的机密计算密码学引擎。若完全依赖主机侧 CPU 进行软件级的加解密与度量，将引发灾难性的性能惩罚。例如，在处理 AI 训练中的大规模梯度同步时，繁重的密码学操作会迅速耗尽显存控制器的有效带宽与 CPU 的时钟周期，导致严重的计算延迟。据 Lee^[47] 等人的量化研究表明，该软件加密开销导致分布式数据并行训练的迭代时间平均激增 8 倍，极端负载下甚至高达 41.6 倍。更为严峻的是，密码学基于固定块的运算模式与加速器内部非连续、细粒度的内存访问特征存在天然的粒度错配，这将进一步放大系统的整体延迟。

2. 静态独占模式的资源复用困境：在高度集成的智能嵌入式场景中，往往需要并发部署多个 AI 模型以服务于不同的业务域。例如，在智能座舱与自动驾驶域融合的架构中，车载娱乐系统的 AI 交互（运行于不可信域）与底盘控制环境的感

知推理（运行于机密域），均需竞争使用同一块硬件加速器。因此，传统的将物理设备静态永久绑定至单一机密虚拟机的独占模式，无法满足端侧算力弹性共享的客观工程需求。

根据硬件加速器与 CPU 的耦合程度，业界通常将其划分为三类：（1）指令集扩展（直接嵌入 CPU 流水线）；（2）片上协处理器（与 CPU 共享物理内存，经 MMIO 控制）；（3）片外离散加速器（具备独立显存，经 PCIe 互连，多见于服务器端）。为严格控制功耗与 BOM 成本，当前主流的端侧 SoC 普遍采用第一或第二种形态，即加速器以片上协处理器的模式集成，遵循统一内存架构。这一物理前提决定了，安全架构不仅需要物理隔离加速器的 MMIO 控制寄存器区域，还必须严密保护其从系统主存中动态分配的数据缓冲区。

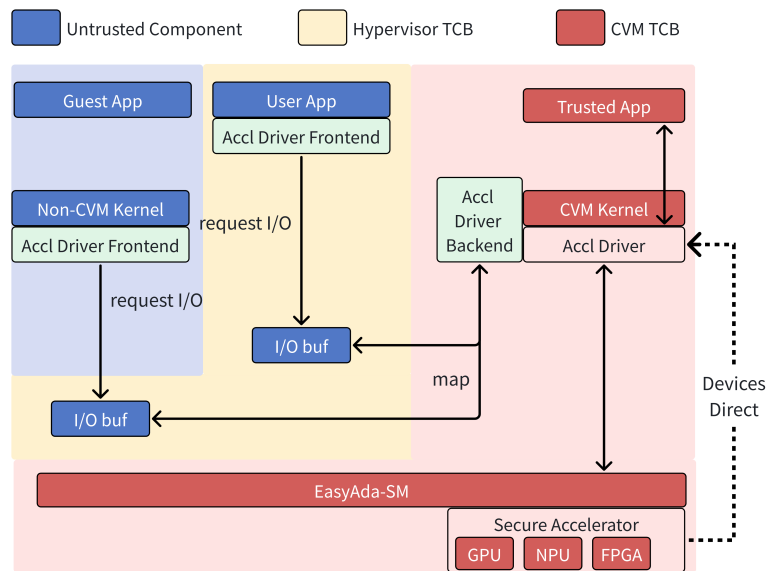


图 3-8 硬件加速器管理机密虚拟机设计

基于上述需求分析与物理前提，如图3-8所示，EasyAda-trust 引入了专用的加速器管理机密虚拟机。该设计的核心系统级考量在于：若将体量庞大且极其复杂的加速器驱动软件栈直接整合至 TSM 中，将导致可信基的急剧膨胀，进而引入较大的漏洞攻击面。得益于独立代理机密虚拟机的引入，系统不仅在架构上维持了底层 TSM 的极简与绝对安全，开发者更能无缝复用现有成熟的 Linux/RTOS 加速器驱动生态，免除了代价高昂的驱动移植工作。

在具体的数据流保护机制上，CPU 侧与外设侧协同实现防御。在 CPU 侧，TSM 通过 PMP 硬件原语实施强制拦截，确保加速器的 MMIO 区域与不可信组件绝对物理隔离；同时，由于加速器驱动直接从该机密虚拟机内部的安全内存池中动态分配数据缓冲区，该缓冲区天然继承了机密虚拟机安全区域的内存隔离保障。在外设侧，TSM 协同 IOPMP 扩展实施白名单鉴权，授予且仅授予该硬件加速器访问

上述专属安全缓冲区的 DMA 权限。通过这一系列的收口管控，从根本上杜绝了因 DMA 配置越界而导致的内存污染。至此，在全链路硬件隔离的庇护下，机密虚拟机得以直接以零拷贝、明文的形态与硬件加速器进行数据交互，避免了高昂的软件加解密开销。

该架构进一步通过半虚拟化技术实现了加速器在多域间的时分复用。此时，加速器管理机密虚拟机转换为半虚拟化加速器设备后端，向下层不可信组件或其他业务域暴露出虚拟硬件加速器接口。不可信组件与加速器机密虚拟机之间的虚拟设备发现机制，在系统构建期即通过读取受保护的硬件设备树配置文件静态确立。鉴于 EasyAda-trust 的防御边界是单向保护机密虚拟机免受外界侵害，不对不可信组件自身的 I/O 安全性作任何承诺，因此，不可信组件向代理机密虚拟机提交的推理任务仅通过常规的不可信共享内存进行低开销的数据投递，且系统并不强制施加任何额外的加密或完整性哈希校验，从而在保障核心机密资产安全的前提下，优化异构算力的全局吞吐量。

3.5 本章小结

本章系统论述了面向端侧场景的 RISC-V 微内核机密虚拟化方案 EasyAda-trust 的总体架构与核心安全机制。在底层架构层面，本方案以 EasyAda 微内核与 Hypervisor 作为常规资源调度支撑，明确将 Hypervisor 排除在核心信任边界之外，并引入由 Rust 重构的安全监视器 EasyAda-SM。该设计通过标准化的安全内存原语抽象，有效收敛了系统的可信基，从源头消除了传统底层固件的内存安全隐患。在全生命周期防护层面，本方案自底向上构建了严密的机密虚拟机隔离体系：首先，依托本地证明确立了安全启动的双向信任链；其次，结合机密执行流抽象与二阶段页表重构，实现了 CPU 微架构状态与物理内存的强隔离，并设计了旁路不可信组件的安全中断路由；最后，针对端侧异构算力需求，方案融合 IOPMP 硬件扩展，构建了基于加速器管理 CVM 的加速器管理机制。上述综合设计在规避高昂软件密码学开销的前提下，成功实现了高价值异构硬件在多执行域间的高效、安全复用，为端侧机密计算提供了坚实的系统级底座。

第四章 内存安全的模块化安全监视器实现

4.1 内存安全固件

为了消除安全监视器中不安全的内存访问，EasyAda-SM 使用适合嵌入式场景的内存安全语言 Rust 实现，通过静态的编译时内存检查消除代码中可能存在的不安全内存访问，避免了昂贵的运行时内存检查和回收。EasyAda-SM 基于 RustSBI^[48] 而非基于内存不安全语言 C 的 SBI 固件（如 OpenSBI），进一步消除了 SBI 固件中不安全的内存访问。

4.2 安全区域和内存管理层实现

安全区域和内存管理层的目标是实现不可信组件与安全内存间隔离的同时提高系统整体的内存利用率。如图4-1所示，安全区域和内存管理层以安全内存管理器作为安全内存的顶级管理单元，对外提供安全内存管理接口，每个安全内存管理器包含多个安全区域（Secure Region）和一个安全内存保护器。

每个安全区域关联一块连续内存区域和一个内存分配器（allocator），内存分配器对应一套内存管理算法实现。严格来说安全区域管理的内存存在被强制隔离之前并不属于安全内存，因为不同 TEE 方案对于安全内存的保护时机存在较大差异，因此 EasyAda-SM 将安全内存保护器和 allocator 分离，由安全内存管理器自行决定内存的保护时机，为了方便叙述，后文以安全内存代指安全区域管理的内存区域。

安全内存保护器则负责管理用于隔离安全内存的安全硬件如 PMP，并对外提供内存保护接口。安全内存保护器的保护粒度是安全区域，主要目标是隔离不可信组件和安全内存，并辅助实现飞地间内存隔离。

为了在同一飞地管理模式提供统一的安全内存视图，每个飞地管理模块通常使用一个全局的安全内存管理器实现统一的安全内存管理，并使用每个安全区域独立的 allocator 实现精细化的内存管理。

根据上一章中介绍 EasyAda-SM 的分层设计思想，安全内存管理器抽象了安全内存管理原语，allocator 抽象了内存管理原语，而安全内存保护器则抽象了内存保护原语，三者共同实现对安全内存生命周期的管理。下面将按序说明每个原语的具体实现。

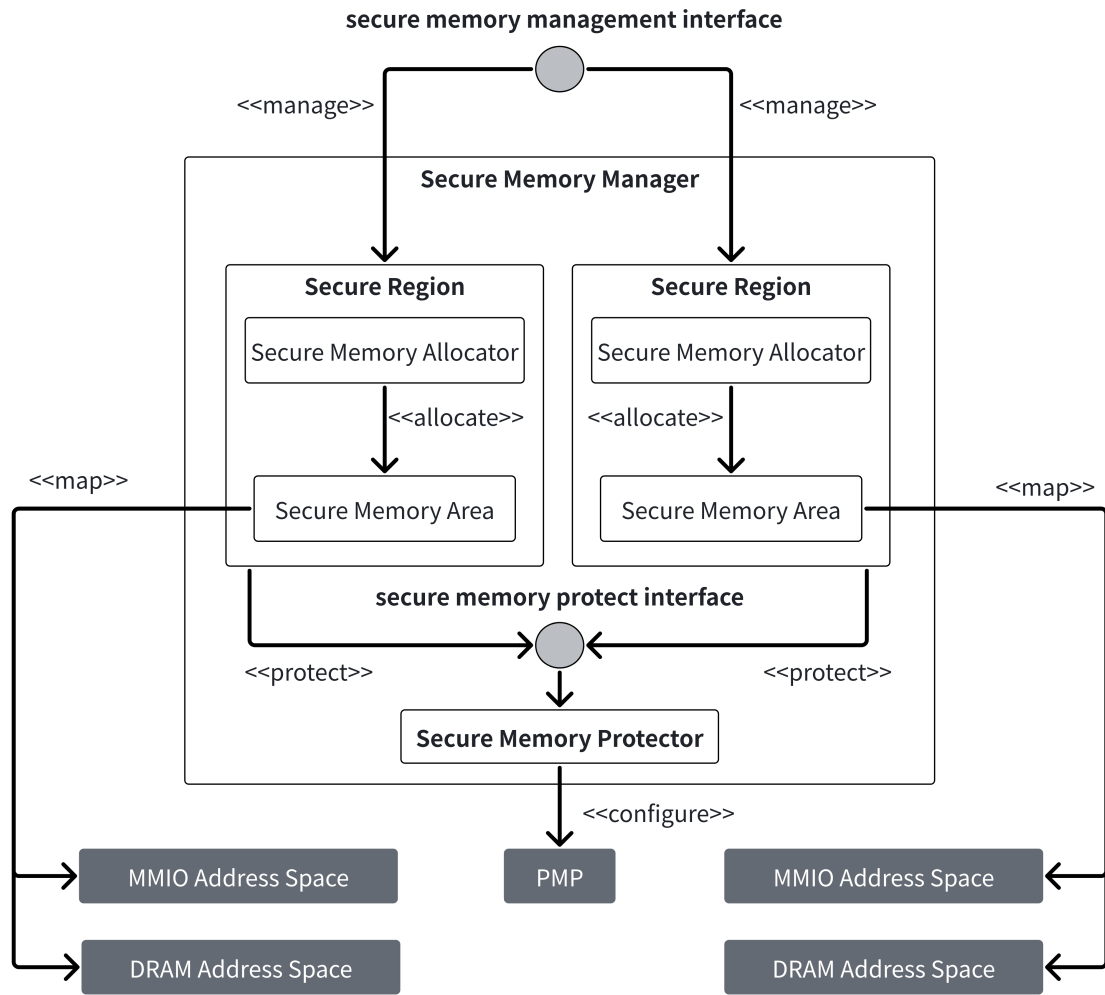


图 4-1 安全区域和内存管理层结构图

4.2.1 内存管理原语

如表4-1所示，内存管理原语抽象了安全内存管理所需的操作。包括初始化安全内存，申请/释放安全内存等。为了简化实现并避免重复安全检查引入的系统性能开销，内存管理原语不进行任何安全检查，如不在 `free` 原语中检查释放内存是否属于当前安全区域以避免破坏内存管理元数据等。

本文分析了不同飞地抽象的安全内存使用特点：

1. 基于应用的飞地抽象（应用飞地）：此类轻量化飞地通常直接运行在用户态，无权访问和控制任何内核态资源，如 MMU 和中断配置等。因此 TA 的虚拟地址空间和物理地址空间在飞地运行时不会进行任何修改。

2. 基于 OS 的飞地抽象（运行时飞地）：此类飞地更加复杂，拥有完整的用户态和内核态资源，TA 的虚拟地址空间在运行时可能修改；但是因为直接运行在与宿主 OS 相同的特权级，为了保证高效的内存隔离，飞地的物理地址空间在运行时

不会进行修改。

3. 基于虚拟机的飞地抽象（虚拟机飞地）：因为开启了硬件虚拟化，此类飞地兼具前两种飞地的特点：在虚拟化特权级下拥有完整的用户态和内核态资源，又可以在运行时无感可控地调整飞地物理地址空间并且不破坏当前内存隔离。

表 4-1 EasyAda-SM 内存管理原语

原语	功能
<code>new</code>	创建一个空的 <code>allocator</code> （此时无安全内存需要管理）
<code>init</code>	使用指定连续内存区域初始化一个 <code>allocator</code>
<code>alloc</code>	从当前安全内存中申请内存
<code>free</code>	释放安全内存到当前安全区域中
<code>available</code>	获取当前安全内存剩余可分配内存
<code>total</code>	获取当前安全内存总大小

EasyAda-SM 针对不同类型飞地使用安全内存的特点，提供了三种参考内存管理原语实现，下面将介绍其具体实现。

4.2.1.1 基于空分配算法的原语实现

该实现为安全区域的内存管理器的空实现，仅用于安全区域初始化，因为部分关键安全区域（如安全监视器自身代码和数据占用内存区域）仅需要进行隔离，不允许进行任何分配，需要使用空分配算法作为分配器占位符。在空分配算法上尝试调用任何初始化或分配操作都会导致异常。

4.2.1.2 基于伙伴分配算法的原语实现

Buddy 分配器基于对 RustSBI 现有系统堆管理算法的复用，使用 buddy 算法管理安全内存，支持多个飞地共享相同的安全区域。该实现针对 App 飞地，以 2 的幂次为粒度分配和管理安全内存，足以满足 App 类型飞地的内存需求，可以较好的兼顾分配效率和内存利用率，同时内存管理元数据直接存储在安全内存中，对安全监视器本身的内存依赖较少。劣势是为了简化管理逻辑需要预留一部分安全内存用于存储内存管理元数据，在大片内存分配场景下容易产生外部碎片。

该算法可以确保内存分配行为的安全，即同一内存块同一时刻能且仅能属于一个飞地内存区域。但是对内存块间的隔离不提供任何保障，依赖飞地管理模块配置 MMU 实现隔离，因此不可用于 Rt 类型飞地的内存管理。

4.2.1.3 基于一次性分配器的原语实现

一次性分配器将当前安全区域内所有安全内存作为一个内存块进行分配和释放，同一安全区域仅供一个飞地使用。该实现专门针对 Rt 类型飞地和安全设备的内存管理。因为 Rt 类型飞地在内存管理上具备两个特点：（1）可以在飞地内部进行内存分配，利用预留的空闲内存区域（2）可以配置 MMU，因此不能通过 MMU 实现飞地间内存隔离。而一个安全设备的内存区域应当完全由一个实体进行管理，不应进行拆分。一次性分配器仅对飞地内存需求和类型进行简单判断，如果类型匹配且安全内存满足飞地需求，则直接完整分配所有安全内存为飞地内存，并返回内存信息，由飞地在内部进行二次内存管理；同时复用安全内存保护器提供的安全区域间隔离实现飞地内存间的隔离（因为此时飞地独占安全区域）。

该算法可以确保任意类型飞地间的内存隔离，并且分配逻辑简单，开销较低。但是因为没有对内存进行任何拆分，对于不具备内存管理能力的飞地容易引入内部碎片降低内存利用率；同时，安全内存保护器可以使用的安全硬件数量较为有限（如 RISC-V PMP 条目数量多为 16，最多不超过 64 个），该方法会快速耗尽可用的安全硬件，因此不应大量使用。

4.2.1.4 基于页分配器的原语实现

页分配器（Page Allocator）是专为虚拟机类型飞地设计的内存管理组件。该分配器充分利用 RISC-V 架构多级页表支持特定映射粒度（如 4KB、2MB 和 1GB）的特性，并深度结合 Rust 语言的所有权机制。与依赖不安全内存操作的传统管理方案相比，该设计从根源上消除了内存管理元数据遭破坏的安全隐患。如图4-2所示

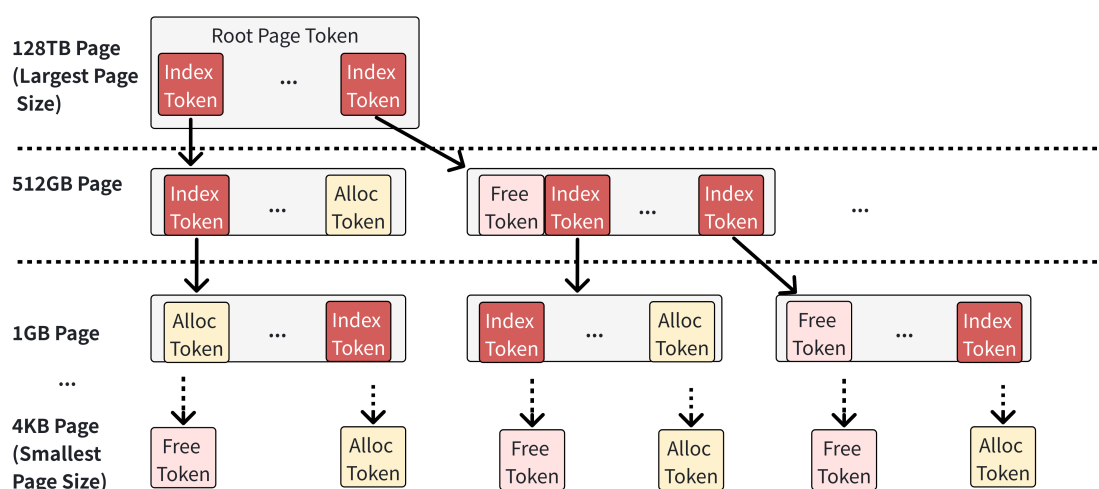


图 4-2 PageAllocator 结构

示，页分配器采用类似多级页表与 B 树结合的层级拓扑来索引与管理物理内存块。

其层级划分严格对齐 RISC-V 硬件支持的物理页尺寸。分配器中的基本管理单元定义为页令牌，每个页令牌封装了对应物理页的基址与跨度范围，并包含一个子令牌数组，用于指向当前物理页进一步拆分后的次级物理页令牌。依据状态与功能，页令牌可划分为以下三类：

1. 索引页令牌：作为路由与索引节点，该令牌不直接参与物理内存分配，仅负责标识其管辖的次级物理页已被分配或使用。索引页令牌内部动态维护当前内存区域经分割后所能支持的最大单次分配粒度（即最大连续物理页尺寸），从而加速分配时的容量校验；同时，它会为其子令牌数组实际分配内存，以持久化存储次级令牌状态。

2. 空闲页令牌：作为可供直接分配的叶子节点，该令牌标识其映射的物理页处于完全空闲状态。为优化内存开销，空闲页令牌不会为其子令牌数组预分配实际内存空间。

3. 已分配页令牌：用于标识对应物理页已完成分配且不可用。在工程实现上，已分配页令牌并非实体数据结构，不占用任何元数据内存。依托 Rust 的所有权转移语义，一旦某空闲页令牌被分配，其所有权即被消耗，原有资源被自动释放。此机制在编译期确保了该令牌无法被重复引用，彻底杜绝了非法的二次分配。

分配器初始化阶段：系统首先对目标安全内存区域进行合法性校验，强制要求其基址对齐至 4KB 物理页边界。为避免直接将理论地址上限（如 128TB）作为根节点所导致的内存空间浪费与搜索深度膨胀，分配器会依据实际管理区域的容量动态界定根页令牌的物理页尺寸。随后，分配器遍历安全内存以构建页令牌树。为最小化元数据开销，系统通过计算当前基址所能对齐的最大物理页尺寸来实例化空闲页令牌，并递归查找或建立其父级索引页令牌以完成挂载。遍历结束即标志分配器初始化完成。

页令牌生命周期管理：在分配器初始态，系统中仅存在空闲页令牌与索引页令牌。当处理物理页分配请求时，系统执行如下操作流：（1）自根页令牌发起广度优先搜索，定位首个容量达标的空闲页令牌；（2）若该令牌映射的物理页尺寸与请求严格匹配，则直接消耗此空闲页令牌（逻辑上转为已分配页令牌）并返回物理地址；（3）若其尺寸溢出请求大小，则将该空闲页令牌分裂为若干次级空闲页令牌，原节点降维转化并驻留为索引页令牌，随后对新生成的次级空闲页令牌递归执行上述匹配与分裂逻辑。

在处理物理页释放请求时，系统执行逆向回收逻辑：（1）自根页令牌起始，通过计算目标物理页在安全内存中的绝对偏移量，将其映射为各级索引页令牌的子节点索引；（2）寻址至位移与尺寸均匹配的空缺位置后，利用该物理页重构出新的空闲页令牌并重新挂载入树；（3）执行碎片合并检查，若某索引页令牌下的所

有次级节点均已恢复为空闲页令牌，则触发层级合并，消耗所有次级节点并将其父级索引页令牌向上聚合提升为单一的宏观空闲页令牌。

4.2.2 内存保护原语

表 4-2 EasyAda-SM 内存保护原语

原语	功能
alloc	申请一个安全硬件用于隔离安全区域，返回 ID
free	根据 ID 释放一个安全硬件
disable	根据 ID 关闭指定安全硬件
enable	根据 ID 使能指定安全硬件
grant_access	在指定 CPU/Hart 集合中使用指定安全硬件授权访问指定内存区域
retrive_access	在指定 CPU/Hart 集合中使用指定安全硬件限制访问指定内存区域
is_protectable	检查指定内存区域是否可以被保护

如表4-2所示，内存保护原语抽象了隔离安全区域所需操作，包括安全硬件管理和限制/授权指定内存区域访问。

因为不同的安全硬件和内存隔离机制间存在较大差异，如 PMP 机制下每个条目直接保护连续内存区域，而 STT 下的 MPT 条目则存储地址空间的权限表，两者的管理模式完全不同。因此安全内存保护器不对安全设备管理进行约束，仅要求每种实现显式告知安全区域和内存管理层安全区域与硬件的关联关系（alloc/free 原语），以及根据关联关系实现对指定内存区域的访问权限控制（grant/retrive）。

因为不同执行环境间通常拥有各自独立的 Hart，因此 RISC-V 下硬件内存保护机制通常以 Hart 为单位，每个 Hart 拥有独立的硬件且仅负责配置当前 Hart 内存保护视图。为了实现全局统一的内存保护视图，访存控制配置需要同时同步到所有相关 Hart，以安全区域为例，安全监视器需要同时完成在不可信组件使用的 Hart 集合中限制访问安全内存和在飞地使用的 Hart 集合中授权访问安全内存，因此 grant 和 access 以 Hart 集合为单位进行配置，不同 Hart 间通过核间中断和原子操作确保配置的一致性。同时，具体的 Hart 集合由更上层的飞地管理模块管理，内存保护原语仅负责同步配置到指定 Hart 集合。

因为 EasyAda-trust 主要使用 RISC-V PMP 机制保护安全内存，因此下面主要介绍基于 PMP 的内存保护原语实现。

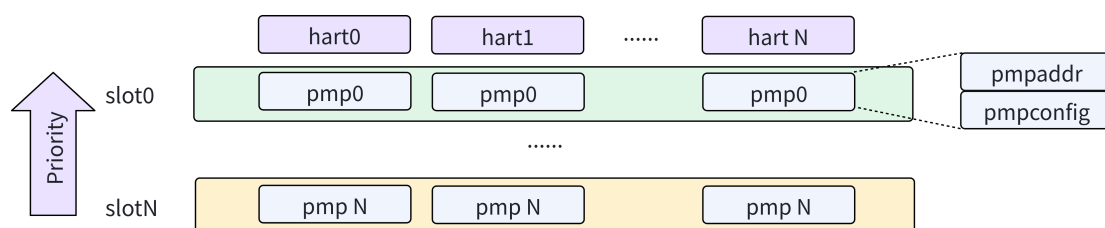


图 4-3 PMP 槽结构

4.2.2.1 基于 PMP 的内存保护原语实现

PMP slot: RISC-V 下每个 Hart 有独立的 PMP 条目和相关配置，仅控制当前 Hart 内存访问。为了实现所有 Hart 拥有统一的安全内存视图，如图4-3所示，PMP 管理器以 PMP slot 为单位管理 PMP 硬件，每个 PMP slot 是所有 Hart 上索引相同的 PMP 条目的集合。

PMP 管理器: EasyAda-SM 定义的 PMP 管理器使用位图管理 PMP slot，为了提高灵活性，位图中包含三个掩码：（1）当前管理器管理的 PMP slot 索引的掩码，标记当前 PMP 管理器管理的所有 PMP slot 的索引，运行时不可变；（2）当前管理器可分配的 PMP slot 索引的掩码，标记当前 PMP 管理器总共可分配给安全区域的 PMP slot 索引，运行时不可变；（3）当前管理器已分配 PMP slot 索引的掩码，标记当前 PMP 管理器中已分配 PMP slot 索引，运行时动态修改。

在初始化时，开发者初始化当前管理器可使用和管理的 PMP slot 索引掩码（两者的区别在于，可使用的 PMP slot 可以动态分配，用于隔离安全区域，而被管理的 PMP slot 同时包括可使用和开发者预留的 PMP slot，如最高权限的 PMP slot 被用于保护安全监视器，虽然该 slot 可以被配置，但是不能被使用）。在配置时，安全区域可以选择需要配置的 Hart 的集合，并在这些 Hart 上配置关联的 PMP slot 限制/授权访问安全内存。

授权/限制访问内存: 上层模块可以自由决定本次授权/限制操作覆盖的 Hart 范围。当前请求权限控制时，上层模块需要传入需要控制的内存区域，需要操作的 Hart 集合以及被使用的安全硬件 ID。PMP 管理器会对安全硬件可用性进行基本检查，然后在当前 Hart 上配置对应的安全硬件，如果涉及多个 Hart，则需要通过 RISC-V 下软件中断向远端 Hart 发起请求，确保配置的原子性和一致性。

4.2.3 安全内存管理原语

前文介绍了安全区域和内存管理层对内存管理和保护原语的抽象和实现，本节将介绍安全区域和内存管理层对安全内存管理原语的抽象和实现。如表4-3所示，安全内存管理原语总体上可以分为安全内存管理器生命周期管理，安全区域生命

表 4-3 EasyAda-SM 安全内存管理原语

原语	功能
new	创建新的安全内存管理器
init	初始化安全内存管理器
deinit	重置软硬件状态并销毁安全内存管理器
extend	创建一个新的安全区域
reclaim	回收一个空闲安全区域
alloc_em	从符合飞地类型的安全区域中申请飞地内存
free_em	释放飞地内存到对应安全区域中
grant_access	在指定 Hart 集合上授权访问指定内存区域
retrieve_access	在指定 Hart 集合上限制访问指定内存区域

周期管理，飞地内存管理和内存保护四类。

安全内存管理器生命周期管理：安全内存管理器作为安全区域和内存管理层的顶级管理单元，负责统筹安全内存资源和安全硬件资源，并使用安全硬件保护安全内存。在 **init** 阶段，安全内存管理器预期完成安全硬件的初始化和关键安全区域的初始化（尤其是安全监视器本身的代码和数据），此时安全内存管理器中不存在可以被用于分配飞地内存的安全区域，后续可以在安全内存管理器中动态扩展/回收安全区域。在 **deinit** 阶段，安全内存管理器会验证所有安全区域的空闲状态。为避免中断飞地的正常执行，系统强制要求飞地管理模块在发起 **deinit** 请求前，主动完成所有飞地实例的清理。同时，因为内存清理已经在飞地内存释放时完成，因此无需二次清理以避免敏感信息泄漏。然后安全内存管理器会消耗所有安全区域的所有权并记录安全内存信息归还到不可信宿主 OS。

安全区域生命周期管理：安全区域的初始化分为三个部分：内存分配，安全内存隔离，以及内存分配器初始化。内存分配通常由不可信的宿主 OS 完成；在扩展安全区域时，安全内存管理器会首先检查当前安全硬件是否支持保护安全区域，以及是否与已有安全区域重叠，确保不会破坏飞地间内存隔离。然后安全内存管理器会尝试从安全内存保护器中申请一个安全硬件，使用该安全硬件隔离安全内存和不可信组件，此时安全内存仅能被安全内存管理器访问，最后使用安全内存初始化安全区域，设置当期区域为未使用，完成安全区域初始化，但是内存分配器初始化时机由安全内存管理器自行决定，本文将在后文展示延迟初始化的实现；在回收安全区域时，安全内存管理器会检查安全区域是否是未使用状态，飞地管理模块必须主动销毁使用当前安全区域的飞地，然后才能安全回收内存。

飞地内存生命周期管理：安全内存管理器最终的目标是管理飞地分配，当飞地管理模块申请飞地内存时，需要告知安全内存管理器当前飞地类型以选择合适

的安全区域及内存分配器，安全内存管理器会遍历所有安全区域查找符合类型和内存需求的安全区域，然后尝试申请飞地内存，如果申请成功则返回内存信息和安全区域编号，否则继续重复该操作直到遍历所有安全区域；当释放飞地内存时，飞地管理模块需要提供飞地内存信息，由安全内存管理器负责遍历所有安全区域并查找飞地内存属于的安全区域，然后通过安全区域的内存分配器释放飞地内存。

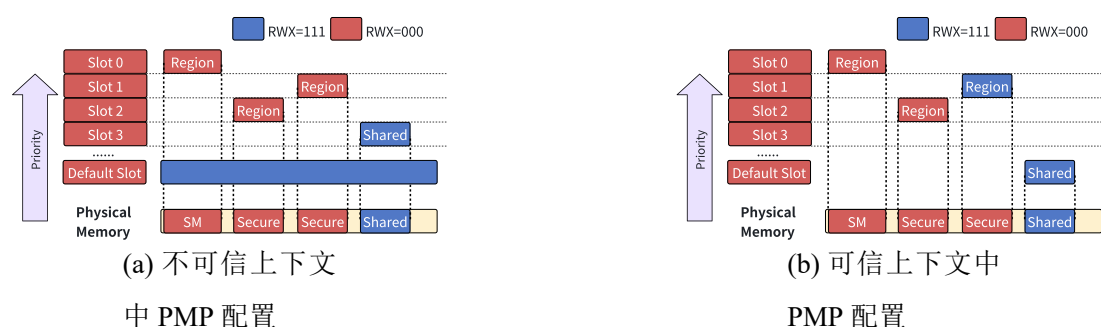


图 4-4 PMP 运行时配置

内存保护：如图4-4所示，本段以 PMP 为例介绍不安全上下文和飞地上下文间切换是内存保护配置。对于不可信的上下文，安全监视器会通过 `retrive` 限制 Hart 访问任何安全区域，其它内存可以正常访问；而对于可信上下文，安全监视器会通过 `grant` 授予 Hart 访问飞地内存所属的安全区域访问权限。因为复杂飞地可能使用多个不连续的飞地内存和多个 Hart，需要在每个 Hart 上授权访问多个安全区域，并通过 Hart 生命周期管理接口实现其它飞地使用的 Hart 上下文的安全切换。

这里需要额外关注 `default slot`，该 slot 用于赋予任何 Hart 默认访问所有内存区域的权限，避免因为保护安全内存导致 Hart 无法访问不可信内存，图4-4(b)展示了 OS 飞地的管理方式，因为其具备配置 MMU 的能力且直接运行在内核态下，需要修改 `default slot` 禁止访问所有物理内存空间，而对于 App 和虚拟机飞地则不强求，因为飞地与 MMU 相互隔离。

下面将详细介绍 EasyAda-SM 针对多种飞地抽象提供统一管理的安全内存管理原语实现。

4.2.3.1 面向通用飞地部署场景的安全内存管理原语实现

为了支持通用的飞地管理，降低用户开发难度，EasyAda-SM 实现了通用安全内存管理器（简称为通用内存管理器），将飞地和安全区域划分为一一对应的五类：（1）Application （2）Runtime （3）VirtualMachine （4）Devices（Devices 会进一步划分为串口，硬件加速器等设备类型）（5）None（仅用于保护的安全区域），并

且如图4-5所示，为每类飞地绑定了特定的内存分配器。具体来说，不同飞地内存

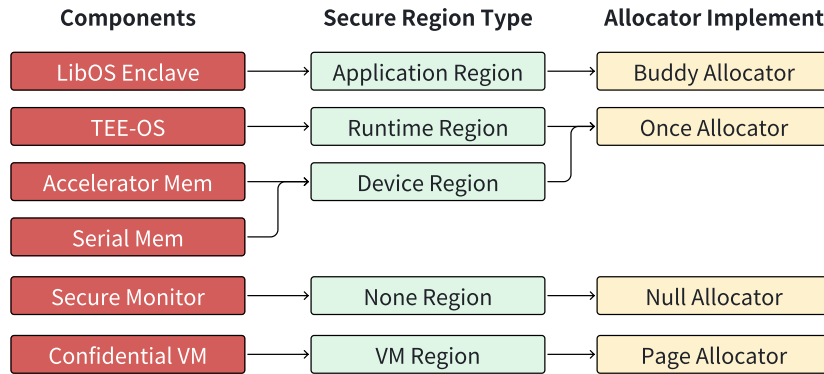


图 4-5 不同类型飞地与区域类型和分配器的关联关系

与不可信组件的隔离机制相同均基于 PMP 实现，区别在飞地内部内存管理和 PMP 权限配置不同。因此 EasyAda-SM 引入了通用安全区域表示多种安全区域状态，为了符合 Rust 的类型检查要求，EasyAda-SM 定义了包含所有分配器实现的枚举（类似 C/C++ 中的 union），安全区域通过在不同的分配器枚举间切换实现状态切换。并且状态切换按照以下状态机进行。

安全区域状态转换：初始状态下，所有安全区域均为未使用状态，使用空分配器管理内存，仅实现内存隔离，无法进行任何内存分配；当申请飞地内存时需要显式指定飞地类型，通用内存管理器会遍历查找类型匹配的安全区域并尝试申请安全内存，若当前类型匹配则直接分配，或者当前区域未被使用则转化为目标飞地类型区域，初始化内存分配器并尝试分配内存，否则继续查找符合条件的区域；当释放飞地内存时，通用内存管理器会遍历查找包含飞地内存的安全区域并尝试释放内存，若释放后安全区域无内存分配，则消耗当前安全区域所有权，转换为未使用安全区域。

除内存管理外，不同飞地抽象在安全区域保护逻辑上存在较大差异，但是该部分不属于通用内存管理器的职责范围，下一节将介绍如何基于通用内存管理器实现面向机密虚拟机的飞地管理模块。

4.3 面向机密虚拟机的飞地管理模块实现

如图4-6所示，机密虚拟机管理模块包括四个主要模块：

1. 通用安全内存管理器：如前文所说，通用安全内存管理器目标是实现安全内存与不可信组件间的隔离，负责统一管理安全设备和安全内存区域。
2. 机密虚拟机保护器：机密虚拟机保护器的目标是实现机密虚拟机间的地址空间隔离。因为机密虚拟机管理模块使用二阶段地址翻译机制在机密虚拟机间实

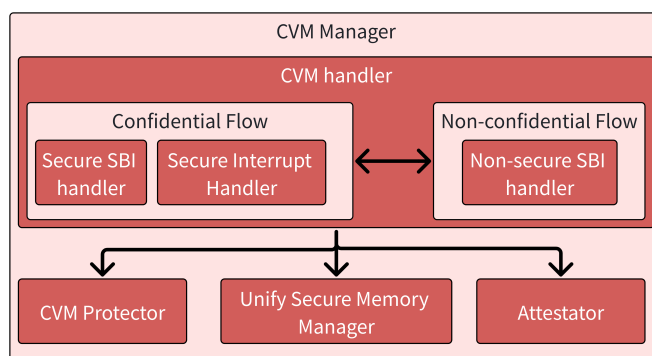


图 4-6 机密虚拟机管理器结构

现地址空间隔离的，因此该组件实际负责机密虚拟机的二阶段页表管理。

3. 机密虚拟机证明器：机密虚拟机证明器的目标是实现机密虚拟机的可信启动，负责完成机密虚拟机初始状态度量，TEE 证明载荷（TEE Attest Payload, TAP）的解密和密钥管理。

4. 机密虚拟机处理程序：机密虚拟机处理程序是机密虚拟机管理模块的统一入口，负责实现对不可信组件和机密虚拟机所有异常和中断的拦截和处理，机密处理流和非机密流间敏感数据的隔离和安全上下文切换。

可以将该模块视为 TSM 的一种实现，后文直接以 TSM 代指机密虚拟机管理器。

4.3.1 飞地间内存安全

飞地间内存安全的核心是隔离不同飞地的地址空间，安全内存管理器实现了不可信组件和安全内存间的隔离，如果直接基于其实现机密虚拟机间隔离会快速耗尽可用安全硬件，减少可管理的机密虚拟机数量。因此 TSM 利用机密虚拟机基于硬件虚拟化扩展运行的特点，引入了机密虚拟机保护器基于二阶段页表管理和实现飞地间内存安全。

机密虚拟机保护器的核心职责是维护当前机密虚拟机的二阶段页表，包括初始化机密虚拟机二阶段页表，映射/解映射安全内存和不可信内存以及使用二阶段页表翻译得到物理地址。为了避免机密虚拟机保护器耗尽安全监视器的堆内存，机密虚拟机保护器构建二阶段页表使用的内存主要来自于从安全内存管理器中申请的内存。为了提高系统内存利用率，机密虚拟机保护器允许从多个安全区域中申请，因为二阶段地址翻译和页分配器保证同一物理页仅能分配给一个机密虚拟机且机密虚拟机仅能访问指定的地址空间，因此不会破坏机密虚拟机间内存隔离。

在机密虚拟机初始化时，机密虚拟机保护器使用安全内存根据不可信虚拟机的二阶段页表构建完全对等的机密虚拟机初始二阶段页表，并将所有物理页中的

内容拷贝到安全内存中，该操作会耗费大量内存和 CPU 运行周期，但是嵌入式场景下机密虚拟机部署之后通常会持续运行直到下一个电源周期，而且安全性是 EasyAda-SM 的首要考虑目标，因此 EasyAda-SM 认为在系统启动时该操作带来的延迟和性能降低是可以接受的。在系统运行时，机密虚拟机保护器会负责二阶段页表的动态维护。

4.3.2 安全启动

安全启动的核心是确保只有被验证证明了可信且未被篡改的软件实体才能访问敏感资源。机密虚拟机管理模块引入了验证器模块用于管理密钥并对机密虚拟机进行度量。

在系统开发阶段，EasyAda-trust 的硬件平台供应商会将硬件密钥固化在信任根中，并对上提供硬件公钥，可以进一步使用硬件公钥对 EasyAda-SM 的预期度量值进行签名。在系统初始化阶段，信任根负责对安全监视器进行度量并结合硬件密钥与数字签名进行对比，如果度量成功则转移控制权到安全监视器，并传递硬件密钥给安全监视器。该私钥后续会由 TSM 用于机密虚拟机的安全启动。

在系统初始化时，机密虚拟机管理模块会初始化验证器模块用于存储硬件私钥，并作为根密钥用于派生其他密钥。因为此时安全监视器所在内存区域已经通过硬件实现隔离，因此硬件密钥无法被不可信组件窃取。

内存布局度量：当 Hypervisor 请求提升虚拟机为机密虚拟机并陷入 TSM 时，TSM 会尝试从安全内存中为机密虚拟机的 Hart 分配元数据区域，用于存储机密虚拟机的 Hart 上下文信息，并使用安全内存创建虚拟机内存布局的安全副本，包括页表本身和物理页中的内容。然后 TSM 将内存布局副本和 FDT 传递给验证器，由验证器遍历内存布局副本并根据 FDT 对内存中的关键数据进行度量（包括 kernel, initrd 和 FDT 等）并生成运行时度量值，验证器并不关心非关键区域的内容，在度量过程中验证器会负责清理所有非关键内存区域内容，避免不可信组件在其中存储恶意数据和代码。在内存布局度量完成后，验证器会将度量值存储到机密虚拟机在安全内存中的元数据中。

机密虚拟机上下文度量：完成内存布局度量后，验证器会进一步度量机密虚拟机元数据区域中存储启动 Hart 的初始上下文信息，包括通用寄存器和控制寄存器信息。因为除了启动 Hart，其他 Hart 的启动均由启动 Hart 通过 Hart 状态管理扩展进行管理，初始上下文由 TSM 进行设置和保护，所以无需进行度量。验证器同样会将初始上下文度量值存储在机密虚拟机对应的元数据中。

对比预期度量值：在系统初始化阶段，TSM 从信任根中获取了硬件私钥。验证器完成对机密虚拟机的度量之后，使用硬件密钥解密机密虚拟机开发者提供的

锁盒，然后使用解密得到的对称密钥解密 TAP，得到机密虚拟机开发者提供的预期度量值和开发者密钥。如果本地计算生成的机密虚拟机上下文和内存布局度量值与 TAP 解密得到的预期度量值或数字签名匹配，则证明当前机密虚拟机可信。

密钥管理：除了硬件私钥外，验证器还需要保护和管理机密虚拟机开发者提供的密钥。这些密钥通常会用于（1）机密虚拟机解密机密应用或根文件系统等（2）作为根密钥直接或衍生密钥对网络，串口和磁盘块设备等 I/O 数据进行加解密。机密虚拟机可以在运行时通过 SBI 请求从 TSM 获取密钥。

4.3.3 中断和异常安全

表 4-4 机密虚拟机管理模块在机密流支持的句柄类型

类别	功能描述
标准 SBI 扩展	包括（1）探测 SBI 基础扩展支持，获取硬件基本信息（2）发送处理器间中断，远程刷新指令缓存和虚拟内存地址等（3）Hart 状态管理，启动、停止、挂起机密 Hart，查询 Hart 运行状态等（4）系统控制，请求系统重置或关机
I/O 虚拟化扩展	（1）MMIO 服务请求，向不可信宿主 OS 发起 MMIO 读写请求（2）不可信的共享内存管理，向不可信的宿主 OS 请求共享内存，用于实现 I/O 半虚拟化（3）安全设备管理，请求直接映射/移除安全设备物理内存地址到机密虚拟机，用于实现设备直通。
密钥管理	从 TSM 中获取机密虚拟机的特定密钥等机密数据
机密异常处理	处理机密虚拟机二阶段地址访问异常，虚拟指令等

中断和异常安全的核心是确保机密流和非机密流之间的敏感数据互不泄露，并且阻止不可信组件通过中断或异常攻击机密虚拟机。

在标准流程中，M 态中断和异常的入口函数由 SBI 固件实现和初始化。而在 EasyAda-SM 中，机密虚拟机管理模块会修改 M 态陷入入口函数以确保机密虚拟机管理模块负责响应和监控所有异常和中断，再根据需要调用 RustSBI 固件提供的能力。这样的好处是无需在执行环境切换时重新配置入口函数以实现对机密虚拟机中断和 SBI 请求的接管和确保机密虚拟机管理模块可以监控所有敏感操作。

对于非 TEE 环境，即标准的 Hypervisor 和不可信虚拟机流程，中断和异常默认委派到 HS 态进行处理（需要由飞地管理的设备的外部中断必须在 M 态下进行处理），Hypervisor 可以进一步委派中断到 VS 态进行处理。但是对于 TEE 环境，默认情况下机密虚拟机管理模块会将所有 VS 态中断委派到机密虚拟机中，但是

表 4-5 机密虚拟机管理模块在机密流支持的句柄类型

类别	功能描述
标准 SBI 扩展	所有 RustSBI 实现的标准 SBI 扩展，由机密虚拟机处理程序调用 RustSBI 的接口实现。
机密虚拟机生命周期管理	实现不可信宿主 OS 对机密虚拟机生命周期的管理，包括将普通虚拟机提升为机密虚拟机，运行机密虚拟机和销毁机密虚拟机。
I/O 虚拟化管理扩展	(1) MMIO 请求回复，回复机密虚拟机的 MMIO 请求处理结果。(2) 共享内存请求回复，回复机密虚拟机共享内存请求处理结果以及分配的共享内存信息。
安全区域管理扩展	管理安全区域的创建和回收，不可信宿主 OS 通过此扩展捐赠内存创建新的安全区域/回收空闲安全区域使用内存

其它中断均需要陷入 M 态。而对于机密流下的异常，大部分会委派到机密虚拟机中进行处理，而 GUEST_PAGE_FAULT 等异常则直接陷入到 M 态进行处理。

不管是异常还是中断，最后都会进入机密虚拟机句柄进行统一处理。机密虚拟机句柄实现了所有机密虚拟机管理模块的对外提供的服务接口，如表4-4和4-5所示，根据 Hart 当前状态划分为机密流和非机密流两类，当前 Hart 如果运行在飞地中，则会进入可信流处理，反之则进入不可信流处理。

4.3.3.1 机密流和非机密流

因此，机密虚拟机句柄的核心工作之一是实现不可信组件和机密虚拟机上下文间的安全切换。EasyAda-SM 将其 Hart 的上下文切换分为普通上下文切换和安全上下文切换两个阶段。机密虚拟机/Hypervisor 进入安全监视器时首先会执行普通上下文切换，保存当前执行必须的通用寄存器和配置寄存器，然后进行处理。如果需要切换执行环境，即跨处理流处理时，机密虚拟机管理模块会进行安全上下文切换，将 Hart 当前所有执行状态保存到安全内存中，同时加载目标上下文的执行状态，并且，机密虚拟机管理模块还会通过通用内存管理器执行关键的内存访问权限调整和清理操作，避免当前软件实体的敏感信息泄漏给将要执行的一方，并确保正确配置将要执行的软件实体相关内存资源的权限。

4.3.4 IOPMP 管理

为了防止支持 DMA 的恶意外设绕过 CPU 的内存隔离直接访问安全内存，EasyAda-trust 使用 IOPMP 扩展限制了每个硬件可以访问的内存区域。在开发时，

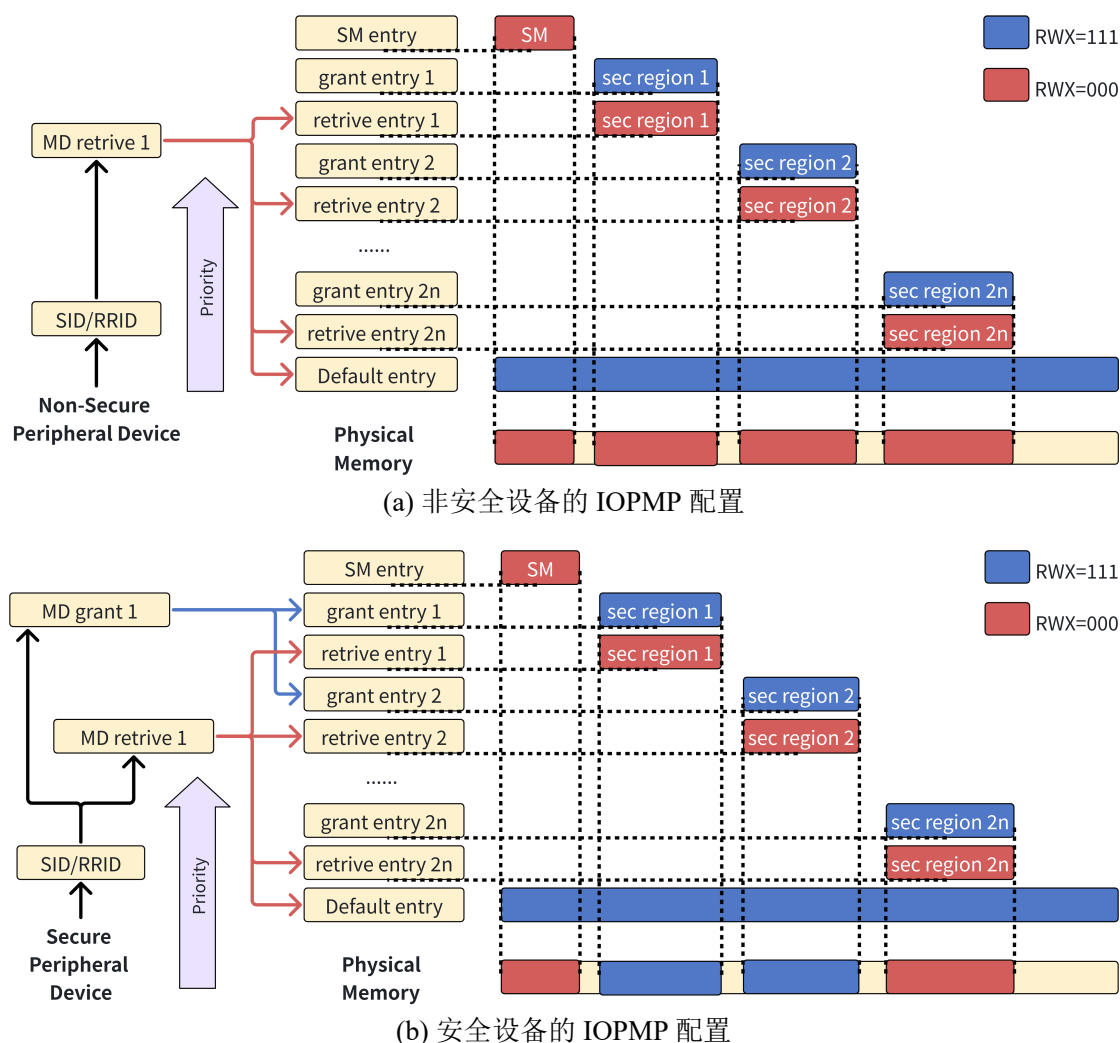


图 4-7 EasyAda -安全监视器中非安全/安全设备的 IOPMP 管理

硬件供应商需要为可信设备静态分配 SID 作为唯一标识符号。

在系统启动时，TSM 创建安全区域是会同完成对最高特权级的 IOPMP 条目（安全监视器条目）的配置，限制任何设备对安全监视器内存区域的读写，同时配置最低优先级的条目（default 条目）赋予设备默认访问所有不可信内存区域的权限。此时设备可以正常使用，但是无法读写安全监视器中的代码和数据。同时，TSM 会为所有设备的 SID 创建，绑定并启用一个用于限制访问的 MD，初始状态下每个 MD 中仅包含安全监视器和 default 条目。

图4-7展示了 EasyAda-trust 使用 IOPMP 控制设备访问安全区域的流程。在系统运行时，EasyAda-trust 以 IOPMP 条目对的形式管理 IOPMP 硬件，每当一个安全区域创建时，TSM 会按序申请第一对空闲的 $2n-1$ 和 $2n$ 表项， $2n-1$ 用于授权访问安全区域的， $2n$ 则用于限制访问安全区域。如图4-7(a)所示，对于不安全的设备，每当 TSM 创建新的安全区域和 IOPMP 表项时，会同步当前配置到所有设备的限

制 MD 中，默认限制所有设备对安全区域的读写。

同时，Hypervisor 可以在任何时刻请求将可信设备提升为安全设备，TSM 会通过可信的硬件配置文件验证设备可信，并创建安全区域隔离安全设备的 MMIO 区域，以及为安全设备创建未初始化的，用于授权访问的 MD。当安全设备实际分配给机密虚拟机时，TSM 会将机密虚拟机关联的所有安全区域的授权条目加入设备的授权 MD 中，并切换安全设备的 MD 为授权 MD。如图4-7(b)所示，此时安全设备仅能访问机密虚拟机关联的安全区域，杜绝了安全设备将敏感数据写入不可信内存的隐患。

TSM 仅在设备提升为安全设备时，或从安全设备回退为不安全设备时进行重新配置，避免频繁切换 IOPMP 配置时上下文切换和缓存一致性带来的性能开销。

4.4 本章小结

本章系统论述了基于微内核的可信虚拟化方案的工程实现。在 CPU 虚拟化层面，通过重构 vhart 分区架构，并由 TSM 接管两阶段上下文切换，实现了机密虚拟机运行状态对不可信环境的彻底隐匿。在内存虚拟化层面，实施了资源分配与安全管控的深度解耦：Hypervisor 仅负责物理资源的宏观调度，而 TSM 全权把控安全区域的生命周期与二阶段页表的隔离映射。在 I/O 虚拟化层面，确立了主机分配、TSM 鉴权的安全直通设备管理流程，依托 IOPMP 扩展构筑了防御 DMA 攻击的物理屏障，并结合动态密钥供应实现了数据平面的端到端保护。最后，面向端侧异构计算需求，构建了专用的硬件加速器管理机密虚拟机，基于不可信共享内存与半虚拟化技术安全打通了跨域通信信道，实现了加速器的机密部署与安全复用。

第五章 基于微内核的可信虚拟化方案实现

5.1 安全启动流程实现

EasyAda-trust 下信任根是安全启动的起点。如图5-1所示，硬件供应商需要提供验签密钥对。验签公钥对安全监视器预期度量值生成数字签名，并将数字签名存储在信任根中。验签私钥则用于在开发时生成可信安全监视器的数字签名。在系统启动流程中，信任根会计算安全监视器的实际度量值，并根据信任根中的验签公钥对比预存的数字签名完成验证，验证通过后信任根会将控制权转交给安全监视器并进入安全启动的下一阶段。除了硬件供应商统一的验签密钥对，每个设备拥有独有的，无法篡改的设备私钥对，设备公钥向平台供应商和开发者开放，用于加密验证信息或敏感数据，而设备私钥存储在设备信任根中，用于在运行时进行解密。

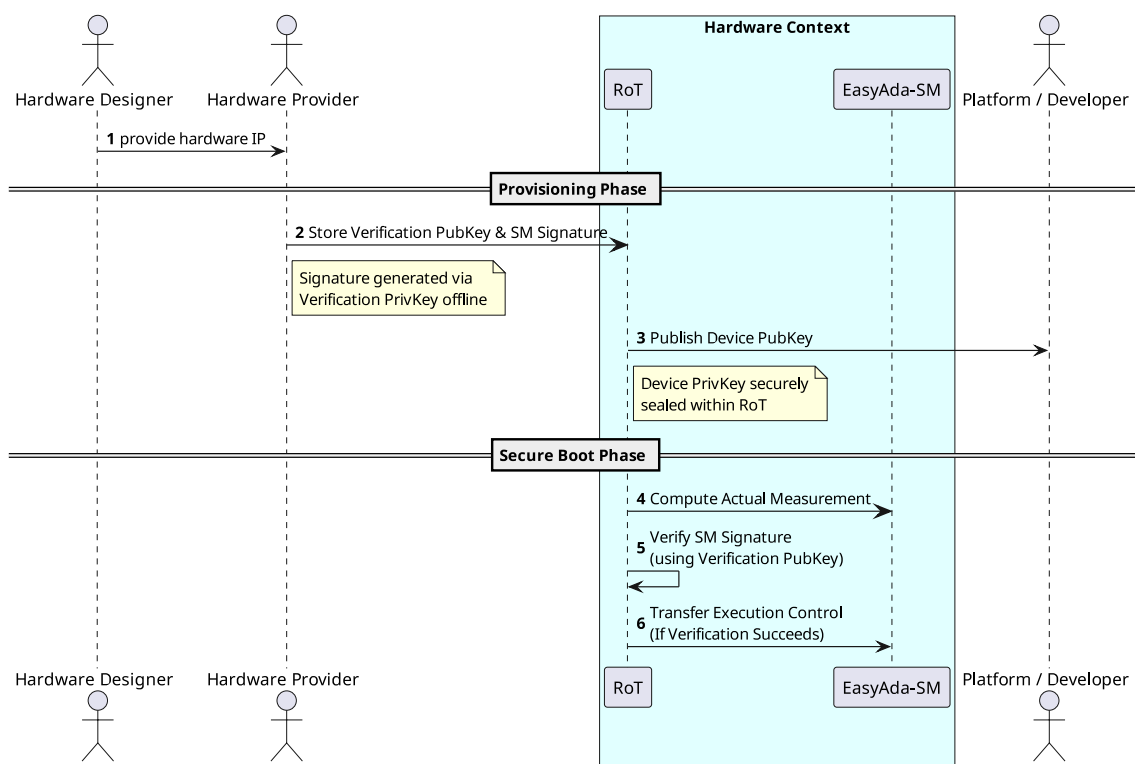


图 5-1 EasyAda-SM 签名验证流程

安全监视器初始化流程分为两个部分：TSM 初始化和 SBI 运行环境初始化。因为 SBI 固件被视为不可信或实现可能存在漏洞，因此安全监视器初始化流程由可信的 TSM 主导。TSM 初始化时会首先判断当前硬件环境是否满足部署 EasyAda-trust 的最低要求，然后清空信任根副本并初始化根安全区域隔离安全监视器自身

内存，防止安全监视器在系统运行过程中被运行在更低特权级的不可信组件非法访问和篡改。

完成安全监视器内存隔离后，TSM 初始化流程会调用 SBI 固件实现的接口完成 SBI 运行环境初始化，然后由 TSM 完成剩余模块如验证器等模块的初始化并重新配置安全监视器的陷入入口，确保低特权软件的执行在明确定义的入口点陷入到 TSM 和安全监视器中，最终转移控制权到下一阶段引导程序或 Hypervisor。当 TSM 初始化完成时，安全监视器已经通过硬件实现了与不可信组件的强制隔离，可以安全进入更低特权级并转移控制权到 Hypervisor 继续下一阶段初始化。

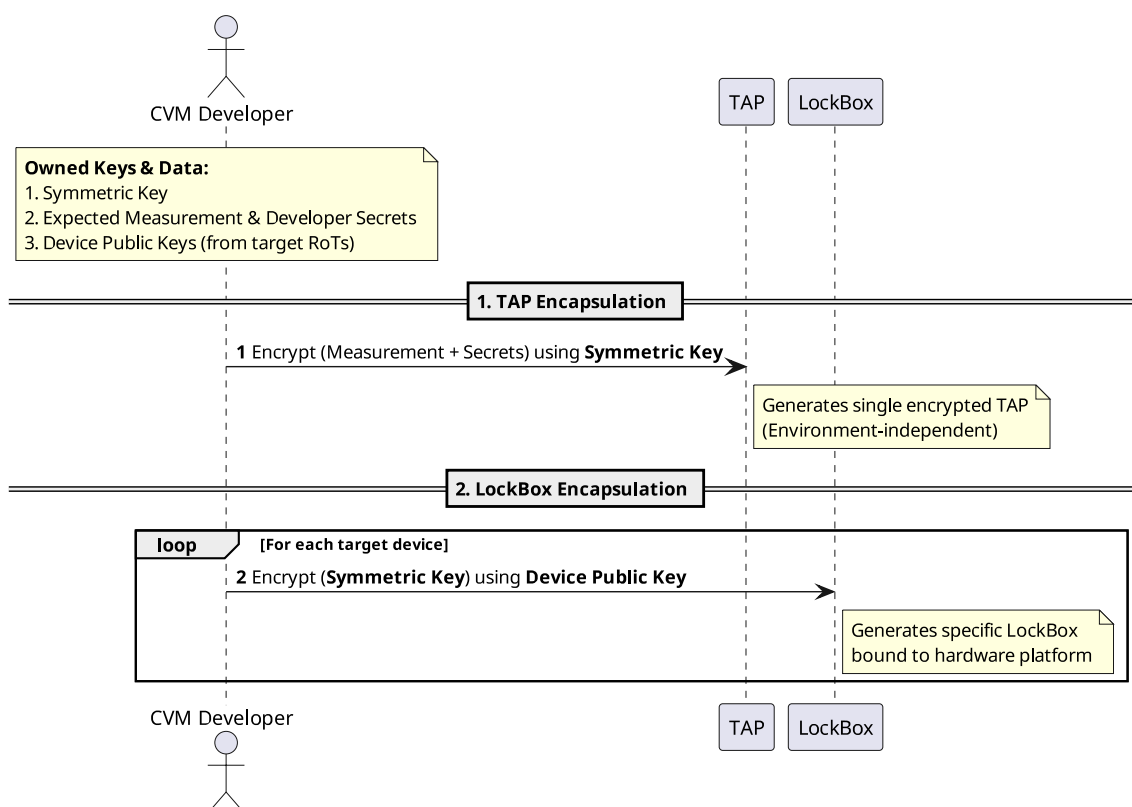


图 5-2 机密虚拟机验证信息封装流程

为了支持机密虚拟机和硬件环境的本地证明。如图5-2所示，机密虚拟机开发者必须使用开发者提供的密钥对称加密 TAP 文件。该文件用于存储机密虚拟机预期度量值和开发者提供的用于加解密敏感 I/O 数据，代码和参数等的秘密，并且配合一个使用当前设备公钥加密，用于存储加密 TAP 的对称密钥的锁盒（LockBox）。为了支持在多个设备上部署，同一 TAP 可以关联多个使用不同设备公钥加密的锁盒。

在部署机密虚拟机时，安全监视器会将机密虚拟机的初始化状态保存到安全内存和 Hart 结构体中并通过验证器模块对关键数据进行度量等到当前度量值，然

后使用信任根提供的设备私钥解密锁盒，并使用锁盒中的对称密钥解密 TAP，最终得到机密虚拟机预期度量值和开发者提供的秘密。如果实际和预期度量值对比成功则验证通过。TSM 会将从 TAP 中解密获得的开发者秘密存储在验证器模块中。机密虚拟机可以通过 SBI 请求获取当前机密虚拟机开发者提供的秘密，其它机密虚拟机或不可信组件无法窃取或篡改。

本地证明可以实现机密虚拟机与硬件环境的双向证明，如图5-3所示，因为存储 TAP 加密密钥的锁盒仅能在持有对应硬件私钥的硬件环境上被正确解密，即使机密虚拟机被强行拉起，不可信环境仍然无法获取机密虚拟机开发者提供的特定秘密，无法从机密虚拟机中解密任何敏感代码或数据；另一方面，任何对 TAP，锁盒或者机密虚拟机镜像的篡改都会导致最终的度量失败，确保只有可信的，未经过篡改的机密虚拟机可以被部署。同时，本地证明与远程证明并不冲突，在具备网络条件的环境上仍然可以使用远程证明验证可信环境。

5.2 安全 CPU 虚拟化实现

EasyAda-Hypervisor 将虚拟 CPU（vhart）抽象为微内核操作系统中的线程进行调度。在其线程控制块中，除包含标准的通用寄存器状态外，还额外记录了当前虚拟机的控制寄存器信息，以确保能够精确恢复虚拟机的运行上下文。此外，EasyAda-Hypervisor 为每个 vhart 构建了两个独立的执行上下文：vhart 运行线程与 vhart 服务线程。前者负责存储与调度虚拟机的运行时计算上下文，后者则专职处理虚拟机的异步 I/O 请求。

为提升系统整体性能，EasyAda-Hypervisor 将物理 Hart 静态划分为三类功能区：(1)vhart 运行区：主要承担虚拟机及其原生应用的计算任务；(2)vhart 服务区：专用于承载 vhart 服务线程，以异步方式高效响应虚拟机的 I/O 请求，最大程度降低对虚拟机计算任务的干扰；(3)系统服务区：专用于运行内存管理、进程管理及文件系统等微内核核心系统服务，旨在消除因任务调度抖动而引发的系统性能衰退。

EasyAda-trust 在此 CPU 虚拟化架构的基础上进行了安全维度的扩展，引入了额外的隔离机制，以实现机密虚拟机与不可信 Hypervisor 之间上下文的严格物理隔离。

5.2.1 安全上下文管理

在 EasyAda-trust 架构中，底层可信安全监视器 (TSM) 驱动不介入不可信虚拟机的 vhart 管理，该职责仍由 Hypervisor 保留。然而对于机密虚拟机的 vhart，尽管

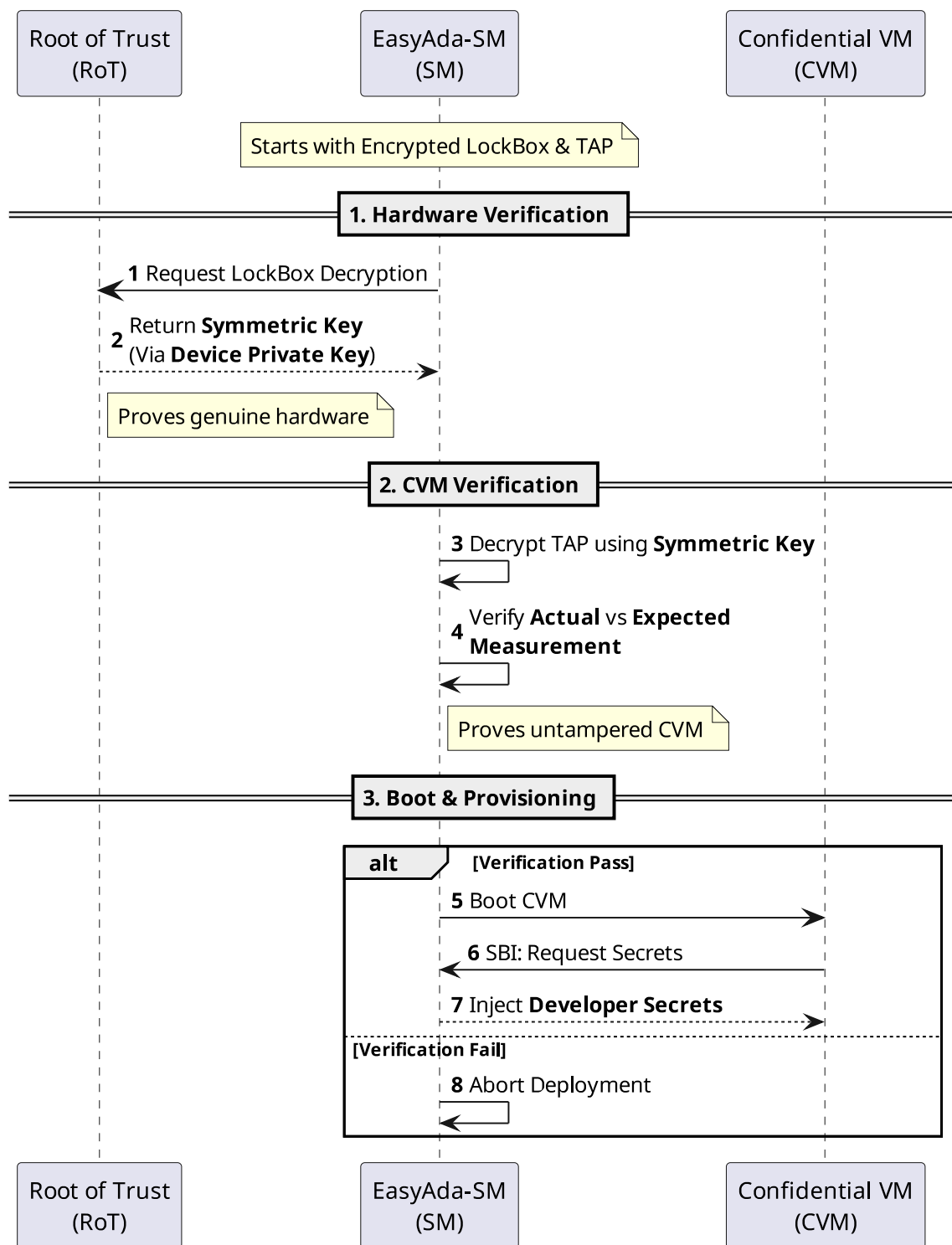


图 5-3 EasyAda-trust 中双向验证流程

其宏观调度策略仍由 Hypervisor 负责，但其核心上下文状态必须由 TSM 进行机密管理，并对 Hypervisor 呈现完全的不可见性。Hypervisor 仅能通过 TSM 暴露的有限的机密虚拟机生命周期管理接口，间接完成对机密虚拟机 vhart 的调度操作。

如图5-4所示，EasyAda-trust 在原生 Hart 分区机制的基础上，将 vhart 运行区

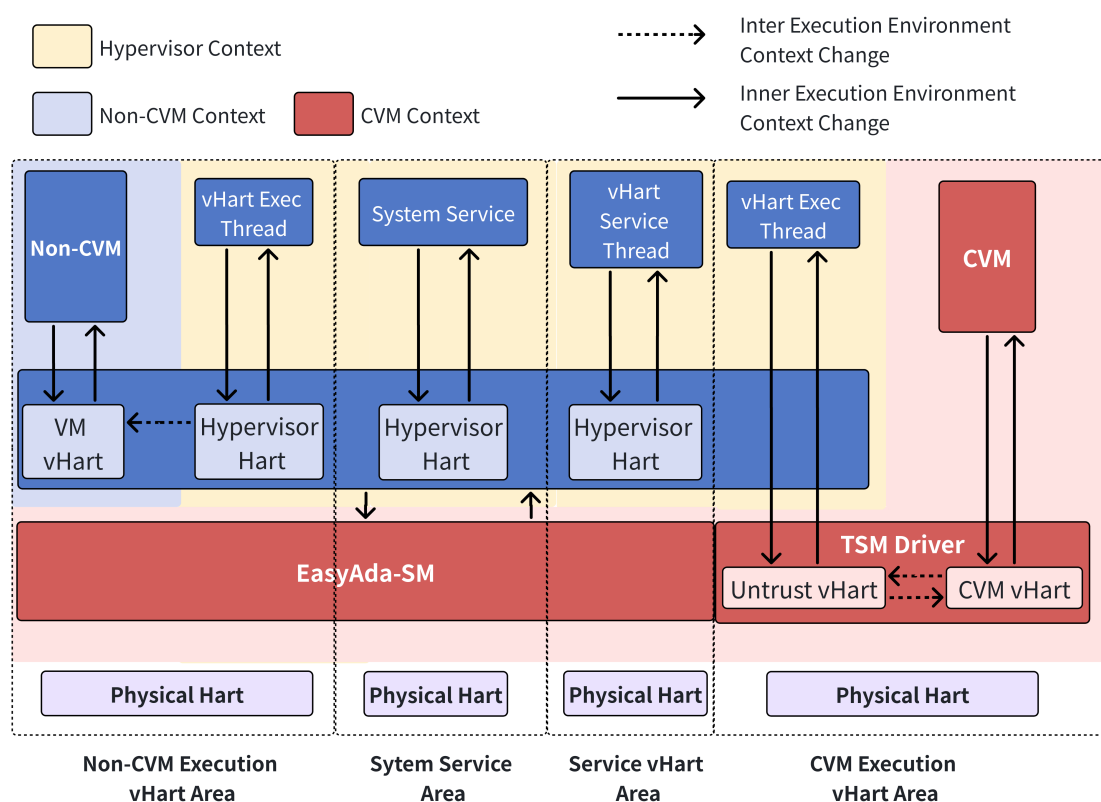


图 5-4 EasyAda-trust 下 Hart 分区示意图

进一步细分为机密虚拟机 vhart 运行区与不可信虚拟机运行区。两者在架构上的核心差异在于跨执行环境时的上下文切换流程。

为了规避非必要的上下文保存与恢复开销，EasyAda-Hypervisor 将 vhart 上下文切换机制划分为执行环境内轻量级切换与跨执行环境安全切换两个层级。对于不可信虚拟机，Hypervisor 在逻辑上将其上下文解耦为虚拟机侧的虚拟机上下文与 Hypervisor 侧的 vhart 运行上下文。两者均各自维护一套完整的状态集合，具体包括：(1) 最小架构状态寄存器集合 (含通用寄存器及 TSM 运行时可能变更的控制寄存器)；(2) 虚拟化控制寄存器状态；(3) 虚拟化功能寄存器状态。由于 Hypervisor 在自身运行期间不会修改上述 (2) 与 (3) 中的寄存器状态，因此虚拟机与 Hypervisor 之间的上下文切换仅需保存集合 (1) 的信息即可。然而，在不同虚拟机实例之间进行切换时，则必须完整保存并清理涵盖 (1)、(2)、(3) 在内的全量上下文信息。为此，Hypervisor 通过 vhart 运行线程统一管理这两类逻辑上下文状态。

相比之下，机密虚拟机的上下文切换呈现出以下安全特征：

1. 扩展的上下文保护范围：鉴于机密虚拟机对 Hypervisor 处于完全不信任状态，除上述三类基础上下文信息外，TSM 还必须额外切换并保护 M 态与 HS 态的 CSR（例如中断与异常委派状态寄存器等）。

2. 物理内存视图的严格隔离：机密虚拟机与 Hypervisor 分别运行于相互隔离的物理内存视图中。在触发上下文切换时，TSM 必须动态重配置底层安全硬件 (如 PMP)，以维持安全内存与不可信组件之间的物理隔离边界。

3. 执行流的安全阻断与切换：机密虚拟机与 Hypervisor 的执行流彼此互斥。在处理特定异常 (如 MMIO 访问异常) 而触发执行流切换时，必须确保 Hypervisor 无法窥探或获取任何属于机密虚拟机及 TSM 的上下文残余信息。

针对上述特征，EasyAda-trust 将机密虚拟机相关的上下文严格划分为“不可信 vhart 上下文”与“机密 vhart 上下文”两类。前者负责暂存 Hypervisor 侧 vhart 运行线程的全部状态，后者则专用于安全存储机密虚拟机的全量状态。两者均涵盖所有通用寄存器以及 VS/HS/M 态系统控制寄存器；若底层硬件使能了浮点或向量指令扩展，还需额外保护浮点或向量处理单元的上下文信息。

为缓解机密虚拟机与 Hypervisor 间频繁交互带来的性能损耗，若当前 Hart 控制流未跨越信任边界，则仅执行轻量级上下文切换；反之，则必须执行完整的安全上下文切换流程。

轻量级上下文切换：当当前 Hart 从 Hypervisor 或机密虚拟机陷入 TSM 执行特定服务时，由 TSM 直接原地完成请求处理。在此过程中，Hart 仅需临时保存当前执行环境的通用寄存器，待 TSM 处理完毕后直接恢复。由于该过程不涉及执行环境的越界切换，因此无需动态重构物理内存保护视图，切换开销极小。

安全上下文切换：当当前 Hart 必须在 Hypervisor 与机密虚拟机之间转移控制权时，需严格遵循机密虚拟机管理模块的状态机约束。在完成必要参数的安全校验与传递后，TSM 会将当前全量上下文状态加密 (或隔离) 存储至对应的 vhart 结构中，随后彻底清理当前物理环境的残余寄存器状态，最后加载目标执行环境的上下文，从而完成执行流的安全迁移。在执行目标指令前，TSM 必须重新配置安全硬件以切换物理内存视图。具体而言，当控制流由不可信 vhart 切换至机密 vhart 时，TSM 会授权当前 Hart 访问除安全监视器 (安全监视器) 自身内存外的所有关联安全区域及不可信内存 (此时依靠二阶段页表机制，TSM 足以严密管控机密虚拟机的最终物理寻址能力)；反之，当控制流由机密 vhart 切回不可信 vhart 时，TSM 将立刻剥夺当前 Hart 对所有安全区域的物理访问权限。

5.2.2 安全异常处理

如第4.3.3节所述，EasyAda-SM 通过 Rust 语言的静态内存检查机制，从编译源头消除了因不安全内存访问而导致的敏感数据泄漏隐患，并凭借构造-处理-析构的规范化生命周期模型，严格界定了各项处理服务所能触达的资源边界。与此同时，TSM 在架构设计上刻意摒弃了向不可信组件内存空间写入数据的能力，从根本上

阻断了机密数据向不可信域泄漏的物理路径。

然而必须强调的是，尽管 TSM 在底层机制上保障了不同处理器与执行流之间的物理安全，但机密虚拟机内部的客户操作系统仍需肩负起正确调用 TSM 安全接口的责任。例如，当 TSM 为机密虚拟机分配不可信的共享物理内存以实现 I/O 半虚拟化时，由于该内存区域暴露于不可信环境中，TSM 本身无法为其提供端到端的机密性与完整性保障。因此，机密虚拟机必须在应用层或客户内核层，自主引入必要的密码学加解密与度量校验机制，以确保共享数据的流转与使用严格符合安全预期。

5.3 安全内存虚拟化实现

内存虚拟化主要涵盖虚拟机运行时内存、设备 MMIO 区域以及用于 I/O 半虚拟化的共享内存的二阶段页表映射。二阶段页表的根页表基址作为虚拟机配置的核心信息，被保存在 vhart 运行线程的上下文中。EasyAda-Hypervisor 按照以下标准流程构建并维护二阶段页表：（1）用户态 VMM 解析当前虚拟机的配置文件（如 FDT），计算虚拟机所需的运行时物理内存规模（不包含 MMIO 区域），并通过系统服务申请相应的物理内存页；（2）依据配置文件约束，将虚拟机的 Kernel、FDT 等关键镜像加载至满足对齐要求的物理地址中；（3）依据配置文件构建 GPA 至 HPA 的映射关系，由运行在内核态的 Hypervisor 驱动创建初始的二阶段页表，并将其基址存入对应的 vhart 运行线程上下文中。

MMIO 区域的映射主要服务于设备直通（Device Passthrough）场景。在创建虚拟机时，运行于内核态的 Hypervisor 驱动会解析配置文件中的物理设备信息，基于 GPA 与 HPA 的映射关系，将目标物理设备的 MMIO 区域直接映射至虚拟机的独立地址空间中。此外，Hypervisor 还会将该硬件设备的物理中断号注册并绑定至目标虚拟机，以确保后续的虚拟中断注入机制能够正确触发，从而保障虚拟机对直通设备的正常访问与中断响应。

共享内存映射主要服务于基于半虚拟化技术的虚拟设备管理。其初始化过程采用按需分配机制：当虚拟机在系统运行过程中首次访问虚拟设备的 MMIO 区域（GPA）并触发异常时，内核态的 Hypervisor 驱动将拦截该异常。经与虚拟设备注册信息匹配确认后，若该 GPA 属于合法的虚拟设备 MMIO 范围，Hypervisor 驱动将切换至 vhart 运行上下文，并在用户态向微内核系统服务申请对应的物理共享内存。随后，由 Hypervisor 驱动在内核态完成该共享内存到虚拟机二阶段地址空间的映射，并从异常处理返回，恢复虚拟机的继续执行。在 EasyAda 的底层实现中，共享内存被抽象为 UNIX 风格的文件对象，该文件的所有权归属于 vhart 运行线程，

并在常规架构下被映射至 `vhart` 服务线程中，以支撑高效的异步 I/O 交互。

在 EasyAda-trust 架构中，针对不可信虚拟机的内存虚拟化流程保持原貌，但将机密虚拟机的内存虚拟化控制流与不可信 Hypervisor 进行了深度解耦。具体而言，Hypervisor 仅保留宏观物理层面的安全区域创建与共享内存分配权限，而核心的受保护内存/设备分配权及二阶段页表映射逻辑，则全权由受信任的 TSM 接管。

5.3.1 基于 TSM 接口的安全区域生命周期管理

如前文所述，安全区域的具体物理抽象、隔离机制与内部内存管理层已在 EasyAda-SM 固件中实现并封装。在 EasyAda-trust 架构中，Hypervisor 无需感知安全区域底层的 PMP 配置与物理鉴权细节，而是作为上层资源调度者，通过 TSM 对外暴露的安全管理扩展接口，完成机密虚拟机所需核心物理内存的动态构建、分配与回收。

5.3.1.1 安全区域的动态创建触发

在系统运行期，当 Hypervisor 判定需为机密虚拟机扩充安全环境时，将主动通过微内核系统服务申请满足连续性约束的物理内存或物理外设的 MMIO 区域。随后，Hypervisor 将该物理地址空间作为参数，通过特权级调用触发 TSM 的安全区域创建接口。在此协同流程中，Hypervisor 仅负责宏观物理资源的供给与上层业务的无中断调度，而核心的基于可信硬件配置的合法性校验、物理属性确权以及底层硬件强隔离机制，均交由 EasyAda-SM 在极高特权级下透明完成。一旦 TSM 返回创建成功信号，Hypervisor 即在内部资源表中将该物理区间标记为“不可见”，彻底剥离不可信环境对该区域的寻址能力。

5.3.1.2 基于单次请求的特权提升与按需分配

在初始化阶段，Hypervisor 首先遵循标准虚拟机的创建流程，为目标虚拟机分配常规的内存与设备资源，并完成其基础逻辑架构的初始配置。当业务场景需要将该标准虚拟机提升为机密虚拟机时，Hypervisor 不再精细化介入底层安全资源的调度过程，而是仅通过一次独立的 SBI 请求，向 TSM 下达特权提升与转换指令。

TSM 在截获该 SBI 请求后，将全权接管后续的安全资源配置工作：其会自主从当前已就绪的全局安全区域池中，为该机密虚拟机按需划拨容量匹配的内存型安全区域（用于承载机密虚拟机镜像与二阶段页表等核心元数据），并尝试预分配必需的设备型安全区域。待该机密虚拟机完成本地度量证明并被系统确认为绝对可信后，由 TSM 独立执行实质性的安全区域物理分配与二阶段页表的安全重映射，从而以最小的 Hypervisor 交互代价完成了机密虚拟机的资源构建。

5.3.1.3 安全区域的安全回收调度

当系统面临物理内存或外设资源瓶颈，抑或机密虚拟机生命周期终止时，Hypervisor 作为全局资源管理者，可向 TSM 发起主动的资源回收调度请求。针对内存型安全区域，Hypervisor 请求 TSM 扫描并释放特定或空闲的安全区域；针对设备型安全区域，Hypervisor 需显式提供目标设备的 MMIO 地址以请求解绑。TSM 在底层安全释放硬件隔离限制并完成必要的擦除后，会将物理控制权交还。Hypervisor 接收到释放成功的确认后，重新将该物理内存或设备 MMIO 区间纳入不可信系统的可用资源池中，从而形成完整的资源调度闭环。

5.4 安全 I/O 虚拟化实现

EasyAda-Hypervisor 原生支持三种 I/O 虚拟化技术：（1）全虚拟化：用于模拟逻辑简单或安全性要求较高的基础组件（如串口与中断控制器等）；（2）半虚拟化：用于支撑大吞吐量 I/O 数据流的设备（如网卡、块存储设备等）；（3）设备直通：用于管理驱动逻辑复杂或具备专有计算能力的硬件外设（如 AI 硬件加速器等）。

在原生架构中，虚拟机的设备初始化被设计为虚拟机无感的被动触发流程，旨在最大程度减少对客户 OS 代码的侵入。系统采用静态分区与动态分配相结合的硬件发现机制，其完整流程涵盖三个阶段：（1）系统静态配置阶段：硬件供应商提供包含全局视角的 FDT，应用开发者依此划分直通设备，并在虚拟机的专属硬件配置文件中建立静态绑定关联；（2）虚拟机初始化阶段：Hypervisor 解析该配置文件，若存在匹配的直通外设则将其逻辑分配给目标虚拟机，否则降级为全虚拟化模拟；（3）首次访存触发阶段：设备被分配后，虚拟机的首次 MMIO 访问将触发缺页异常并陷入至 Hypervisor。针对全虚拟化设备，由 Hypervisor 直接接管处理；针对半虚拟化设备，Hypervisor 将建立不可信共享内存及主从异步事件通知信道（虚拟机主动发起访问触发事件，vhart 服务线程异步处理后通过中断注入返回结果）；针对直通设备，则由 Hypervisor 完成物理 MMIO 至 GPA 的最终映射，交由虚拟机自主完成驱动初始化。

针对机密虚拟机的高安全需求，EasyAda-trust 在复用上述全虚拟化与半虚拟化基础路由逻辑的同时，重点对设备直通与数据通道进行了深度的安全扩展。安全设备的生命周期依然由 Hypervisor 驱动宏观调度，但其底层鉴权与物理隔离全权由 TSM 把控。本节将重点阐述 EasyAda-trust 如何保障机密虚拟机使用 I/O 设备过程中的全链路安全性。

5.4.1 安全直通设备生命周期管理

EasyAda-trust 对安全外设实施“主机分配，TSM 鉴权”的协同管理机制。在系统构建阶段，硬件供应商需在底层固化 Trusted FDT，同时应用开发者需提供机密虚拟机专用的虚拟硬件配置文件。这一机制将复杂的运行时设备发现转化为静态的密码学信任链校验，极大降低了安全架构的动态攻击面。

在标准虚拟机创建阶段，Hypervisor 仍遵循常规逻辑进行设备初始化与逻辑映射，但此时虚拟机并不具备对该物理外设的实际读写权限。当 Hypervisor 请求将该普通虚拟机提升为机密虚拟机时，TSM 将介入并执行严格的设备鉴权：其将机密虚拟机配置文件中声明的外设与底层 Trusted FDT 进行交叉比对。若匹配成功且被标记为可信，TSM 将为该设备分配专属的设备型安全区域；若匹配失败，则强制将该 MMIO 区域降级回退为全虚拟化处理，防范 Hypervisor 恶意挂载仿冒外设。

在机密虚拟机运行期，当其首次访问 MMIO 区域并陷入至 TSM 时，TSM 将依据 MMIO 地址的属性类别实施安全路由：若属全虚拟化设备，则将请求转换为标准 MMIO 拦截事件并转发至 Hypervisor 侧处理；若属半虚拟化设备，则转换为不可信共享内存映射请求交由 Hypervisor 调度；若属已鉴权的安全直通设备，TSM 将在底层安全构建物理 MMIO 至机密虚拟机地址空间的二阶段页表映射，随后将控制权交还机密虚拟机以完成原生的设备初始化。

5.4.2 基于 IOPMP 的设备侧防 DMA 攻击隔离

仅在 CPU 侧通过 MMU/PMP 隔离 MMIO 空间，仍无法抵御具备 DMA 能力的恶意外设越权访问安全内存。为此，EasyAda-trust 引入了 IOPMP 硬件扩展，从总线级构建外设访存的物理安全网关。在硬件开发阶段，供应商需为所有硬件设备静态分配唯一源标识符（Source ID）。

在系统启动初始化阶段，TSM 会默认接管全局 IOPMP 控制器，并为所有已注册的 SID 创建并绑定初始的不可信 MD。该不可信 MD 的基础访问控制策略被严格设定为允许访问不可信物理内存，但直接屏蔽并拦截对任何受保护的安全区域的寻址请求。所有未被显式提升为安全设备的外设，其 SID 均被默认约束于此不可信 MD 内，从而确立了系统初始阶段防范 DMA 攻击的默认拒绝（Default-Deny）物理隔离基线。因为所有设备共享相同的不可信内存视图，所以共享同一个不可信 MD。

在系统运行期，伴随机密虚拟机生命周期的演进，每当 Hypervisor 通过 TSM 成功创建一个全新的安全区域时，TSM 将立即触发全局 IOPMP 状态同步机制。具体而言，TSM 会遍历并修改不可信 MD 的内部规则，将针对该新增安全区域的限

制访问条目动态注入其中。这一严格的实时同步策略确保了即使安全内存是动态分配与衍生的，所有驻留于不可信环境的常规外设亦无法通过 DMA 途径对其进行非法的探测、读取或篡改。

当 Hypervisor 请求将特定物理外设提升为安全设备以供机密虚拟机独占使用时，TSM 首先校验该设备 SID 对应的可信硬件配置属性。鉴权通过后，TSM 会为该设备分配一个处于隔离状态的可信 MD。唯有当该安全设备被实质性地挂载并分配给目标机密虚拟机时，TSM 才会动态更新该可信 MD，将当前机密虚拟机所关联的所有内存型安全区域的授权条目注入其中，并将该设备的 SID 从不可信 MD 解绑，转而与该可信 MD 强绑定。至此，IOPMP 硬件强制限定该直通设备仅能对其从属机密虚拟机的受保护内存发起 DMA 操作，不仅防范了外部入侵，亦从物理总线层面彻底杜绝了设备内部敏感数据向不可信内存泄漏的风险。

反之，当机密虚拟机生命周期终止并触发安全设备释放时，TSM 将立即重置并安全擦除该设备所对应可信 MD 中的所有授权规则。在 Hypervisor 请求回收该外设资源时，TSM 会将其 SID 重新降级并绑定至不可信 MD，从而在确保物理安全的界限下，完成硬件外设的归还与状态复原。

5.4.3 I/O 数据平面安全与端到端保护

确立了设备控制平面的物理隔离后，必须进一步保障 I/O 数据平面的机密性与完整性。机密应用（如部署于机密虚拟机中的 AI 推理引擎）是高价值 I/O 数据（如 AI 模型权重与用户输入）的核心消费者。

I/O 数据保护的基石在于密钥的安全供应。在系统启动阶段，机密虚拟机与底层硬件环境之间将执行双向本地证明。确认环境可信后，TSM 利用信任根内置的设备私钥解锁受保护的锁盒（Lockbox），获取对称密钥并进一步解密目标机密虚拟机提供的 TAP，从而安全提取出开发者预置的机密素材（Secrets）并暂存于 TSM 内部。在机密虚拟机运行时，客户 OS 可通过安全的 SBI 请求，从 TSM 处按需获取相应的开发者密钥或会话密钥。

获取密钥后，机密虚拟机需针对不同类型的 I/O 通道采取差异化的安全策略：

1. 安全直通设备的数据流：得益于 PMP 与 IOPMP 构建的双向物理隔离屏障，不可信组件（包括 Hypervisor）无法以任何形式窥探直通外设的 MMIO 与 DMA 区域。因此，机密应用可直接以明文形态向安全设备（如直通硬件加速器）下发控制指令与 AI 模型参数，而无需承受额外的软件加解密性能损耗。

2. 半虚拟化设备的数据流：对于基于不可信共享内存构建的半虚拟化信道（如网络与磁盘 I/O），数据必须穿越不可信 Hypervisor 的控制边界。在此场景下，机密虚拟机必须利用预先供应的对称/非对称密钥，在软件栈顶层实施端到端的密码

学加解密与报文认证码校验，坚决拒绝接收及执行任何被 Hypervisor 篡改、重放或伪造的 I/O 载荷。

5.5 机密异构计算环境实现

EasyAda-trust 一个重要的目标是保障用户 AI 模型和敏感数据的机密性和完整性。该目标可以进一步细分为两个目标：

1. 保障用户运行环境的可信和隔离：为了保护用户 AI 模型，机密虚拟机仅能在可信的硬件环境上对用户模型进行解密和部署，并且由 EasyAda-trust 确保机密虚拟机运行环境与不可信组件的隔离。

2. 保障用户 I/O 数据的机密性和完整性：为了确保用户敏感数据不被窃取或篡改，EasyAda-trust 需要对用户 I/O 数据采取必要的加密和校验，并确保用户模型使用硬件加速器过程中的安全。

同时，EasyAda-trust 为了增强系统的可扩展性，需要实现硬件加速器在不同执行环境间的复用，为了实现以上目标，EasyAda-trust 实现了专用的硬件加速器管理机密虚拟机，并通过 I/O 半虚拟化技术实现硬件加速器在不同执行环境间的复用。下面将详细介绍 EasyAda-trust 如何实现机密异构计算环境。

5.5.1 硬件加速器的保护

为了确保机密虚拟机使用硬件加速器过程中的安全性，Hypervisor 必须显式将硬件加速器提升为安全设备，TSM 在加速器管理机密虚拟机启动时，会检查并尝试分配安全设备，只有经过验证的安全硬件加速器可以映射到机密虚拟机的地址空间中进行管理。同时，硬件加速器使用的内存缓冲区由加速器管理机密虚拟机内的驱动从当前机密虚拟机的安全内存中进行分配，不可信组件无法访问安全的内存缓冲区和 MMIO 区域。

5.5.2 加速器管理机密虚拟机初始化

在未引入机密虚拟机的架构下，EasyAda-Hypervisor 为了实现对现有驱动的复用，避免增大微内核操作系统的攻击面，会在启动阶段将硬件加速器的 MMIO 地址透传到硬件加速器管理虚拟机中，由该虚拟机加载驱动并完成硬件加速器初始化，后续通过 I/O 半虚拟化技术（如 VirtIO 协议）实现硬件加速器在虚拟机间以及宿主 OS 间的时分复用。

加速器管理机密虚拟机的创建与其他机密虚拟机类似，在创建加速器管理机密虚拟机前，Hypervisor 需要主动将可信的硬件加速器提升为安全硬件，因为提升时机由 EasyAda-Hypervisor 决定，应当由其自身确保无任何组件正在访问硬件加

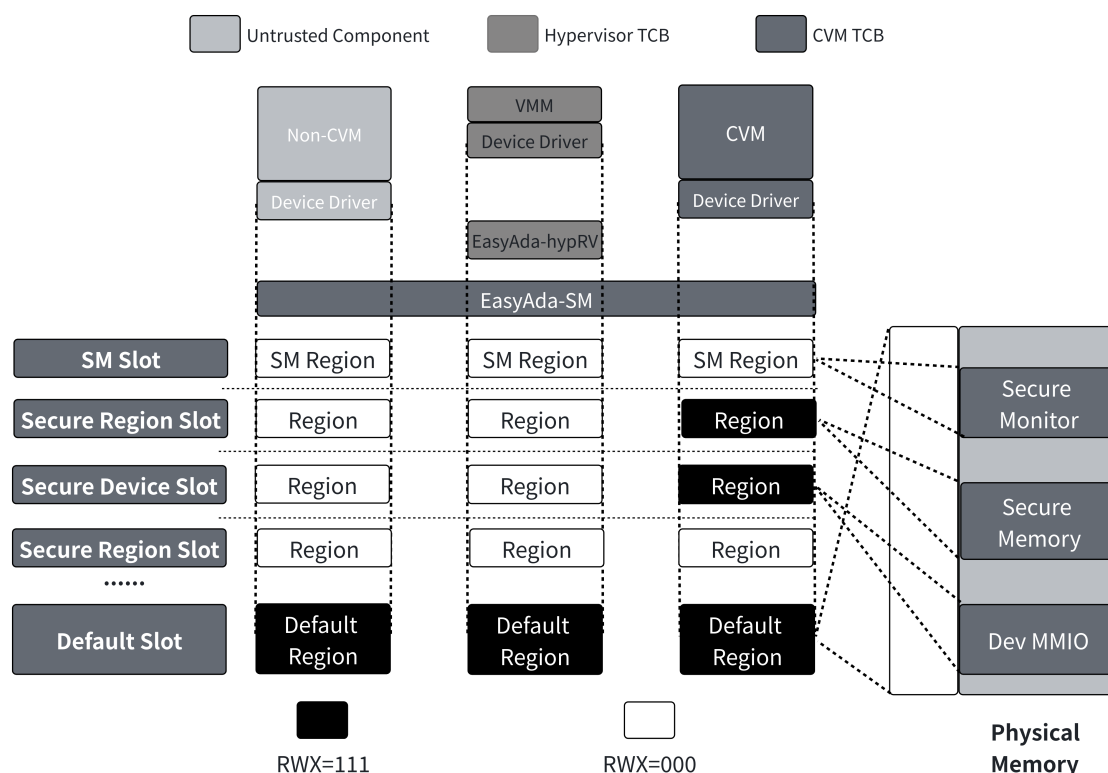


图 5-5 引入安全设备后的机密虚拟机安全内存视图

速器，避免访问流程的异常中断。当硬件加速器被提升为安全设备后，不可信组件无法访问硬件加速器的 MMIO 区域，

接下来由 Hypervisor 负责创建机密虚拟机，在 TSM 为机密虚拟机分配安全设备时，会同时为硬件加速器创建授权 MD 用于授予当前硬件加速器访问安全区域的能力，因为端侧硬件加速器多以协处理器的形式集成在 SoC 中，因此其数据缓冲区需要动态进行分配，为了降低 IOPMP 管理的开销，TSM 配置 IOPMP 授权硬件加速器访问加速器管理机密虚拟机关联的所有安全区域以确保可以正常访问动态数据缓冲区，因为配置 DMA 时会通过 IOMMU 或 TSM 软件翻译 GPA 为物理地址，避免了硬件加速器因为 DMA 配置错误写入其它安全内存中。隔离完成后，不可信组件无法访问硬件加速器的 MMIO 区域，硬件加速器也无法写入不可信的内存区域。

加速器管理机密虚拟机创建成功后，如图5-5所示，此时加速器管理机密虚拟机是除 TSM 外唯一可以访问硬件加速器软件实体。当加速器管理机密虚拟机实际运行时，需要按照安全启动流程，机密虚拟机从 TSM 获取开发者密钥并对开发者提供的 AI 模型和参数进行解密和部署，并对不可信的半虚拟化和全虚拟化设备进行初始化。

加速器管理机密虚拟机创建成功之后，可以直接对其上的机密应用提供服务。

如果需要与不可信环境间进行复用，则需要与请求使用硬件加速器的不可信组件建立信道，下面将介绍信道建立的过程。

5.5.3 建立信道

加速器管理机密虚拟机其它组件均通过 I/O 半虚拟化技术——VirtI/O 协议实现硬件加速器的时分复用。

虚拟设备的发现在开发是通过静态的硬件配置文件如 FDT 文件静态完成，EasyAda-trust 应用开发者需要根据硬件供应商提供的可信物理硬件配置完成设备静态分区，并在加速器管理机密虚拟机的硬件配置文件中预留内存区域用于建立映射共享内存。

加速器管理机密虚拟机会对设备进行初始化，当访问到预留的内存区域时会陷入 TSM，由 TSM 检查 MMIO 区域类型后，将共享内存申请请求转发到 EasyAda-Hypervisor 处理。EasyAda-Hypervisor 负责完成共享内存分配并返回地址给 TSM，由 TSM 完成共享内存到机密虚拟机地址空间的映射以及机密虚拟机上下文的恢复，并准备进行驱动后端初始化。同时，EasyAda-Hypervisor 会标记该共享内存为虚拟硬件加速器设备区域，等待映射到驱动前端中。

当不可信虚拟机尝试初始化虚拟硬件加速器时，会陷入到 EasyAda-Hypervisor 中，由 EasyAda-Hypervisor 将会获取之前加速器管理机密虚拟机申请的共享内存，并将其映射到请求方虚拟机对应的虚拟设备地址中。之后虚拟机和加速器管理机密虚拟机可以通过该共享内存完成数据交换，由 TSM 和 EasyAda-Hypervisor 转发和注入中断进行 I/O 请求通知和回复。EasyAda-trust 并不要求加速器管理机密虚拟机和请求方虚拟机的初始化时序，双方可以由任意一方进行初始化并完成共享内存的申请和标记，EasyAda-trust 不感知实际的进一步的协议交互和信道建立流程，通信双方可以基于共享内存，通过轮询或中断完成通信协议的初始化。

5.6 本章小结

本章详细论述了基于微内核的可信虚拟化方案的工程实现，重点阐述了如何适配 EasyAda-Hypervisor 以实现机密虚拟机的安全管理与资源虚拟化。在 CPU 虚拟化方面，方案通过扩展 EasyAda-Hypervisor 的 Hart 分区架构，引入了机密与非机密 vhart 运行区，并利用 TSM 接管机密虚拟机的两阶段安全上下文切换流程，确保了机密虚拟机的 CPU 状态对不可信 Hypervisor 完全隐匿。在内存与 I/O 虚拟化层面，方案将安全资源的直接控制权从 EasyAda-Hypervisor 中剥离，确立了由 Hypervisor 负责物理内存分配、设备发现与宏观生命周期调度，进而委托 TSM 完

成安全区域动态构建、二阶段页表隔离映射及安全设备鉴权配置的职责解耦机制。此外，针对端侧异构计算需求，方案利用 EasyAda-Hypervisor 辅助建立不可信共享内存通信信道，成功实现了专用硬件加速器管理机密虚拟机的安全部署与跨环境复用。综上所述，本章通过对 EasyAda-Hypervisor 实施职责解耦与非侵入式安全扩展，在充分复用微内核统一资源管理优势的同时，为机密虚拟机提供了一个高安全、高可靠的可信虚拟化运行底座。

第六章 系统测试与分析

6.1 测试目标

本章旨在对基于微内核的可信虚拟化架构 EasyAda-trust 及其核心组件 EasyAda-SM 进行系统性的定量评估与安全性验证。评测过程重点围绕以下两个核心科学问题展开：

1. 生态兼容性：采用 Rust 语言重构并引入安全区域抽象的 EasyAda-SM，是否能够以较小的性能与工程代价兼容现有的 RISC-V TEE 生态系统？
2. 基础性能开销：在消除不安全内存访问隐患并引入强隔离边界后，EasyAda-trust 架构在执行核心计算任务时引入的额外性能损耗是否被控制在合理的阈值范围内？

6.2 测试硬件环境

鉴于当前 RISC-V 硬件生态的客观局限，现阶段量产的商用芯片在安全特性支持上呈现出显著的路线分化：面向端侧的芯片普遍集成 PMP 与 IOPMP 扩展，但缺乏对 H 扩展的支持；而面向高性能计算的芯片虽支持 H 扩展，却通常采用复杂的 IOMMU 与高级中断架构（Advanced Interrupt Architecture, AIA），导致当前难以获取同时原生支持 H 扩展与 IOPMP 扩展的物理硬件平台。

基于上述硬件现状，本研究选用 QEMU（版本 9.2.4）Virt 平台进行硬件仿真。平台配置启用 RV64GCH 与 IOPMP 扩展，禁用 IOMMU 扩展。

6.3 EasyAda-SM 可扩展性测试

参考 EasyAdaZone 的实验设计^[49]，本文基于 EasyAda-SM 实现了兼容 Penglai 和 Keystone 的飞地管理模块，以此验证 EasyAda-SM 的架构可扩展性。

该测试在极小化代码侵入的前提下，除引导程序与安全监视器外，完全复用了 Keystone 与 Penglai 框架中的全量软件栈。复用范围涵盖宿主机用户态接口、编译工具链、软件依赖库、测试用例以及受保护侧的 TA，并以 Linux（内核版本 4.15）作为宿主操作系统。

性能评估采用 Coremark 与 RV8 benchmark 基准测试集，以原生 Linux 环境执行数据为基线，对比原生环境与 EasyAda-SM 环境下的 Keystone 与 Penglai 方案性能。测试配置中，原生 Linux 的 Coremark 版本为 v1.01，RV8 benchmark 基于主分支。依照 Keystone 与 Penglai 架构规范，测试程序被重构为 TA。原生与 EasyAda-SM

环境均执行相同 Coremark 与 RV8 benchmark TA 进行测试。

6.3.0.1 Coremark 测试套

如图6-1所示，所有测试结果均以原生 Linux 环境（Native Linux）的得分为基线（100%）进行归一化处理。实验数据表明，在引入 EasyAda-SM 后，系统的底层计算性能得到了最大程度的保留。在 Linux、Keystone 与 Penglai 三种工作负载下，与未引入安全防护的原生环境相比，EasyAda-SM 环境的相对性能衰减均被严格控制在 2% 以内。

鉴于 Coremark 属于典型的 CPU 密集型载荷，其执行路径高度集中于用户态及飞地内部。由于本文所提架构充分利用了 RISC-V 微架构级的 PMP 扩展，安全隔离校验在硬件流水线中隐式完成，且 Rust 语言的内存安全检查均发生在编译期。因此，在未触发跨特权级上下文切换的前提下，EasyAda-SM 不会对核心运算流水线引入任何显著的运行时开销。

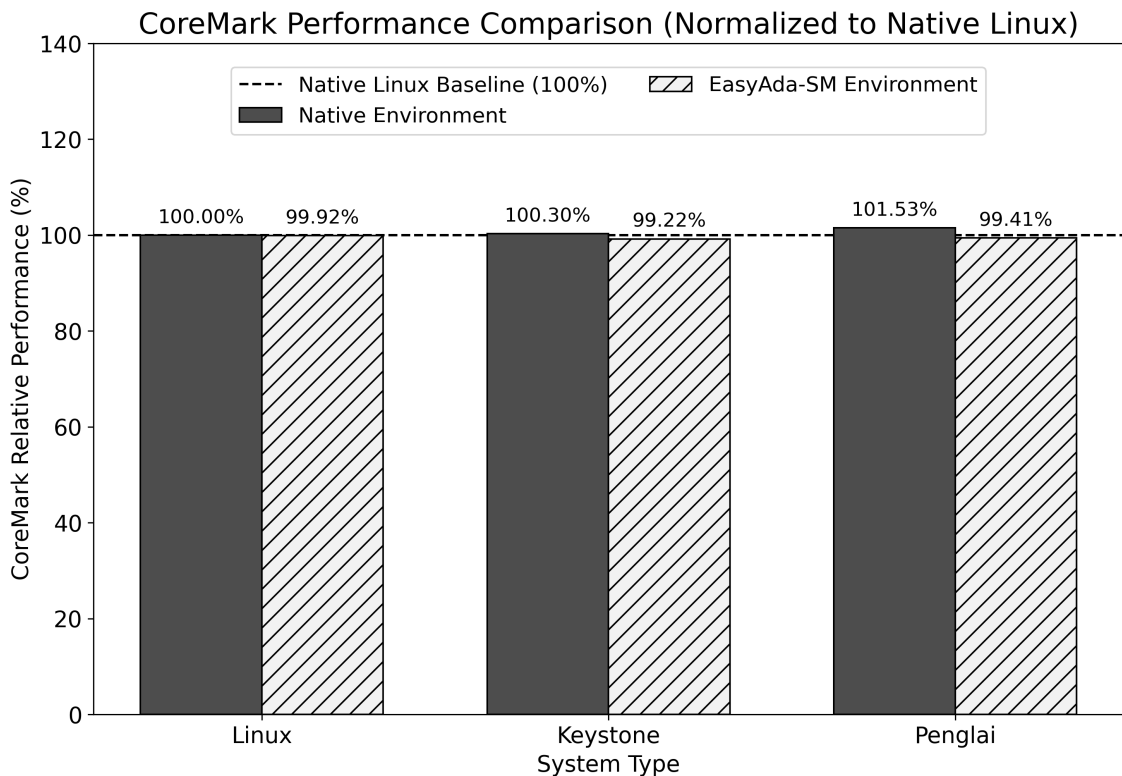


图 6-1 原生环境与 EasyAda-SM 环境下 Coremark 的对比测试

6.3.0.2 RV8 基准测试套

为进一步探究 EasyAda-SM 架构在面对具有复杂指令分布与高频访存行为的应用时的性能表现，本文引入了 RV8 benchmark 测试集。该测试集包含 AES（加

密)、Miniz (数据压缩)、Primes (素数计算)、Bigint (大数运算) 以及 Norx (认证加密) 等五项典型负载。所有测试数据均以原生 Linux 环境为归一化基准 (1.0)。

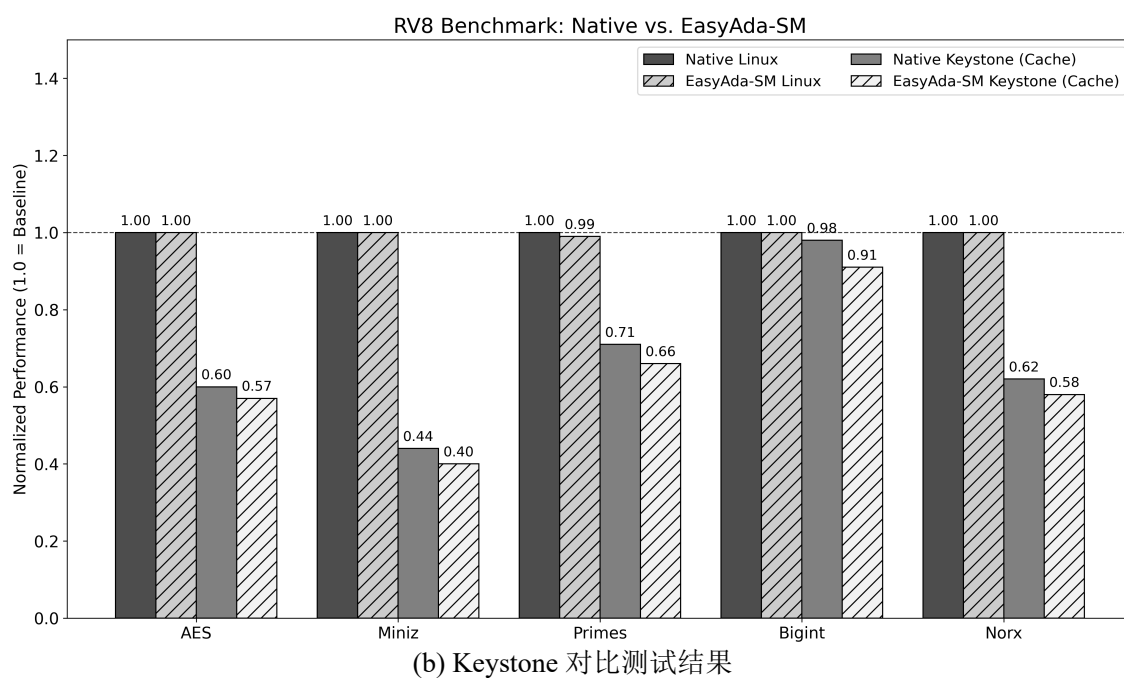
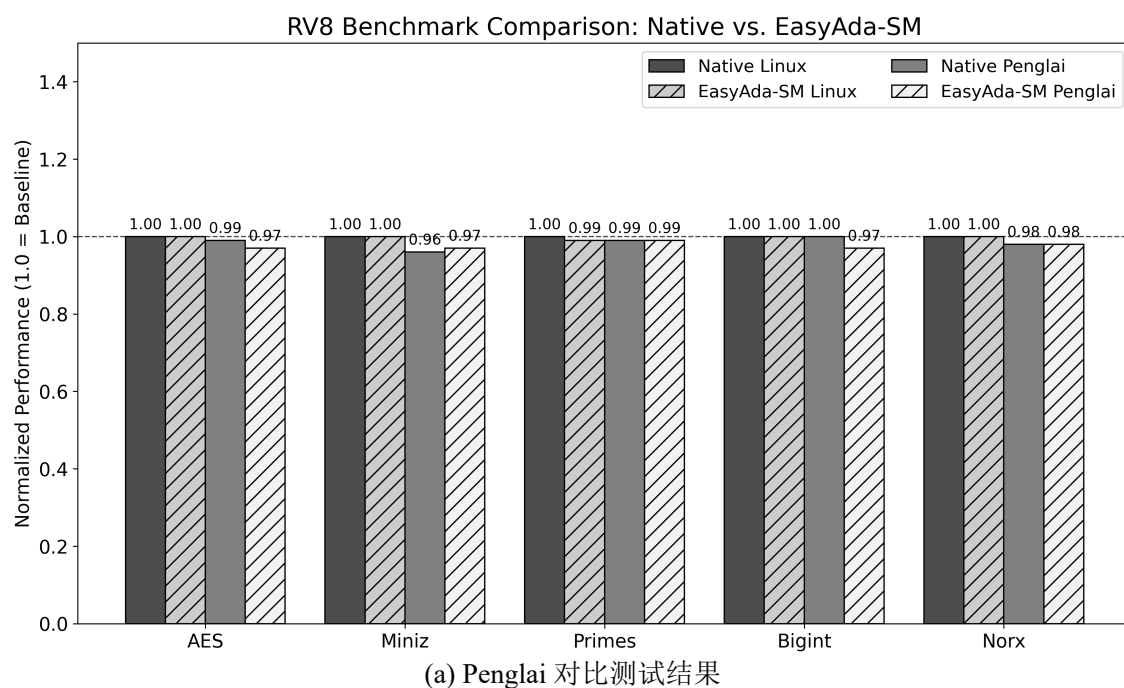


图 6-2 原生环境与 EasyAda-SM 环境下 RV8 benchmark 测试结果对比

如图6-2(a)所示, 在未显式开启微架构侧信道防御的 Penglai 轻量级飞地场景下, Penglai 在原生环境和 EasyAda-SM 环境中均展现出卓越的执行效率, 各项负载的归一化得分均不低于 0.96。以访存密集型的 Miniz 负载为例, 二者得分分别为 0.96 与 0.97。这充分证实了在引入基于 Rust 重构的 EasyAda-SM 及安全区域内存

抽象后，系统的额外性能损耗被严格限制在 1% 至 2% 的区间内，表明底层特权级调度对上层复杂业务逻辑的执行干扰较低。

然而，在适配 Keystone 框架的测试中，系统性能呈现出显著的差异化衰减：如图6-2(b)所示，Miniz 与 AES 的相对性能分别断崖式下跌至约 0.40 与 0.57，而计算密集型的 Bigint 受影响较小，分析可能是因为 Keystone 为防御缓存侧信道攻击，在物理层面启用了硬件缓存分区隔离机制，严格限制了飞地可用的末级缓存路数，导致高度依赖数据局部性的算法缓存缺失率剧增。在未启用缓存分区的环境中，横向对比原生环境和 EasyAda-SM 环境可知，EasyAda-SM 引入的绝对软件开销仅为 3% 至 7%，性能折损主要源于 EasyAda-SM 模块化分层抽象所导致的底层执行链路延长。

综合上述交叉验证与基准测试结果表明，本文提出的基于 Rust 构建的内存安全监视器 EasyAda-SM，在确立坚实底层信任根的同时，兼具高度的执行效率与架构弹性。具体而言，在纯计算密集型负载下，得益于 PMP 机制的硬件级隐式校验以及 Rust 语言的零成本抽象特性，该安全架构所引入的运行时额外开销趋近于零。

6.4 EasyAda-trust 测试

6.4.1 系统启动测试

系统的安全启动流程可严格划分为基础固件引导与可信安全监视器接管两个关键阶段。如图6-3所示，本系统将 RustSBI 作为静态库集成至 EasyAda-SM 中。在系统上电初期，执行流首先跳转至 RustSBI 以完成底层硬件环境的初始化配置。在移交控制权前，RustSBI 主动激活了前 8 组 PMP 条目，将其自身驻留的物理内存区域配置为无访问权限（NONE），从而在硬件层构建了防御低特权级代码篡改的基础安全边界。

当 RustSBI 完成底层硬件初始化后，系统控制权正式交接至 EasyAda-SM 以推进核心初始化逻辑。如图6-4所示，多 Hart 的初始化日志交替输出，表明系统已成功唤醒并构建了对称多处理（SMP）环境。鉴于 RISC-V 架构中各 Hart 均拥有独立的控制与状态寄存器，EasyAda-SM 确保了所有核心严格同步执行一致的安全策略配置，从而有效规避跨核越权访问等微架构级侧信道漏洞。

在全局内存隔离策略的构建方面，测试环境中的 Hart 均支持 16 组 PMP 条目。EasyAda-SM 利用最高优先级的条目划定安全监视器的根安全区域，通过写入配置值 0x18（即 NAPOT 匹配模式且权限为 NONE），对预留的 16MB 安全监视器专有物理内存实施强制硬件隔离；同时，利用最低优先级的条目设定背景策略，授予系统默认访问公共物理内存与 MMIO 区域的权限。此外，为实现对 RustSBI 的最小


```

[RustSBI] INFO - Hello RustSBI!
[RustSBI] INFO - RustSBI version 0.4.0
[RustSBI] INFO -
[RustSBI] INFO - | _ _ \ | | | | / | | | | / | | | | _ _ \ | | |
[RustSBI] INFO - | |_) | | | | | (---`---| |---`---| (---`---| |_) | | |
[RustSBI] INFO - | | / | | | | \ \ | | | | \ \ | | | | \ \ | |
[RustSBI] INFO - | | \ \----| | `---' |.----) | | | | .----) | | |_) | | |
[RustSBI] INFO - | | | \____| \____/ |____/ | | | |____/ |____/ | | |
[RustSBI] INFO - Initializing RustSBI machine-mode environment.
[RustSBI] INFO - Platform Name : riscv-virtio,qemu
[RustSBI] INFO - Platform HART Count : 4
[RustSBI] INFO - Enabled HARTs : [0, 1, 2, 3]
[RustSBI] INFO - Platform IPI Extension : SiFiveClint (Base Address: 0x2000000)
[RustSBI] INFO - Platform Console Extension : Uart16550U8 (Base Address: 0x10000000)
[RustSBI] INFO - Platform Reset Extension : Available (Base Address: 0x1000000)
[RustSBI] INFO - Platform HSM Extension : Available
[RustSBI] INFO - Platform RFence Extension : Available
[RustSBI] INFO - Platform SUSP Extension : Available
[RustSBI] INFO - Platform PMU Extension : Available
[RustSBI] INFO - Memory range : 0x80000000 - 0x180000000
[RustSBI] INFO - Platform Status : Platform initialization complete and ready.
[RustSBI] INFO - PMP Configuration
[RustSBI] INFO - PMP Range Permission Address
[RustSBI] INFO - 0 OFF NONE 0x0000000000000000
[RustSBI] INFO - 1 TOR RWX 0x0000000080000000
[RustSBI] INFO - 2 TOR RWX 0x0000000080000000
[RustSBI] INFO - 3 TOR R 0x0000000080027000
[RustSBI] INFO - 4 TOR NONE 0x0000000080036000
[RustSBI] INFO - 5 TOR RW 0x0000000080071000
[RustSBI] INFO - 6 TOR RWX 0x0000000180000000
[RustSBI] INFO - 7 TOR RWX 0xfffffffffffffc
[RustSBI] INFO - Boot HART ID : 3
[RustSBI] INFO - Boot HART Privileged Version: : Version1_12
[RustSBI] INFO - Boot HART MHPM Mask: : 0x07ffff
[RustSBI] INFO - The patched dtb is located at 0x8005c000 with length 0x2258.

```

图 6-3 SBI 固件初始化

化侵入式修改，系统保留了其独立的堆内存区；而前述 16MB 预留空间的主体则被接管为 EasyAda-SM 的全局堆内存。上述初始化序列执行完毕后，控制权将正式移交至 EasyAda-Hypervisor，从而开启上层虚拟化环境的引导流程。

6.4.2 机密虚拟机安全启动测试

在普通虚拟机动态提升为机密虚拟机（机密虚拟机）的过程中，系统必须完成机密虚拟机与 EasyAda-SM 的双向本地证明。本文模拟了以下三种边界场景以进行安全启动流程测试：（1）机密虚拟机与平台均可信；（2）机密虚拟机不可信，平台可信；（3）机密虚拟机可信，平台不可信。

图6-5展示了可信机密虚拟机在可信平台上启动的基准流程。在此时序中，EasyAda-SM 接管虚拟机提升请求后，即对机密虚拟机的内存布局、内核镜像及 FDT 执行完整性度量。日志显示，系统运行时计算得出的 PCR4 度量值与 TAP 中预置的参考值完全一致。本地证明成功后，安全监视器向机密虚拟机安全注入 TAP 中存储的机密数据，随后客户机 Linux 内核成功引导。该结果验证了系统在

```
#EasyAda-SM: Setting up hardware hart id=1
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
#EasyAda-SM: Setting up hardware hart id=0
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
#EasyAda-SM: Setting up hardware hart id=2
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
#EasyAda-SM: Setting up hardware hart id=3
#EasyAda-SM: Number of PMPs=16
#EasyAda-SM: pmp0 value: 40000000, shifted 100000000
#EasyAda-SM: pmp0 cfg: 11000
#EasyAda-SM: pmp15 value: 0, shifted ffffffff
#EasyAda-SM: pmp15 cfg: 11111
```

图 6-4 EasyAda-SM 初始化测试

理想状态下能够正确完成鉴权并启动合法的机密虚拟机。

[illegible]

图 6-5 可信机密虚拟机在可信平台上启动测试

图6-6展示了不可信机密虚拟机（例如内核镜像被恶意篡改）在可信平台上的启动拦截过程。当 EasyAda-SM 对篡改后的机密虚拟机进行度量时，生成的 PCR4 值与预期参考值发生显著偏移。本地证明机制捕捉到该完整性破坏并抛出异常。随后，安全监视器立即阻断引导流，拒绝为该实例分配执行权限，并将其从核心数据

备 EasyAda-SM 内存隔离保护的执行环境；（3）机密虚拟机：运行于 EasyAda-trust 架构之上，受 EasyAda-SM 全生命周期管控、内存强隔离及机密上下文切换保护的执行环境。

如图6-8所示，所有测试结果均以原生 Linux 环境的执行时延为基线进行归一化处理（数值越低代表操作时延越短、性能越优）。具体分析如下：

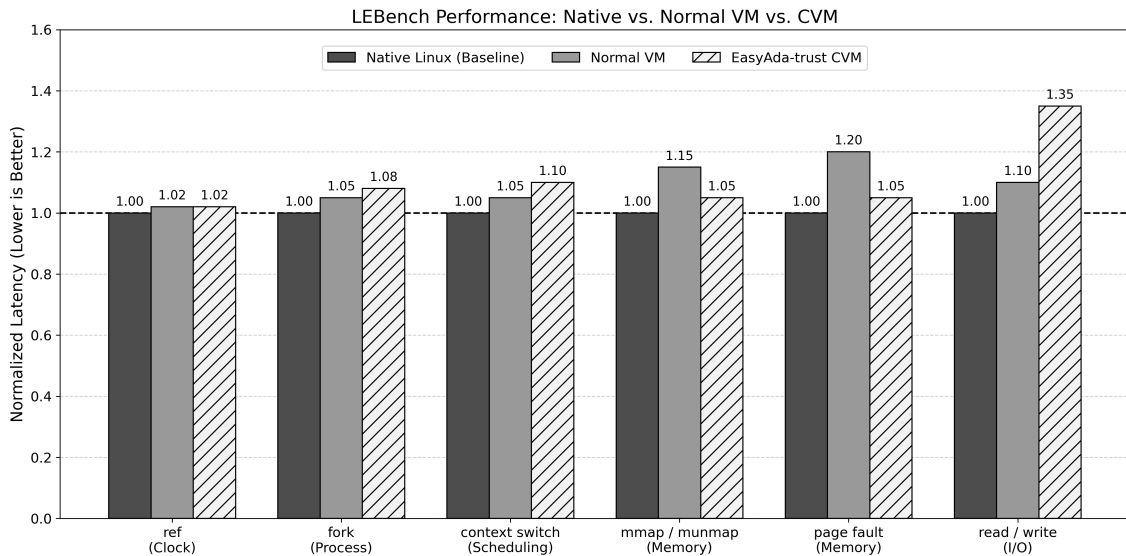


图 6-8 LEBench 测试结果

在读取系统时钟的测试（ref）中，普通虚拟机与机密虚拟机的归一化延迟几乎与物理裸机完全一致，归功于底层微架构对 S 态时钟中断（Sstc）扩展的完备支持。该扩展允许虚拟机直接在 VS 态配置定时器并接收虚拟中断，从硬件层面规避了传统架构中需要陷入 M 态再由软件注入中断的昂贵开销，契合了 TSM 将机密虚拟机时钟中断直接委派至 VS 态处理的架构设计。

在内存分配与缺页异常处理方面，普通虚拟机与机密虚拟机均表现出较高的执行效率。其核心原因在于，普通虚拟机与机密虚拟机的运行期物理内存已在实例启动阶段一次性预分配完毕，且二阶段页表在初始化阶段固化并不支持动态换页。因此，虚拟机在运行期触发的缺页异常完全收敛于客户系统内部闭环处理。除 MMU 固有的两阶段地址翻译硬件开销外，该机制并未在关键访存路径上引入任何额外的软件级运行时惩罚。

在进程调度层面，机密虚拟机的时延表现略高于普通虚拟机。LEBench 测量的核心操作是虚拟机内部（VS 态与 VU 态之间）的用户态进程切换。尽管常规的域内切换不会触发 TSM 陷入，但在涉及跨执行环境（如系统调用或外部中断）的上下文协作时，控制流必须陷入 TSM 以执行严苛的两阶段安全上下文保存、微架构残余状态清洗以及安全内存视图的动态重配。这些必须的安全原语导致了机密

虚拟机侧可预期的性能衰减。

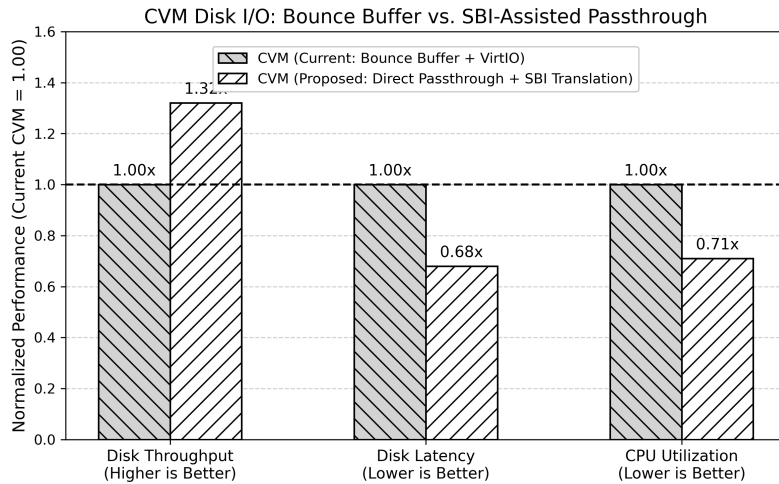


图 6-9 安全设备直通架构与半虚拟化机制（不可信共享内存）下的磁盘 I/O 对比测试结果

基础 I/O 测试结果深刻表明，底层设备路由架构对机密虚拟机的 I/O 效能具有决定性影响。在传统的全/半虚拟化架构中，为遵循严密的内存隔离安全模型，TSM 在物理层面阻断了不可信 Hypervisor 的直接 DMA 路径。这一限制迫使机密虚拟机不得不采用高开销的安全中转机制：I/O 数据流必须经历繁重的内部密码学加解密、向不可信共享缓冲区的冗余数据拷贝，以及由 I/O 事件高频触发的 TSM 安全域上下文切换。纯软件密码学计算、数据多重拷贝与微架构缓存失效的相互叠加，共同导致了该模式下 I/O 性能的显著衰减。

相比之下，图6-9清晰展示了安全设备直通架构与基于不可信共享内存的半虚拟化机制在磁盘 I/O 吞吐量上的对比测试结果。安全直通架构将受 PMP 保护的外设 MMIO 区域直接映射至机密虚拟机的二阶段地址空间，并协同 IOPMP 扩展在总线层级实施抗 DMA 攻击的物理内存鉴权。得益于此，机密应用得以合法旁路不可信的软件中转栈，在彻底免除高昂软件密码学开销的同时，实现了与受信异构硬件外设的零拷贝、明文高速通信。

6.5 防御链路分析

为了进一步验证本系统安全防护机制的有效性，并阐明防范 DMA 攻击机制及内存漏洞消除模块的作用机理，本节针对典型的边缘计算与机密虚拟化威胁模型，选取恶意设备提权与 DMA 攻击以及二次释放攻击两个特定攻击场景，进行详细的防御链路追踪分析。

6.5.1 恶意设备提权与 DMA 攻击

在云边协同场景中，不可信虚拟机或恶意 Hypervisor 遭到攻击者提权控制。攻击者试图利用分配给不可信环境的直通外设（如网卡或硬件加速器），通过伪造目的内存地址，发起未授权的 DMA 请求。其目的是绕过 CPU 侧的 MMU 与 PMP 隔离机制，非法读取或篡改机密虚拟机以及底层 EasyAda-SM 所在的受保护物理内存区域。

在系统冷启动和机密虚拟机实例化阶段，作为 EasyAda-SM 负责核心 IOPMP 规则的静态配置。系统根据硬件设备的拓扑结构，为该不可信外设分配唯一的 SID 并将其绑定至不可信 MD（仅授权访问通用内存区域）。同时，机密虚拟机的安全内存区域仅对安全 MD 开放读写权限。

恶意实体（如不可信虚拟机和 Hypervisor）构造恶意的 DMA 请求，命令不可信设备向机密虚拟机的安全内存区域发起 DMA 读写请求。不可信设备设在系统总线上发起内存事务，总线控制器将该事务及其绑定的 SID 路由至 IOPMP 硬件检查模块。

IOPMP 依据查表逻辑获取该 SID 绑定的 MD。因为该 MD 为不可信 MD，不具备对任何安全内存的访问权限，因此 IOPMP 硬件级拦截该访存事务，向处理器抛出访问违例异常，中止外设的 DMA 传输。

6.5.2 二次释放攻击

在传统的基于 C/C++ 开发的底层安全监视器（如 OpenSBI 的部分闭源扩展）中，攻击者常利用内存管理模块中的释放后重用或二次释放等内存破坏漏洞，篡改堆内存中的关键元数据，从而劫持底层固件的控制流。本节以 EasyAda-SM 的页分配器为例介绍 EasyAda-SM 如何通过 Rust 防御不安全的内存访问。

EasyAda-SM 采用 Rust 语言重构底层的内存管理原语。在页分配器模块中，物理内存页的生命周期被抽象为严格的类型状态机，空闲页和已分配页被分别定义为互不兼容的数据结构类型和。

EasyAda-SM 采用 Rust 语言重构底层的内存管理原语。在页分配器中，物理内存页的生命周期被抽象为严格的类型状态机，利用静态标记将未分配页和已分配页定义为编译期互不兼容的数据结构。

当页面状态发生转换（如分配和释放时），核心接口均采用传值语义而非引用传参，因此每一次状态转移都会强制消耗当前页令牌的所有权，并返回带有新类型状态的实例，在逻辑上实现了物理页生命周期的单向移动。

基于上述机制，若存在逻辑缺陷或恶意控制流试图对同一个已被释放的物理

页执行二次释放，或是尝试继续解引用已归还给分配器的悬垂页令牌，均会触发对已转移所有权的内存对象发起二次访问。**Rust** 的借用检查器会在静态编译期捕获此类非法行为，并抛出异常，避免生成包含该漏洞的可执行文件。

6.6 本章小结

本章对所提出的 **EasyAda-trust** 微内核可信虚拟化架构及其核心固件 **EasyAda-SM** 进行了系统性的定量评估与安全性验证。首先，通过 **Coremark** 与 **RV8** 基准测试证明了引入 **Rust** 内存安全机制后的 **EasyAda-SM** 具备极高的生态兼容性，且在核心计算场景下引入的额外运行时开销趋近于零。其次，验证了系统安全启动与双向本地证明机制的有效性，结果表明该机制能精准识别并拦截受损的虚拟机镜像与不可信的底层运行环境，确保了信任链的完整性。

此外，通过 **LEBench** 微基准测试与网络 **I/O** 测试对机密虚拟机的全生命周期开销进行了深度剖析。数据表明，得益于静态安全内存预分配策略与 **Sstc** 硬件扩展的支持，机密虚拟机在系统时钟与内存管理等核心原语上实现了与原生实现相同的性能。而在面临物理隔离边界导致的 **I/O** 性能衰减时，本文提出的基于 **IOPMP** 扩展的安全设备直通架构，成功绕过了高昂的软件密码学计算与不可信内存中转栈，在保障系统隔离强度的同时，实现了异构外设的明文访问。综合而言，本系统在内存安全保障与总体执行效能之间取得了较理想的架构折中。

第七章 总结和展望

7.1 全文总结

虚拟化技术与可信执行环境作为优化资源利用、保障系统安全的核心技术手段，在物联网与端侧异构计算领域扮演着愈加重要的角色。机密虚拟机（机密虚拟机）作为现代 TEE 的演进方向，直接关系到用户敏感数据与 AI 模型资产的机密性、完整性及系统实时性。当前，基于 x86 和 ARM 架构的机密计算方案已广泛部署，但面向开源 RISC-V 架构的微内核机密虚拟化研究仍处于起步阶段，且传统 C 语言构建的底层固件面临严峻的内存安全威胁，因此该方向具有重大的学术与工程研究价值。

本文围绕基于 RISC-V 架构的机密虚拟化方案实现与底层安全隔离机制展开系统性研究。基于 RISC-V 硬件虚拟化扩展（H 扩展）、物理内存保护机制（PMP/IOPMP）以及自研微内核操作系统 EasyAda 及微内核虚拟化架构 EasyAda-Hypervisor，设计并实现了基于微内核虚拟化架构的机密虚拟化架构 EasyAda-trust 及内存安全固件 EasyAda-SM，并在 QEMU 仿真环境及多种基准测试套件下对其核心性质与性能开展了系统性评估。总体而言，本文主要贡献可概括为以下几点：

1. 系统阐述了微内核虚拟化、TEE 架构演进及其在异构计算环境下面临的安全挑战。在端侧与物联网系统中，硬件算力复用与 AI 数据隐私保护需求日益凸显。本文分析了机密虚拟机在实际系统中的关键作用，并指出引入 Rust 内存安全语言与 RISC-V 开放架构对于削减可信基、消除内存漏洞及降低 TEE 准入门槛的重要意义。

2. 针对底层安全监视器易受内存破坏漏洞威胁的痛点，本文提出并使用 Rust 语言从零构建了内存安全固件 EasyAda-SM。该方案创新性地结合 Rust 的所有权机制设计了页分配器（Page Allocator），从编译期彻底杜绝了内存管理元数据被破坏的可能。同时，通过模块化分层设计，抽象出统一的物理内存保护原语与安全区域管理层，实现了不同类型飞地与不可信组件间的物理级强隔离。

3. 针对机密虚拟机全生命周期管理与异构安全扩展的需求，本文在微内核虚拟化方案 EasyAda-Hypervisor 的基础上设计实现了 EasyAda-trust 架构。通过将物理资源调度与安全内存映射鉴权进行深度解耦，实现了安全的 CPU 机密流上下文切换与二阶段页表隔离。在此基础上，本文结合 IOPMP 技术构建了防御恶意 DMA 攻击的总线屏障，并创新性地设计了加速器管理机密虚拟机，利用 I/O 半虚拟化技术实现了底层硬件加速器在不可信与机密环境间的安全时分复用。

4. 本文在 QEMU 环境中对 EasyAda-trust 及其核心组件开展了多维度的定量评估与功能验证。测试结果表明, EasyAda-SM 具备极高的生态兼容性, 在核心计算负载(如 Coremark 与 RV8 benchmark)下引入的性能损耗不到 2%; 在安全机制验证方面, 本文构建的基于本地证明的双向验证测试, 成功验证了 EasyAda-trust 架构在机密虚拟机不可信或平台不可信等多种边界情况下的防御拦截能力。

通过上述研究工作, 本文提供了一种将微内核虚拟化技术与 Rust 内存安全固件深度融合的 RISC-V 机密计算解决方案。与已有的 Keystone 或 Penglai 等纯应用级/系统级 TEE 框架相比, 本文聚焦于支持机密虚拟机抽象的虚拟机级强隔离, 并在异构外设的机密复用维度提出了完整的软硬协同范式, 为基于 RISC-V 架构的下一代端侧机密计算底座提供了一种高可靠、高可扩展的工程实现框架。

7.2 典型场景：基于机密虚拟机的自动驾驶混合架构

智能网联汽车属于 EasyAda-trust 目标场景, 其系统约束具备以下特征:

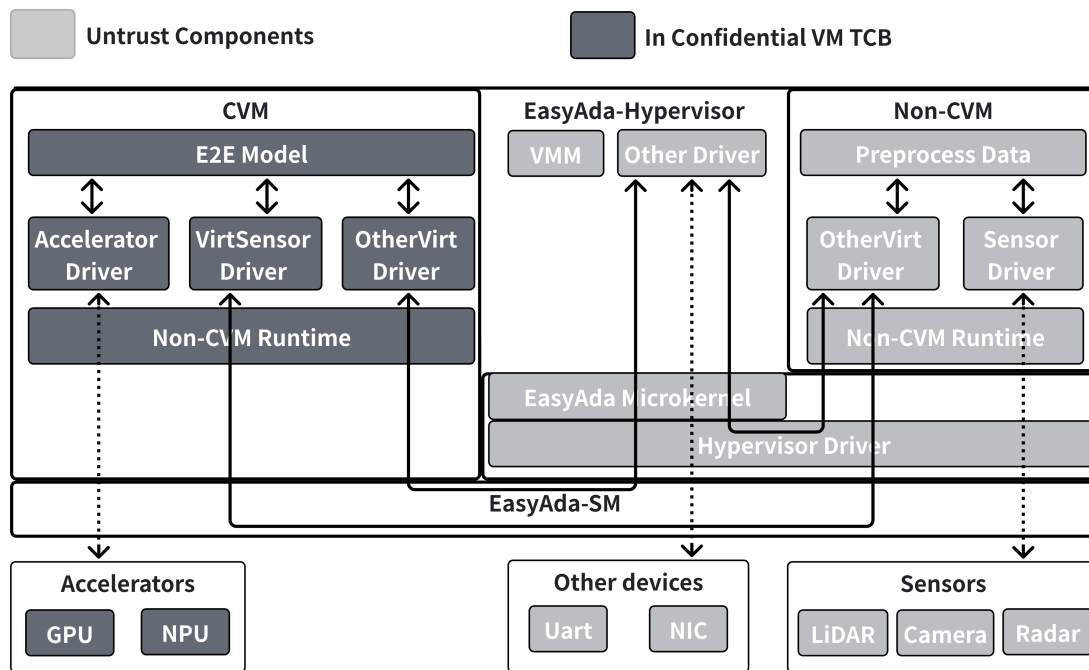


图 7-1 基于 EasyAda-trust 的自动驾驶混合架构

1. 系统架构复杂度：自动驾驶与智能座舱系统依赖异构硬件平台及多层通用软件栈, 全栈重构面临工程开发成本与技术周期限制。
2. 数据机密性与资产属性：自动驾驶运行期生成的决策数据、模型参数及用户数据具备高价值资产属性, 亟需在硬件环境中构建运行期隔离域。
3. 攻击面暴露度：车路云生态的扩展引入了多元化远程通信接口, 导致越权

访问与代码注入等安全威胁的潜在接入节点增多。

针对上述系统约束，基于 EasyAda-trust 的实现方案如图7-1所示，具体架构设计逻辑阐述如下：

系统在逻辑上划分为不可信组件域与可信域。不可信组件域由不可信虚拟机与 EasyAda-Hypervisor 构成。为了降低机密虚拟机压力并缩小 TCB，同时维持对现有软件栈的兼容性，激光雷达、车载相机及毫米波雷达等物理传感器的硬件驱动。和原始数据的采集与预处理任务由机密虚拟机卸载到不可信虚拟机中。

系统核心的 TCB 由机密虚拟机与 EasyAda-SM 组成。机密虚拟机内部署核心资产——端到端决策模型，模型通常在云端完成高算力训练，通过加密的云边协同通道下发至端侧，由端侧的 EasyAda-SM 依托本地证明机制解密并部署于机密虚拟机中，实现云端训练-端侧安全部署的云边端协同架构。同时，机密虚拟机的加速器驱动直接绑定图形处理器与神经网络处理器等硬件加速器。跨安全域的数据传输由 EasyAda-SM 接管，输出的预处理数据经由不可信共享内存实现的虚拟设备路由至机密虚拟机的虚拟传感器驱动。EasyAda-SM 在底层实施物理内存管理与外设访问控制策略，阻断不可信虚拟机和微内核对安全内存和硬件加速器的读写请求，保障模型资产与运行时数据的机密性。

网卡和串口等通用外设资源由 EasyAda-Hypervisor 统一调度和管理。微内核执行进程调度与进程间通信机制，承载时间敏感型车辆控制任务。EasyAda-SM 在系统底层维持隔离边界，限制微内核与 Hypervisor 的硬件访问特权，从而收敛整车控制系统的攻击面，并通过微内核的确定性调度算法保障车辆控制指令的实时响应。

面对云端远程升级等导致的并发机密虚拟机生命周期管理，EasyAda-trust 实现了管理平面与安全平面的职责解耦：机密虚拟机的粗粒度物理资源预分配由不可信的 EasyAda-Hypervisor 通过静态分区完成，而 EasyAda-SM 仅按需介入执行轻量级的安全鉴权、页表映射与上下文清洗等。这种协同调度机制有效避免了安全监视器因高频启停请求产生单点拥塞。此外，结合底层安全硬件的透传直通机制，机密虚拟机内的高负载计算任务基本独立于底层固件并且免受不可信 Hypervisor 的调度干扰，从而在复杂的边缘服务化部署中依然能提供可靠的实时响应性能。

7.3 后续工作展望

尽管本文在 RISC-V 微内核机密虚拟化及安全隔离机制上取得了一定成果，但面向未来复杂系统的演进，仍有以下四个方向亟待研究与改进：

1. 多核机密并发调度与中断优化：当前跨环境上下文切换策略较为保守。未

来将结合 RISC-V 高级中断架构，探索细粒度上下文感知与懒加载策略，优化核间中断与共享资源的并发控制，提升高并发负载下的吞吐量与实时响应能力。

2. 真实硬件部署与直通开销优化：受限于硬件生态，本文评估主要基于 QEMU。未来拟将系统移植至高阶 RISC-V 物理开发板进行全面压测。同时，针对 IOPMP 高频总线鉴权引入的 DMA 瓶颈，将探索基于硬件表项缓存或动态规则折叠的软硬协同算法，以降低物理访存损耗。

3. 核心可信组件的形式化验证：尽管 Rust 从编译期消除了内存漏洞，但 TSM 的逻辑正确性仍需理论证明。未来将引入 Isabelle/HOL 等证明工具，对 TSM 的异常嵌套处理、二阶段页表映射等开展形式化验证，并扩展无干涉性证明，从理论层面夯实信任根。

4. 异构计算生态与端到端总线安全：面向未来基于 PCIe/CXL 互连的片外独立加速器，现有机制难以防御总线窥探与重放攻击。未来将研究集成机密计算总线协议与内存加密引擎构建硬件级端到端加密链路，并扩展加速器机密虚拟机驱动生态，以原生支撑端侧大语言模型的机密推理。

致 谢

参考文献

- [1] OpenAMP. Welcome to the openamp project documentations[EB/OL]. 2026-01-11, <https://openamp.readthedocs.io/en/latest/index.html>.
- [2] Yan J, Chen Q, Zhou S, et al. Ekc: a portable and extensible kernel compartment for de-privileging commodity os[C]. Proceedings of the 34th USENIX Conference on Security Symposium, USENIX Association.
- [3] ARM. Arm security technology building a secure system using trustzone technology[EB/OL]. 2009-4-1, <https://developer.arm.com/documentation/PRD29-GENC-009492/latest>.
- [4] ARM. Intel® architecture instruction set extensions programming reference[EB/OL]. 2025-11-12, <https://www.intel.com/content/www/us/en/content-details/869288/intel-architecture-instruction-set-extensions-programming-reference.html>.
- [5] AMD. Using sev with amd epyc™ processors[EB/OL]. 2023-10-03, <https://docs.amd.com/v/u/en-US/58207-using-sev-with-amd-epyc-processors>.
- [6] Feng E, Lu X, Du D, et al. Scalable memory protection in the PENGDAI enclave[C]. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), 2021: 275–294.
- [7] Lee D, Kohlbrenner D, Shinde S, et al. Keystone: an open framework for architecting trusted execution environments[C]. Proceedings of the Fifteenth European Conference on Computer Systems, 2020: 1-16.
- [8] GlobalPlatform. Tee system architecture v1.3[EB/OL]. 2022-05-01, <https://globalplatform.org/specs-library/tee-system-architecture/#>.
- [9] Limited A. Learn the architecture - introducing arm confidential compute architecture version 4.0[EB/OL]. 2025-03-19, <https://developer.arm.com/documentation/den0125/latest>.
- [10] AMD. Intel® trust domain extensions[EB/OL]. 2025-06-01, <https://cdrdv2.intel.com/v1/dl/getContent/690419>.
- [11] Cerdeira D, Santos N, Fonseca P, et al. Sok: Understanding the prevailing security vulnerabilities in trustzone-assisted tee systems[C]. 2020 IEEE Symposium on Security and Privacy (SP), 2020: 1416-1432.
- [12] Matsakis N D, Klock F S. The rust language[C]. Association for Computing Machinery, 2014: 103–104.

- [13] Wang H, Wang P, Ding Y, et al. Towards memory safe enclave programming with rust-sgx[C]. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019: 2333–2350.
- [14] Wan S, Sun M, Sun K, et al. Rustee: Developing memory-safe arm trustzone applications[C]. Proceedings of the 36th Annual Computer Security Applications Conference, 2020: 442–453.
- [15] Popek G J, Goldberg R P. Formal requirements for virtualizable third generation architectures[J]. Commun. ACM, 1974, 17(7): 412–421.
- [16] Sahoo J, Mohapatra S, Lath R. Virtualization: A survey on concepts, taxonomy and associated security issues[C]. 2010 Second International Conference on Computer and Network Technology, 2010: 222–226.
- [17] Qumranet A, Qumranet Y, Qumranet D, et al. Kvm: The linux virtual machine monitor[J]. Proceedings Linux Symposium, 2007, 15.
- [18] Barham P, Dragovic B, Fraser K, et al. Xen and the art of virtualization[J]. SIGOPS Oper. Syst. Rev., 2003, 37(5): 164–177.
- [19] Dall C, Li S W, Nieh J. Optimizing the design and implementation of the linux ARM hypervisor[C]. 2017 USENIX Annual Technical Conference (USENIX ATC 17), 2017: 221–233.
- [20] Hwang J Y, Suh S B, Heo S K, et al. Xen on arm: System virtualization using xen hypervisor for arm-based secure mobile phones[C]. 2008 5th IEEE Consumer Communications and Networking Conference, 2008: 257–261.
- [21] Martins J, Tavares A, Solieri M, et al. Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems[C]. Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020), 2020: 3:1–3:14.
- [22] Li H, Xu X, Ren J, et al. Acrn: a big little hypervisor for iot development[C]. Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, 2019: 31–44.
- [23] Trujillo S, Crespo A, Alonso A. Multipartes: Multicore virtualization for mixed-criticality systems[C]. 2013 Euromicro Conference on Digital System Design, 2013: 260–265.
- [24] Matos E d, Ahvenjärvi M. sel4 microkernel for virtualization use-cases: Potential directions towards a standard vmm[J]. Electronics, 2022, (24).
- [25] Limited B. Qnx hypervisor 8.0 empowering embedded systems with trusted virtualization and mixed-criticality support[EB/OL]. 2025-10-01, <https://qnx.software/content/dam/qnx-xwalk/pdf/product-briefs/qnx-hypervisor-8-product-brief.pdf>.
- [26] Steinberg U, Kauer B. Nova: a microhypervisor-based secure virtualization architecture[C]. Proceedings of the 5th European Conference on Computer Systems, 2010: 209–222.

- [27] Jia Y, Liu S, Wang W, et al. HyperEnclave: An open and cross-platform trusted execution environment[C]. 2022 USENIX Annual Technical Conference (USENIX ATC 22), 2022: 437–454.
- [28] Pereira S, Sousa J, Pinto S, et al. Bao-enclave: Virtualization-based enclaves for arm[C]. 2022 IEEE 8th World Forum on Internet of Things (WF-IoT), 2022: 1–6.
- [29] Brasser F, Gens D, Jauernig P, et al. Sanctuary: Arming trustzone with user-space enclaves[C]. 2019: .
- [30] Wang J, Li A, Li H, et al. Rt-tee: Real-time system availability for cyber-physical systems using arm trustzone[C]. 2022 IEEE Symposium on Security and Privacy (SP), 2022: 352–369.
- [31] Feng E, Feng D, Du D, et al. siopmp: Scalable and efficient i/o protection for tees[C]. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, 2024: 1061–1076.
- [32] Du D, Yang B, Xia Y, et al. Accelerating extra dimensional page walks for confidential computing[C]. Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, 2023: 654–669.
- [33] Ozga W. Towards a formally verified security monitor for vm-based confidential computing[C]. Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy, 2023: 73–81.
- [34] AMD. Amd sev-snp: Strengthening vm isolation with integrity protection and more[EB/OL]. 2020-01-01, <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more>.
- [35] Sahita R, Shanbhogue V, Bresticker A, et al. Cove: Towards confidential computing on risc-v platforms[C]. Proceedings of the 20th ACM International Conference on Computing Frontiers, 2023: 315–321.
- [36] Wu X, Tian D J, Kim C H. Building gpu tees using cpu secure enclaves with gevisor[C]. Proceedings of the 2023 ACM Symposium on Cloud Computing, 2023: 249–264.
- [37] Deng Y, Wang C, Yu S, et al. Strongbox: A gpu tee on arm endpoints[C]. Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, 2022: 769–783.
- [38] Fan S, Hua Z, Xia Y, et al. Xputee: A high-performance and practical heterogeneous trusted execution environment for gpus[J]. ACM Trans. Comput. Syst., 2025, 43(1–2).
- [39] Wang C, Zhang F, Deng Y, et al. Cage: Complementing arm cca with gpu extensions[C]. Proceedings of the 31st Annual Network and Distributed System Security Symposium.
- [40] Wang C, Tang D, Ci C, et al. ccAI: A Compatible and Confidential System for AI Computing[M], Association for Computing Machinery, 2025, 340–353.

-
- [41] Lu H, Deng Y, Mertoguno S, et al. Mole: Breaking gpu tee with gpu-embedded mcu[C]. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, 2025: 693–707.
- [42] International R V. The risc-v instruction set manual: Volume ii[EB/OL]. 2025-05-08, https://docs.riscv.org/reference/isa/_attachments/riscv-privileged.pdf.
- [43] Li X, Zhao B, Yang G, et al. A survey of secure computation using trusted execution environments[J]. arXiv preprint arXiv:2302.12150, 2023.
- [44] Consortium C C. Confidential computing – the next frontier in data security[EB/OL]. 2021-10-19, https://confidentialcomputing.io/wp-content/uploads/sites/10/2023/03/Everest_Group_-_Confidential_Computing_-_The_Next_Frontier_in_Data_Security_-_2021-10-19.pdf.
- [45] Wang X, Shen W, Bu Y, et al. Dmaauth: a lightweight pointer integrity-based secure architecture to defeat dma attacks[C]. Proceedings of the 33rd USENIX Conference on Security Symposium.
- [46] Hunt G D H, Pai R, Le M V, et al. Confidential computing for openpower[C]. Proceedings of the Sixteenth European Conference on Computer Systems, 2021: 294–310.
- [47] Lee J, Wang Y, Rajat R, et al. Characterization of gpu tee overheads in distributed data parallel ml training[J]. arXiv preprint arXiv:2501.11771, 2025.
- [48] RustSBI. rustsbi[EB/OL]. 2026-02-10, <https://github.com/rustsbi/rustsbi>.
- [49] Jinzhe L, Kun X, Lei L, et al. Easy adazone: A memory-safe physical memory isolation framework for risc-v[C]. 2025 22nd International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP), 2025: 1-4.
- [50] Ren X J, Rodrigues K, Chen L, et al. An analysis of performance evolution of linux’s core operations[C]. Proceedings of the 27th ACM Symposium on Operating Systems Principles, Association for Computing Machinery, SOSP ’19, 2019: 554–569.

附录 A 核心伪代码

A.1 页分配器

A.2 IOPMP 动态配置

攻读硕士学位期间取得的成果

- [1] Easy adazone: A memory-safe physical memory isolation framework for risc-v[C].
2025 22nd International Computer Conference on Wavelet Active Media Technology
and Information Processing (ICCWAMTIP), 2025: 1-4.